



PROGRAMACIÓN

<https://martinezpenya.es/1DAMProgramacion/>
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1º Programacion (CFGs Desarrollo de Aplicaciones Multiplataforma)

IES Eduardo Primo Marques (Carlet)

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Índice de contenidos

1. UD00	9
1.1  Información importante	9
1.1.1  Denominación del curso	9
1.1.2  Contenidos	9
1.1.3  Resultados de Aprendizaje (RA)	10
1.1.4  Legislación vigente	17
1.1.5  Evaluación	17
2. UD01	18
2.1 Elementos de un programa informático	18
2.1.1 Piensa como un programador	18
2.1.2 Problemas, algoritmos y programas	22
2.1.3 Java	28
2.1.4 Componentes del lenguaje Java	32
2.1.5 Herramientas útiles para empezar	40
2.1.6 Ejemplos UD01	40
2.1.7 Resumen — Conceptos clave	44
2.1.8 Autoevaluación	45
2.1.9 Vídeos recomendados	45
2.2 Ejercicios de la UD01	46
2.2.1 Retos	46
2.2.2 Ejercicios	46
2.2.3 Expresiones Lógicas	49
2.2.4 Actividades	50
2.3 Talleres	52
2.3.1 Taller UD01_01: Instalar NoMachine para el control remoto	52
2.3.2 Taller UD01_02: Instalación y uso de entornos de desarrollo	53
2.3.3 Taller UD01_03: Crear cuenta en GitHub	78
2.3.4 Taller UD01_T03: Markdown	85
3. UD02	95
3.1 Utilización de Objetos y Clases	95
3.1.1 Introducción a la POO	95
3.1.2 Características de la POO	96
3.1.3 Objetos y Clases	96
3.1.4 Utilización de Objetos	99
3.1.5 Utilización de Métodos	103

3.1.6 Librerías de Objetos (Paquetes)	106
3.1.7 Cadenas de caracteres. La clase string	107
3.1.8 Ejemplos UD02	109
3.1.9 Resumen — Conceptos clave	112
3.1.10 Autoevaluación	113
3.1.11 Vídeos recomendados	113
3.2 Ejercicios de la UD02	114
3.2.1 Actividades	114
3.3 Talleres	0
3.3.1 Taller UD02_01: GitHub Classroom	0
4. UD03	0
4.1 Estructuras de control y Excepciones	0
4.1.1 Introducción	0
4.1.2 Sentencias y bloques	0
4.1.3 Estructuras de selección	0
4.1.4 Estructuras de repetición	0
4.1.5 Estructuras de salto	0
4.1.6 Excepciones	0
4.1.7 Aserciones (Assertions)	0
4.1.8 Ejemplos UD03	0
4.1.9 Resumen — Conceptos clave	0
4.1.10 Autoevaluación	0
4.1.11 Vídeos recomendados	0
4.2 Ejercicios de la UD03	0
4.2.1 Retos	0
4.2.2 Ejercicios	0
4.2.3 Actividades	0
4.3 Talleres	0
4.3.1 Taller UD03_01: GitHub Classroom	0
4.3.2 Taller UD03_02: Acepta el reto	0
5. UD04	0
5.1 Estructuras de datos: Arrays y matrices. Recursividad.	0
5.1.1 Introducción	0
5.1.2 Arrays	0
5.1.3 Problemas de recorrido, búsqueda y ordenación	0
5.1.4 Arrays bidimensionales: matrices	0
5.1.5 Arrays multidimensionales	0
5.1.6 Recursividad	0

5.1.7 Ejemplos UD04	0
5.1.8 Resumen — Conceptos clave	0
5.1.9 Autoevaluación	0
5.1.10 Vídeos recomendados	0
5.2 Anexo Cheatsheet Strings en Java	0
5.2.1 Introducción	0
5.2.2 Construyendo string	0
5.2.3 Operando con Métodos de la clase string	0
5.2.4 Ejemplo de todos los métodos de string	0
5.2.5 Arrays de String	0
5.2.6 Los String son inmutables	0
5.2.7 string en Argumentos de Línea de Comandos	0
5.2.8 Concatenar cadenas en Java	0
5.2.9 Formato de salida: printf y String.format	0
5.3 Ejercicios de la UD04	0
5.3.1 Arrays. Ejercicios de recorrido	0
5.3.2 Arrays. Ejercicios de búsqueda	0
5.3.3 Matrices	0
5.3.4 Recursividad	0
5.4 Talleres	0
5.4.1 Taller UD04_01: GitHub Classroom	0
6. UD05	0
6.1 Desarrollo de clases	0
6.1.1 Introducción	0
6.1.2 Estructura y miembros de una clase	0
6.1.3 Atributos	0
6.1.4 Métodos	0
6.1.5 Encapsulación, control de acceso y visibilidad.	0
6.1.6 Utilización de los métodos y atributos de una clase	0
6.1.7 Constructores	0
6.1.8 Clases Anidadas, Clases Internas (<i>Inner Class</i>) [NUEVO]	0
6.1.9 Introducción a la herencia. [NUEVO]	0
6.1.10 Conversión entre objetos (Casting) [NUEVO]	0
6.1.11 Acceso a métodos de la superclase [NUEVO]	0
6.1.12 Empaquetado de clases [NUEVO]	0
6.1.13 Ejercicios resueltos	0
6.1.14 Ejemplos UD05	0
6.1.15 Resumen — Conceptos clave	0

6.1.16 Autoevaluación	0
6.1.17 Vídeos recomendados	0
6.2 Anexo Wrappers	0
6.2.1 Wrappers (Envoltorios)	0
6.2.2 Ejemplo Anexo UD05	0
6.3 Anexo Fechas	0
6.3.1 Clase Date	0
6.3.2 Ejemplo Anexo UD05	0
6.4 Ejercicios de la UD05	0
6.4.1 Ejercicios	0
6.4.2 Actividades	0
6.5 Talleres	0
6.5.1 Taller UD05_01: GitHub Classroom	0
7. UD06	0
7.1 Lectura y escritura de información	0
7.1.1 Streams (Flujos)	0
7.1.2 Ficheros	0
7.1.3 Serialización	0
7.1.4 Sockets	0
7.1.5 Manejo de ficheros y carpetas (File)	0
7.1.6 Ejemplos UD06	0
7.1.7 Resumen — Conceptos clave	0
7.1.8 Autoevaluación	0
7.1.9 Vídeos recomendados	0
7.2 Comparativa CRUD	0
7.3 Ejercicios de la UD06	0
7.3.1 Paquetes Completos	0
7.3.2 Los flujos estándar	0
7.3.3 <code>InputStreamReader</code>	0
7.3.4 Entrada "orientada a líneas".	0
7.3.5 Lectura/escritura en ficheros	0
7.3.6 Uso de buffers	0
7.3.7 Streams para información binaria	0
7.3.8 Streams de objetos. Serialización.	0
7.3.9 Sockets	0
7.3.10 Más ejercicios (Lionel)	0
7.3.11 Aún más ejercicios	0
7.4 Talleres	0

7.4.1 Taller UD06_01: GitHub Classroom	0
7.4.2 Taller UD06_T02: Sockets en la nube (AWS)	0
8. UD07	0
8.1 Colecciones	0
8.1.1 Introducción	0
8.1.2 Estructuras de almacenamiento	0
8.1.3 Clases y métodos genéricos	0
8.1.4 Colecciones	0
8.1.5 Iteradores	0
8.1.6 Comparadores	0
8.1.7 Extras	0
8.1.8 Programación funcional	0
8.1.9 Ejemplos UD07	0
8.1.10 Resumen — Conceptos clave	0
8.1.11 Autoevaluación	0
8.1.12 Vídeos recomendados	0
8.2 Ejercicios de la UD07	0
8.2.1 Ejercicios	0
8.2.2 Actividades	0
8.2.3 Ejercicios Genericidad	0
8.2.4 Programación funcional. Funciones Lambda.	0
8.3 Talleres	0
8.3.1 Taller UD07_01: GitHub Classroom	0
9. UD08	0
9.1 Composición, Herencia y Polimorfismo	0
9.1.1 Relaciones entre clases.	0
9.1.2 Composición	0
9.1.3 Herencia	0
9.1.4 Clases Abstractas	0
9.1.5 Interfaces	0
9.1.6 Polimorfismo	0
9.1.7 Ejemplos UD08	0
9.1.8 Resumen — Conceptos clave	0
9.1.9 Autoevaluación	0
9.1.10 Vídeos recomendados	0
9.2 Ejercicios de la UD08	0
9.2.1 Ejercicios Herencia	0
9.2.2 Ejercicios Polimorfismo	0

9.2.3 Actividades	0
9.2.4 Ejercicios Lionel	0
9.3 Talleres	0
9.3.1 Taller UD08_01: GitHub Classroom	0
9.3.2 Taller UD08_02: Librerías Maven vs Jar	0
9.3.3 Taller UD08_T03: Introducción a JSON y YAML	0
10. UD09	0
10.1 Interfaz gráfica	0
10.1.1 Introducción	0
10.1.2 Gráfico de escena	0
10.1.3 Controles de la interfaz de usuario	0
10.1.4 Diseño (Layouts)	0
10.1.5 Gestión de excepciones y ventanas modalesAlert	0
10.1.6 Estructura de la aplicación	0
10.1.7 Ejemplos UD09	0
10.1.8 Resumen — Conceptos clave	0
10.1.9 Autoevaluación	0
10.1.10 Vídeos recomendados	0
10.2 Ejercicios de la UD09	0
10.2.1 Cuestiones generales	0
10.2.2 Ejercicios (con SceneBuilder)	0
10.2.3 Actividades	0
10.2.4 Más ejercicios (Sin SceneBuilder)	0
10.3 Talleres	0
10.3.1 Taller UD09_02: Scene Builder, ScenicView y FXMLManager	0
10.3.2 Taller UD09_03: Proyecto JavaFX (con Maven)	0
10.3.3 Taller UD09_04: Calculadora en JavaFX (IntelliJ)	0
11. UD10	0
11.1 Acceso a Bases de Datos Relacionales	0
11.1.1 Introducción	0
11.1.2 JDBC	0
11.1.3 Patrones de diseño aplicables	0
11.1.4 Acceso a BBDD	0
11.1.5 Navegabilidad y concurrencia	0
11.1.6 Consultas (Query)	0
11.1.7 Modificación (update)	0
11.1.8 Inserción (insert)	0
11.1.9 Borrado (delete)	0

11.1.10 Sentencias predefinidas	0
11.1.11 Ejemplos UD10	0
11.1.12 Resumen — Conceptos clave	0
11.1.13 Autoevaluación	0
11.1.14 Vídeos recomendados	0
11.2 Ejercicios de la UD10	0
11.2.1 Actividades	0
11.3 Talleres	0
11.3.1 Taller UD10_02: Conectores	0
11.3.2 Taller UD10_03: BBDD en la nube (AWS) con IntelliJ	0
11.3.3 Taller UD10_04: Patron DAO (CRUD completo)	0
12. UD11	0
12.1 Bases de Datos Orientadas a Objetos	0
12.1.1 Introducción	0
12.1.2 Los lenguajes ODL y OQL	0
12.1.3 La librería ObjectDB	0
12.1.4 Anotaciones para ObjectDB	0
12.1.5 Comparativa BDR vs BDOO vs ORM	0
12.1.6 Ejemplos	0
12.1.7 Resumen — Conceptos clave	0
12.1.8 Autoevaluación	0
12.1.9 Vídeos recomendados	0
12.2 Ejercicios de la UD11	0
12.2.1 Actividades	0
12.2.2 Ejercicios	0
12.2.3 Ejercicios propuestos	0
12.2.4 Supuestos prácticos	0
12.2.5 Auto evaluación	0
12.3 Talleres	0
12.3.1 Taller UD11_02: Añadir ObjectDB a un proyecto IntelliJ (Maven)	0
12.3.2 Taller UD11_03: CRUD con ObjectDB	0
13. 🙋 Sobre mí...	0
13.1 🧑 David Martinez Peña	0
13.2 📧 Contacto:	0
14. Fuentes de información	0
14.1 Canales de vídeo	0

1. UD00

1.1 🚀 Información importante



PROGRAMACIÓN

<https://martinezpenya.es/1DAMProgramacion/>
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1.1.1 🏠 Denominación del curso

📖 **Ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma**
☕ Programación (PRG)

1.1.2 📄 Contenidos

		Horas
Bloque	P R I M E R TRIMESTRE	89
B1	UD01: Elementos de un programa informático	19
B2	UD02: Utilización de Objetos	20
	PRUEBA UNIDADES 1 Y 2	6
B3	UD03: Estructuras de control y Excepciones	20
B4	UD04: Estructuras de datos Arrays y matrices. Recursividad	18
	1ª EVALUACIÓN	6
	S E G U N D O TRIMESTRE	77
B5	UD05: Desarrollo de clases	25
B6	UD06: Lectura y escritura de información	25
B4	UD07: Colecciones y Funciones Lambda	21
	2ª EVALUACIÓN	6
	T E R C E R TRIMESTRE	90
B8	UD08: Composición, Herencia y Polimorfismo	20
B9	UD09: Creación de interfaces gráficas	20
B10	UD10: Acceso a Bases de datos	24
B11	UD11: BBDD OO	16
	3ª EVALUACIÓN	6
	CONVOCATÒRIA ORDINÀRIA	4

	Horas
T O T A L	256

1.1.3 Resultados de Aprendizaje (RA)

	Descripció	Pes	UNITAT	Avaluació
RA1	Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	10%		
A	Se han identificado los bloques que componen la estructura de un programa informático.	11%	1	=[[1AVA]]
B	Se han creado proyectos de desarrollo de aplicaciones	11%	2	=[[1AVA]]
C	Se han utilizado entornos integrados de desarrollo.	11%	2	=([1AVA]*0,5)+([FEE]*0,5)
D	Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
E	Se ha modificado el código de un programa para crear y utilizar variables.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
F	Se han creado y utilizado constantes y literales.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
G	Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
H	Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
I	Se han introducido comentarios en el código.	11%	1	=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)
	Descripció	Pes	UNITAT	Avaluació
RA2	Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.	10%		

	Descripció	Pes	UNITAT	Avaluació
A	Se han identificado los fundamentos de la programación orientada a objetos.	11%	2	$=[[1AVA]]$
B	Se han escrito programas simples.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
C	Se han instanciado objetos a partir de clases predefinidas.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
D	Se han utilizado métodos y propiedades de los objetos.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
E	Se han escrito llamadas a métodos estáticos.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
F	Se han utilizado parámetros en la llamada a métodos.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
G	Se han incorporado y utilizado librerías de objetos.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
H	Se han utilizado constructores.	11%	2	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
I	Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.	11%	2	$=([1AVA]*0,05)+([2AVA]*0,15)+([3AVA]*0,30)+([FEE]*0,50)$
	Descripció	Pes	UNITAT	Avaluació
RA3	Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.	10%		
A	Se ha escrito y probado código que haga uso de estructuras de selección.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
B	Se han utilizado estructuras de repetición.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
C	Se han reconocido las posibilidades de las sentencias de salto.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
D	Se ha escrito código utilizando control de excepciones.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
E	Se han creado programas ejecutables utilizando diferentes estructuras de control.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$

	Descripció	Pes	UNITAT	Avaluació
F	Se han probado y depurado los programas.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
G	Se ha comentado y documentado el código.	11%	3	$=([1AVA]*0,05)+([2AVA]*0,15)+([3AVA]*0,30)+([FEE]*0,50)$
H	Se han creado excepciones.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
I	Se han utilizado aserciones para la detección y corrección de errores durante la fase de desarrollo.	11%	3	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
	Descripció	Pes	UNITAT	Avaluació
RA4	Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.	10%		
A	Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
B	Se han definido clases.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
C	Se han definido propiedades y métodos.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
D	Se han creado constructores.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
E	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
F	Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
G	Se han definido y utilizado clases heredadas.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
H	Se han creado y utilizado métodos estáticos.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
I	Se han creado y utilizado conjuntos y librerías de clases.	11%	5	$=([2AVA]*0,5)+([3AVA]*0,5)$
	Descripció	Pes	UNITAT	Avaluació

	Descripció	Pes	UNITAT	Avaluació
RA5	Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.	15%		
A	Se ha utilizado la consola para realizar operaciones de entrada y salida de información.	10%	6	$=([1AVA]*0,05)+([2AVA]*0,15)+([3AVA]*0,30)+([FEE]*0,50)$
B	Se han aplicado formatos en la visualización de la información.	10%	6	$=([1AVA]*0,2)+([2AVA]*0,3)+([3AVA]*0,5)$
C	Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.	10%	6	$=([2AVA]*0,5)+([3AVA]*0,5)$
D	Se han utilizado ficheros para almacenar y recuperar información.	10%	6	$=([2AVA]*0,5)+([3AVA]*0,5)$
E	Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.	10%	6	$=([2AVA]*0,5)+([3AVA]*0,5)$
F	Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.	20%	9	$'=([3AVA]*0,25)+([FEE]*0,50)+([UD09_T01]*0,25)+([UD09_T02]*0,25)+([UD09_T03]*0,5)*0,25)$
G	Se han programado controladores de eventos.	15%	9	$'=([3AVA]*0,4)+([UD09_T01]*0,25)+([UD09_T02]*0,25)+([UD09_T03]*0,5)*0,6)$
H	Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.	15%	9	$'=([3AVA]*0,4)+([UD09_T01]*0,25)+([UD09_T02]*0,25)+([UD09_T03]*0,5)*0,6)$
	Descripció	Pes	UNITAT	Avaluació
RA6	Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.	20%		
A	Se han escrito programas que utilicen matrices (arrays) .	50%	4	$=([2AVA]*0,5)+([3AVA]*0,50)$
B	Se han reconocido las librerías de clases	5%	7	$=([3AVA])$

	Descripció	Pes	UNITAT	Avaluació
	relacionadas con tipos de datos avanzados.			
C	Se han utilizado listas para almacenar y procesar información.	5%	7	=[[3AVA]]
D	Se han utilizado iteradores para recorrer los elementos de las listas.	5%	7	=[[3AVA]]
E	Se han reconocido las características y ventajas de cada una de la colecciones de datos disponibles.	10%	7	=[[3AVA]]
F	Se han creado clases y métodos genéricos.	5%	7	=[[3AVA]]
G	Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.	5%	7	=[[3AVA]]
H	Se han identificado las clases relacionadas con el tratamiento de documentos escritos en diferentes lenguajes de intercambio de datos.	5%	7	$=([UD08_T01]*0,5)+([UD08_T02]*0,5)$
I	Se han realizado programas que realicen manipulaciones sobre documentos escritos en diferentes lenguajes de intercambio de datos.	5%	7	$=([UD08_T01]*0,5)+([UD08_T02]*0,5)$
J	Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.	5%	7	=[[3AVA]]
	Descripció	Pes	UNITAT	Avaluació
RA7	Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.	10%		
A	Se han identificado los conceptos de herencia, superclase y subclase.	10%	8	=[[3AVA]]
B		10%	8	=[[3AVA]]

	Descripció	Pes	UNITAT	Avaluació
	Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.			
C	Se ha reconocido la incidencia de los constructores en la herencia.	10%	8	=[[3AVA]]
D	Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.	10%	8	=[[3AVA]]
E	Se han diseñado y aplicado jerarquías de clases.	10%	8	=[[3AVA]]
F	Se han probado y depurado las jerarquías de clases.	10%	8	=[[3AVA]]
G	Se han realizado programas que implementen y utilicen jerarquías de clases.	10%	8	=[[3AVA]]
H	Se ha comentado y documentado el código.	10%	8	=([[[3AVA]]*0,50)+([[[FEE]]*0,50)
I	Se han identificado y evaluado los escenarios de uso de interfaces.	10%	8	=[[3AVA]]
J	Se han identificado y evaluado los escenarios de utilización de la herencia y la composición.	10%	8	=[[3AVA]]
	Descripció	Pes	UNITAT	Avaluació
RA8	Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.	5%		
A	Se han identificado las características de las bases de datos orientadas a objetos.	13%	11	=([[[UD11_T1]]*0,3)+([[[UD11_T2]]*0,7)
B	Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos.	13%	11	=([[[UD11_T1]]*0,3)+([[[UD11_T2]]*0,7)

	Descripció	Pes	UNITAT	Avaluació
C	Se han instalado sistemas gestores de bases de datos orientados a objetos.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
D	Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
E	Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
F	Se han programado aplicaciones que almacenen objetos en las bases de datos creadas.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
G	Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
H	Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados.	13%	11	$=([UD11_T1]*0,3)+([UD11_T2]*0,7)$
	Descripció	Pes	UNITAT	Avaluació
RA9	Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.	10%		
A	Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.	14%	10	$=([UD10_T1]*0,2)+([UD10_T2]*0,3)+([UD10_T3]*0,5)$
B	Se han programado conexiones con bases de datos.	14%	10	$=([UD10_T1]*0,2)+([UD10_T2]*0,3)+([UD10_T3]*0,5)$
C	Se ha escrito código para almacenar información en bases de datos.	14%	10	$=([UD10_T1]*0,2)+([UD10_T2]*0,3)+([UD10_T3]*0,5)$
D	Se han creado programas para recuperar y mostrar	14%	10	$=([UD10_T1]*0,2)+([UD10_T2]*0,3)+([UD10_T3]*0,5)$

	Descripció	Pes	UNITAT	Avaluació
	información almacenada en bases de datos.			
E	Se han efectuado borrados y modificaciones sobre la información almacenada.	14%	10	$=([UD10_T1]*0,2)+([UD10_T2]*0,3)+([UD10_T3]*0,5)$
F	Se han creado aplicaciones que muestren la información almacenada en bases de datos.	14%	10	$=([UD10_T1]*0,05)+([UD10_T2]*0,15)+([UD10_T3]*0,3)+([FEE]*0,5)$
G	Se han creado aplicaciones para gestionar la información presente en bases de datos relacionales.	14%	10	$=([UD10_T1]*0,05)+([UD10_T2]*0,15)+([UD10_T3]*0,3)+([FEE]*0,5)$

1.1.4 Legislación vigente

-  [RD 450/2010, BOE 20-05-2010](#) (Antigua ley)
-  [RD 405/2023 29-05-2023](#)
-  [RD 500/2024, BOE 21-05-2024](#)
-  [Curriculum C.V.: ORDE 58/2012, de 5 de setembre \(DOGV núm. 6868, 24.09.2012\)](#) (Antiguo)
-  [Propuesta de Decreto del Consell](#)
-  [Horario](#) (Antigua ley)
-  [Horario](#)

1.1.5 Evaluación

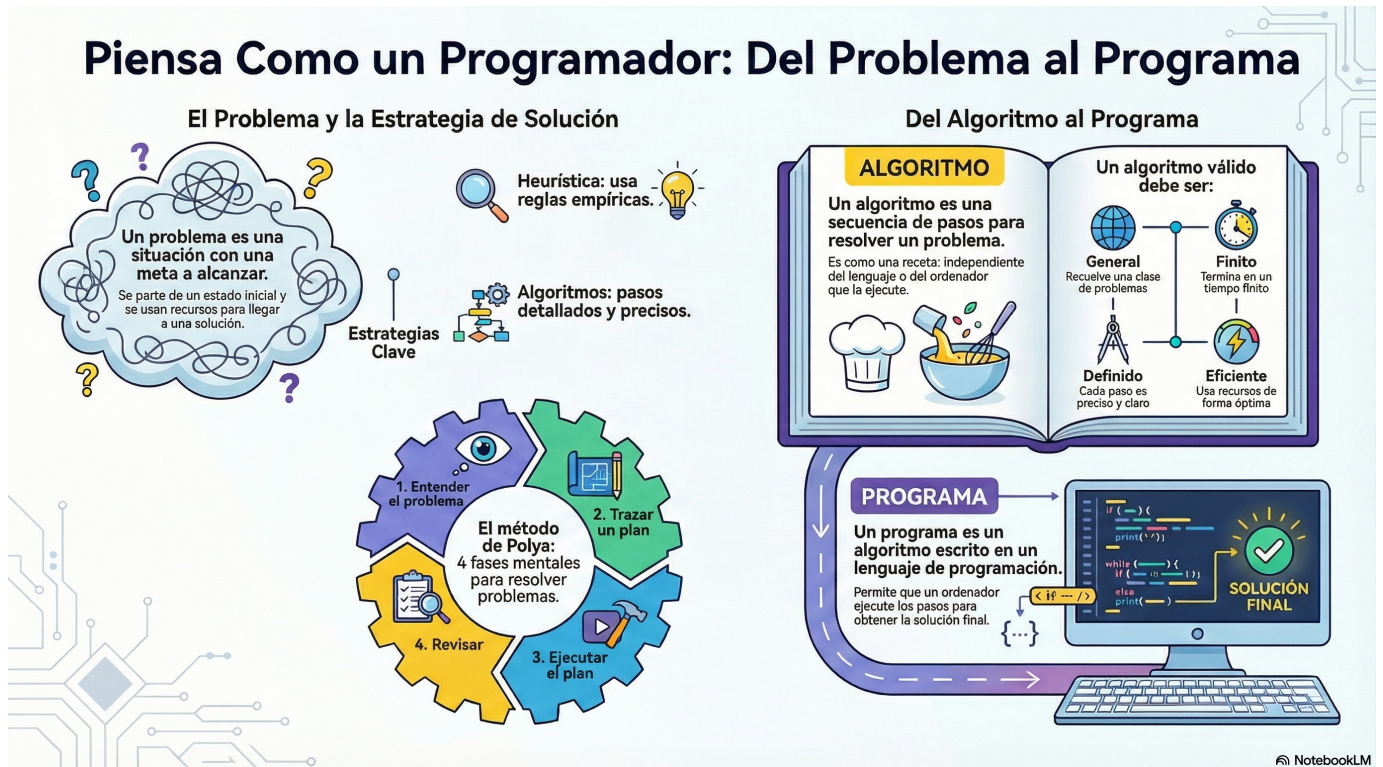
-  La evaluación del módulo se realizará con base en los **Resultados de Aprendizaje (RA)** definidos en el currículo del ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma. Cada RA estará asociado a **criterios de evaluación (CE)** que serán los que determinen el grado de adquisición de las competencias previstas para el módulo.
-  La nota final del módulo se obtendrá a partir de la ponderación de los **RA**, como se mencionó anteriormente. Cada **RA** será evaluado de forma independiente, con calificaciones en una escala de 0 a 10.
-  El alumno debe obtener al menos una nota de **5** en cada **RA** para aprobar el módulo.
-  Si un alumno obtiene menos de un **5** en algún RA, tendrá que recuperarlo mediante las actividades/exámenes de recuperación diseñadas específicamente para esos resultados de aprendizaje.
-  En programación los primeros RA's se distribuyen entre las 3 evaluaciones, así que tener una buena nota en la primera evaluación no quiere decir que has aprobado los RA de esa evaluación.
-  **NUEVO SISTEMA DUAL!!** → Busca tu empresa! 120H (aproximadamente en el mes de mayo, también a partir del 2º trimestre por las mañanas)

Importante

-  Aprobar las distintas evaluaciones no garantiza aprobar el curso.
-  Puedes aprobar (y con muy buena nota) las dos evaluaciones, tener un **RA** suspendido y por tanto suspender el módulo.

2. UD01

2.1 Elementos de un programa informático



Al finalizar esta unidad serás capaz de...

- ✓ Identificar los elementos fundamentales de un programa informático
- ✓ Definir qué es un problema, un algoritmo y un programa
- ✓ Conocer la historia y características del lenguaje Java
- ✓ Declarar variables y constantes utilizando los tipos de datos adecuados
- ✓ Utilizar operadores aritméticos, relacionales y lógicos
- ✓ Realizar conversiones de tipo (implícitas y explícitas)
- ✓ Compilar y ejecutar un programa Java desde la consola

2.1.1 Piensa como un programador

Una de las acepciones que trae el Diccionario de Real Academia de la Lengua Española (RAE) respecto a la palabra Problema es **“Planteamiento de una situación cuya respuesta desconocida debe obtenerse a través de métodos científicos”**. Con miras a lograr esa respuesta.



Definición

Un problema es una situación en la cual se trata de alcanzar una meta y para lograrlo se deben hallar y utilizar unos medios y unas estrategias.

La mayoría de problemas tienen algunos elementos en común: un estado inicial; una meta, lo que se pretende lograr; un conjunto de recursos, lo que está permitido hacer y/o utilizar; y un dominio, el estado actual de conocimientos, habilidades y energía de quien va a resolverlo (Moursund, 1999).

Casi todos los problemas requieren, que quien los resuelve, los divida en submetas que, cuando son dominadas (por lo regular en orden), llevan a alcanzar el objetivo. La solución de problemas también requiere que se realicen operaciones durante el estado inicial y las submetas, actividades (conductuales, cognoscitivas) que alteran la naturaleza de tales estados (Schunk, 1997).

Cada disciplina dispone de estrategias específicas para resolver problemas de su ámbito; por ejemplo, resolver problemas matemáticos implica utilizar estrategias propias de las matemáticas. Sin embargo, algunos psicólogos opinan que es posible utilizar con éxito estrategias generales, útiles para resolver problemas en muchas áreas. A través del tiempo, la humanidad ha utilizado diversas estrategias generales para resolver problemas. Schunk (1997), Woolfolk (1999) y otros, destacan los siguientes métodos o estrategias de tipo general:

- **Ensayo y error** : Consiste en actuar hasta que algo funcione. Puede tomar mucho tiempo y no es seguro que se llegue a una solución. Es una estrategia apropiada cuando las soluciones posibles son pocas y se pueden probar todas, empezando por la que ofrece mayor probabilidad de resolver el problema.

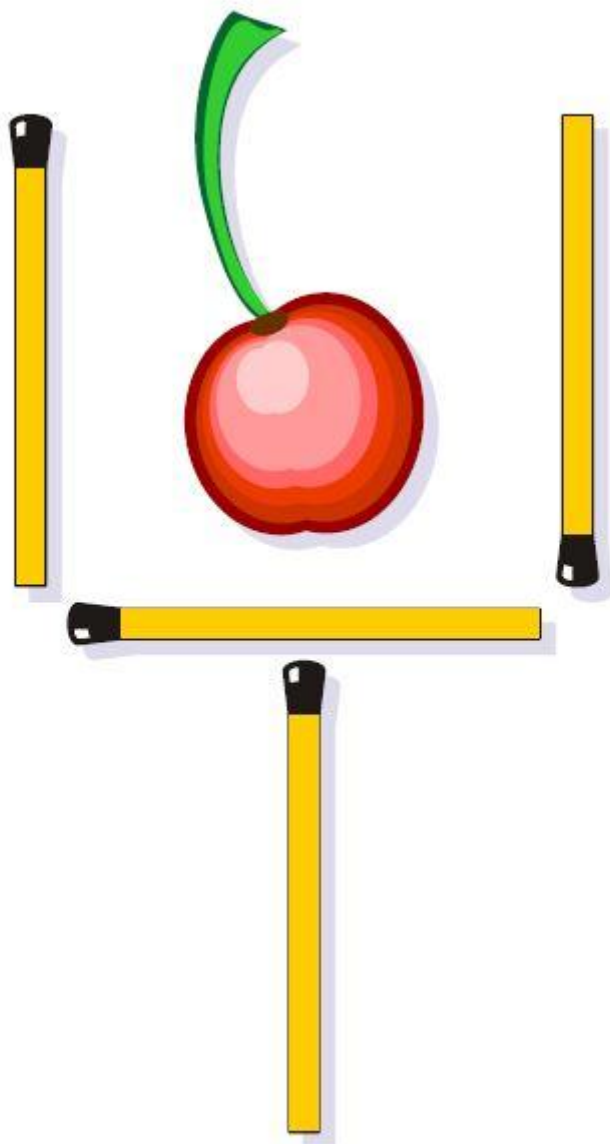
Por ejemplo, una bombilla que no prende: revisar la bombilla, verificar la corriente eléctrica, verificar el interruptor.

- **Iluminación** : Implica la súbita conciencia de una solución que sea viable. Es muy utilizado el modelo de cuatro pasos formulado por Wallas (1921): preparación, incubación, iluminación y verificación.

Estos cuatro momentos también se conocen como proceso creativo. **Algunas investigaciones han determinado que cuando en el periodo de incubación se incluye una interrupción en el trabajo sobre un problema se logran mejores resultados desde el punto de vista de la creatividad.** La incubación ayuda a "olvidar" falsas pistas, mientras que no hacer interrupciones o descansos puede hacer que la persona que trata de encontrar una solución creativa se estanque en estrategias inapropiadas.

Ejemplos:

- Dispones de 6 lapices/palillos/cerillas igual de largos, ¿cómo puedes formar 4 triángulos iguales y equiláteros?
- Mueve 2 cerillas para seguir teniendo una copa pero con la cereza fuera:



- **Heurística** : Se basa en la utilización de reglas empíricas para llegar a una solución. El método heurístico conocido como “IDEAL”, formulado por Bransford y Stein (1984), incluye cinco pasos: Identificar el problema; definir y presentar el problema; explorar las estrategias viables; avanzar en las estrategias; y lograr la solución y volver para evaluar los efectos de las actividades (Bransford & Stein, 1984).

El matemático Polya (1957) también formuló un método heurístico para resolver problemas que se aproxima mucho al ciclo utilizado para programar computadores. A lo largo de esta Guía se utilizará este método propuesto por Polya.

- **Algoritmos** : Consiste en aplicar adecuadamente una serie de pasos detallados que aseguran una solución correcta. Por lo general, cada algoritmo es específico de un dominio del conocimiento. La programación de computadores se apoya en este método.
- **Modelo de procesamiento de información** : El modelo propuesto por Newell y Simon (1972) se basa en plantear varios momentos para un problema (estado inicial, estado final y vías de solución). Las posibles soluciones avanzan por subtemas y requieren que se realicen operaciones en cada uno de ellos.
- **Análisis de medios y fines** : Se funda en la comparación del estado inicial con la meta que se pretende alcanzar para identificar las diferencias.

Luego se establecen submetas y se aplican las operaciones necesarias para alcanzar cada submeta hasta que se alcance la meta global. Con este método se puede proceder en retrospectiva (desde la meta hacia el estado inicial) o en prospectiva (desde el estado inicial hacia la meta).

- **Razonamiento analógico** : Se apoya en el establecimiento de una analogía entre una situación que resulte familiar y la situación problema. Requiere conocimientos suficientes de ambas situaciones.
- **Lluvia de ideas** : Consiste en formular soluciones viables a un problema. El modelo propuesto por Mayer (1992) plantea: definir el problema; generar muchas soluciones (sin evaluarlas); decidir los criterios para estimar las soluciones generadas; y emplear esos criterios para seleccionar la mejor solución. Requiere que los estudiantes no emitan juicios con respecto a las posibles soluciones hasta que terminen de formularlas.
- **Sistemas de producción** : Se basa en la aplicación de una red de secuencias de condición y acción (Anderson, 1990).
- **Pensamiento lateral** : Se apoya en el pensamiento creativo, formulado por Edward de Bono (1970), el cual difiere completamente del pensamiento lineal (lógico). El pensamiento lateral requiere que se exploren y consideren la mayor cantidad posible de alternativas para solucionar un problema. Su importancia para la educación radica en permitir que el estudiante: explore (escuche y acepte puntos de vista diferentes, busque alternativas); avive (promueva el uso de la fantasía y del humor); libere (use la discontinuidad y escape de ideas preestablecidas); y contrarreste la rigidez (vea las cosas desde diferentes ángulos y evite dogmatismos). Este es un método adecuado cuando el problema que se desea resolver no requiere información adicional, sino un reordenamiento de la información disponible; cuando hay ausencia del problema y es necesario percibirse de que hay un problema; o cuando se debe reconocer la posibilidad de perfeccionamiento y redefinir esa posibilidad como un problema (De Bono, 1970).

Ejemplos:

- **El dilema del naufrago.** Un naufrago necesita trasladar a su isla de residencia algunos restos del naufragio de su barco, que afloraron en la orilla de la isla de enfrente. Allí tiene un zorro, un conejo y un racimo de zanahorias, que en su bote puede llevar a razón de uno por viaje. ¿Cómo puede llevarlo todo a su isla, sin que el zorro se coma al conejo, ni éste a las zanahorias?
Respuesta: Deberá llevar primero al conejo y dejar al zorro con las zanahorias. Luego volver y llevarse al zorro, que dejará a solas en su isla, tomar al conejo y llevarlo de vuelta a la de enfrente. Después llevará las zanahorias, dejando al conejo solo y depositándolas junto al zorro. Finalmente regresará para hacer un último viaje con el conejo.
- **El dilema del ascensor.** Un hombre que vive en el décimo piso de un edificio, toma todos los días el ascensor hasta la planta baja, para ir a trabajar. En la tarde, sin embargo, toma de nuevo el mismo ascensor, pero si no hay nadie con él, baja en el séptimo piso y sube el resto de los pisos por la escalera. ¿Por qué?
Respuesta: El hombre es bajito y no logra presionar el botón del décimo piso.
- **La paradoja del globo.** ¿De qué manera podemos pinchar un globo con una aguja, sin que se fugue el aire y sin que el globo estalle?
Respuesta: Debemos pinchar el globo estando desinflado.
- **El dilema del bar.** Un hombre entra a un bar y le pide al barman un vaso de agua. El barman busca debajo de la barra y de golpe apunta al hombre con un arma. Este último da las gracias y se marcha. ¿Qué acaba de ocurrir?
Respuesta: El barman se percató de que el hombre tenía hipo, y decide curárselo dándole un buen susto.

Resumen

Existen muchas estrategias de resolución de problemas, pero esta Guía se enfoca principalmente en dos: **Heurística** (basada en reglas empíricas) y **Algorítmica** (pasos detallados que aseguran una solución correcta).

Según Polya (1957), cuando se resuelven problemas, intervienen cuatro operaciones mentales:

1. Entender el problema;
2. Trazar un plan;
3. Ejecutar el plan (resolver);
4. Revisar;

Es importante notar que estas son flexibles y no una simple lista de pasos como a menudo se plantea en muchos de esos textos (Wilson, Fernández & Hadaway, 1993). Cuando estas etapas se siguen como un modelo lineal, resulta contraproducente para cualquier actividad encaminada a resolver problemas.

Es necesario hacer énfasis en la naturaleza dinámica y cíclica de la solución de problemas. En el intento de trazar un plan, los estudiantes pueden concluir que necesitan entender mejor el problema y deben regresar a la etapa anterior; O cuando han trazado un plan y tratan de ejecutarlo, no encuentran cómo hacerlo; entonces, la actividad siguiente puede ser intentar con un nuevo plan o regresar y desarrollar una nueva comprensión del problema (Wilson, Fernández & Hadaway, 1993; Guzdial, 2000).

**Recuerda**

Las cuatro fases de Polya (Entender, Planificar, Ejecutar, Revisar) son dinámicas y cíclicas, no una simple lista lineal de pasos. Su aplicación a la programación es directa: Analizar, Diseñar el algoritmo, Traducir a código, Depurar.

**Referencia**

La mayoría de los textos escolares de matemáticas abordan la Solución de Problemas bajo el enfoque planteado por Polya.

**Importante**

Las fases de la programación son una traducción directa del método de Polya:

Polya	Programación
Entender el problema	Analizar el problema
Trazar un plan	Diseñar un algoritmo
Ejecutar el plan	Traducir a lenguaje de programación
Revisar	Depurar el programa

Como se puede apreciar, hay una similitud entre las metodologías propuestas para solucionar problemas matemáticos (Clements & Meredith, 1992; Díaz, 1993; Melo, 2001; NAP, 2004) y las cuatro fases para solucionar problemas específicos de áreas diversas, mediante la programación de computadores.

**Actividad****Problema de la Jirafa**

Primera pregunta: ¿Cómo podríamos meter una jirafa dentro de una nevera? Piensa que es un problema para niños y a ellos no se les pasaría por la cabeza trocear al bello animal para resolver un problema.

Segunda pregunta: Repetimos la jugada con distinto protagonista. ¿Cómo metemos un elefante dentro de la nevera?

Tercera pregunta: Imaginemos que el Rey León está celebrando su cumpleaños y ha invitado a todos los animales del reino. Acuden todos excepto uno. ¿Quién falta?

Cuarta pregunta: Estamos frente a un río que debemos cruzar como sea para continuar nuestro camino. El único problema es que esa zona es el hogar de unos cocodrilos muy agresivos y no disponemos de ningún tipo de embarcación para ir al otro lado. ¿Cómo harías para cruzar el río sin morir en el intento?

2.1.2 Problemas, algoritmos y programas

Problemas

**Definición**

La **programación** es una forma de resolución de problemas mediante una secuencia de instrucciones que pueden ejecutarse de forma automática en un ordenador.

Para que un problema pueda resolverse utilizando un programa informático, éste tiene que poder resolverse de forma mecánica, es decir, mediante una secuencia de instrucciones u operaciones que se puedan llevar a cabo de manera **automática** por un ordenador.

Ejemplos de problemas resolubles mediante un ordenador:

- Determinar el producto de dos números a y b .

- Determinar la raíz cuadrada positiva del número 2.
- Determinar la raíz cuadrada positiva de un número n cualquiera.
- Determinar si el número n , entero mayor que uno, es primo.
- Dada la lista de palabras, determinar las palabras repetidas.
- Determinar si la palabra p es del idioma castellano.
- Ordenar y listar alfabéticamente todas las palabras del castellano.
- Dibujar en pantalla un círculo de radio r .
- Separar las sílabas de una palabra p .
- A partir de la fotografía de un vehículo, reconocer y leer su matrícula.
- Traducir un texto de castellano a inglés.
- Detectar posibles tumores a partir de imágenes radiográficas.

Por otra parte, el científico Alan Turing, demostró que existen problemas irresolubles, de los que ningún ordenador será capaz de obtener nunca su solución.

Atención

Los problemas deben definirse de forma **general** y **precisa**, evitando ambigüedades. Una definición ambigua conduce a interpretaciones distintas y, por tanto, a soluciones incorrectas.

Ejemplo: Raíz cuadrada.

- Determinar la raíz cuadrada de un número n .
- Determinar la raíz cuadrada de un número n , entero no negativo, cualquiera.

Ejemplo: Dividir.

- Calcular la división de dos números de dos números a y b .
- Calcular el cociente entero de la división a/b , donde a y b son números enteros y b es distinto de cero. ($5/2 = 2$).
- Calcular el cociente real de la división a/b , donde a y b son números reales y b es distinto de cero ($5/2 = 2.5$).

Algoritmos

Definición

Dado un problema P , un **algoritmo** es un conjunto de reglas o pasos que indican cómo resolver P en un tiempo finito. Es independiente del lenguaje de programación y del dispositivo donde se ejecute.

Ejemplo

Secuencias de reglas básicas que utilizamos para realizar operaciones aritméticas: sumas, restas, productos y divisiones.

Algoritmo para desayunar

```

1  Begin
2      Sentarse
3      Servirse café con leche
4      Servirse azúcar
5      If tengo tiempo
6          While tenga apetito
7              Untar mantequilla en una tostada
8              Añadir mermelada
9              Comer la tostada
10         End While
11     End If
12     Beberse el café con leche
13     Levantarse
14 End

```

Un algoritmo, por tanto, no es más que la secuencia de pasos que se deben seguir para solucionar un problema específico. La descripción o nivel de detalle de la solución de un problema en términos algorítmicos depende de qué o quién debe entenderlo, interpretarlo y resolverlo.

Los algoritmos son independientes de los lenguajes de programación y de las computadoras donde se ejecutan. Un mismo algoritmo puede ser expresado en diferentes lenguajes de programación y podría ser ejecutado en diferentes dispositivos. Piensa en una receta de cocina, ésta puede ser expresada en castellano, inglés o francés, podría ser cocinada en fogón o vitrocerámica, por un cocinero o más, etc. Pero independientemente de todas estas circunstancias, el plato se preparará siguiendo los mismos pasos.



Importante

La diferencia fundamental entre **algoritmo** y **programa**: el algoritmo describe los pasos para resolver un problema (independiente del lenguaje), mientras que el programa es la implementación concreta de ese algoritmo en un lenguaje de programación específico para ser ejecutado en un ordenador.

CARACTERÍSTICAS DE LOS ALGORITMOS



Atención

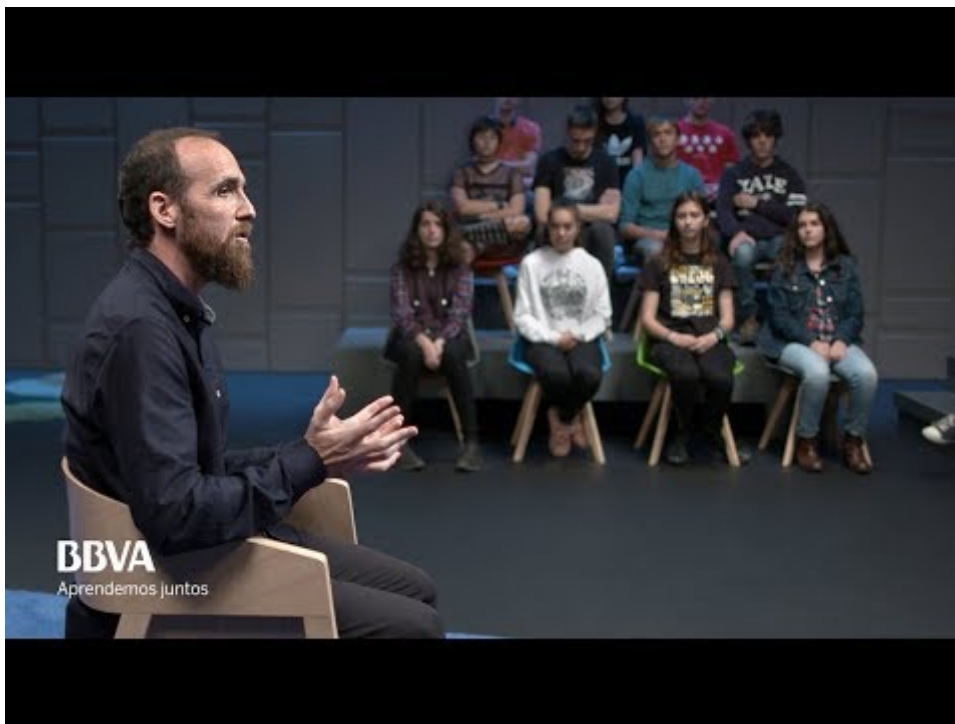
Un algoritmo debe cumplir cuatro características para ser válido:

- **Generalidad**: debe resolver toda una clase de problemas, no uno aislado.
- **Finitud**: debe acabar necesariamente tras un número finito de pasos.
- **Definibilidad**: debe estar definido de forma exacta y precisa, sin ambigüedades.
- **Eficiencia**: debe resolver el problema de forma rápida y eficiente.



Vídeo

Juego de las monedas (Eduardo Sáenz Cabezón)



Desde el comienzo del enlace hasta 7 minutos después.

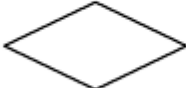
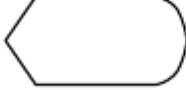
REPRESENTACIÓN DE ALGORITMOS

Los métodos más usuales para representar algoritmos son los diagramas de flujo y el pseudocódigo. Ambos son sistemas de representación independientes de cualquier lenguaje de programación. Hay que tener en cuenta que el diseño de un algoritmo constituye un paso previo a la codificación de un programa en un lenguaje de programación determinado (C, C++, Java, Pascal). La independencia del algoritmo del lenguaje de programación facilita, precisamente, la posterior codificación en el lenguaje elegido.

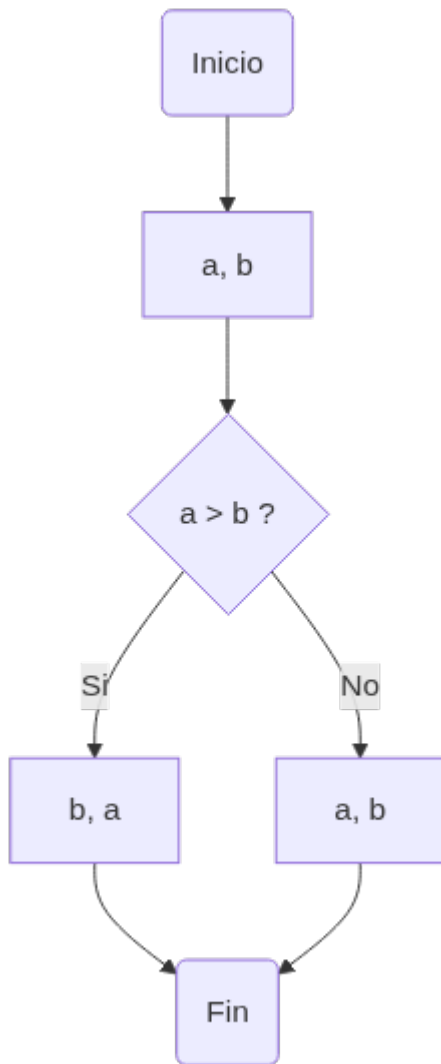
i Definición

Un **diagrama de flujo** es una representación gráfica de un algoritmo mediante símbolos estándar (inicio/fin, proceso, decisión, entrada/salida) unidos por flechas que indican el orden de ejecución.

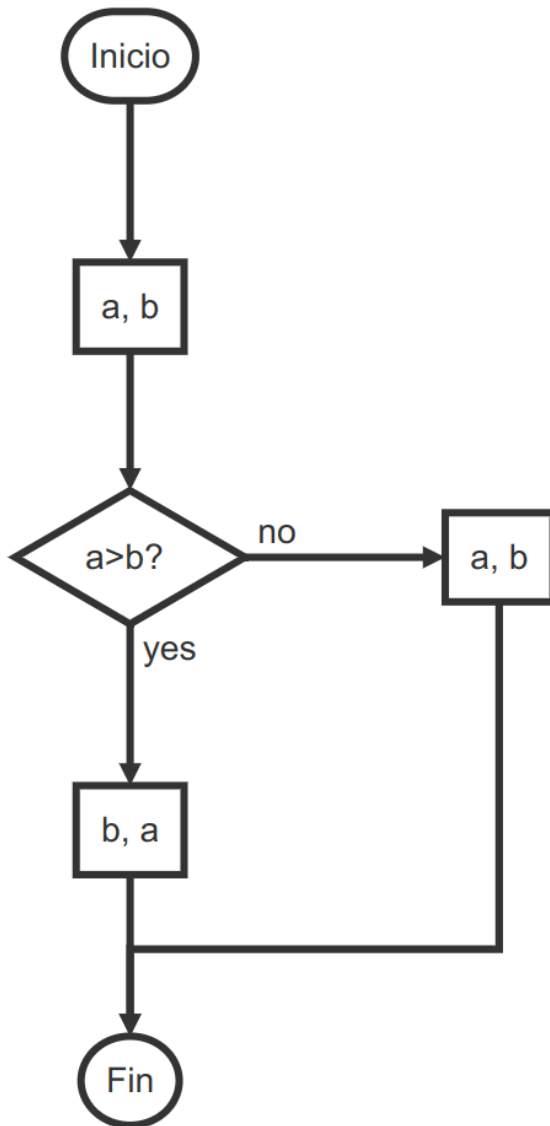
Los símbolos más utilizados son:

Proceso		Cualquier tipo de operación que pueda originar cambio de valor, formato, operaciones aritméticas etc
Decisión		Indica operaciones lógicas o de comparación entre datos y en función del resultado determina el camino a seguir.
Terminal		Representa el comienzo y el final de un programa o un módulo
Entrada / Salida de información		Este símbolo se puede subdividir en otros de teclado, pantalla, impresora disco etc.
Teclado		Representa las entradas de datos desde teclado
Pantalla		Representa las salidas de datos en pantalla
Dirección del flujo		Indica el sentido de ejecución de las operaciones

Ejemplo: Mostrar dos números ordenados de menor a mayor.



O también en otra representación:



Definición

El **pseudocódigo** es un lenguaje informal de descripción de algoritmos, cercano a la sintaxis de los lenguajes de programación, que NO puede ejecutarse en un ordenador. Debe traducirse (codificarse) a un lenguaje real.

El pseudocódigo no se puede ejecutar nunca en el ordenador, sino que tiene que traducirse a un lenguaje de programación (codificación). La ventaja del pseudocódigo, frente a los diagramas de flujo, es que se puede modificar más fácilmente si detecta un error en la lógica del algoritmo, y puede ser traducido fácilmente a los lenguajes estructurados como Pascal, C, fortran, Java, etc.

El Pseudocódigo utiliza palabras reservadas (en sus orígenes se escribían en inglés) para representar las sucesivas acciones. Para mayor legibilidad utiliza la **indentación** (sangría en el margen izquierdo) de sus líneas.

Ejemplo: Mostrar dos números ordenados de menor a mayor.

```

1  Begin
2    Leer (A, B)
3    If (A>B) then
4      Escribir (B, A)
5    Else
6      Escribir (A, B)
7    End If
8  End
  
```

Programas

Los lenguajes de programación son sólo un medio para expresar el algoritmo y el ordenador un procesador para ejecutarlo. El diseño de los algoritmos será una tarea que necesitará de la creatividad y conocimientos de las técnicas de programación. Estilos distintos, de distintos programadores a la hora de obtener la solución del problema, darán lugar a programas diferentes, igualmente válidos.

Pero cuando los problemas son complejos, es necesario descomponer éstos en subproblemas más simples y, a su vez, en otros más pequeños. Estas estrategias reciben el nombre de diseño descendente (Metodología de diseño de programas, consistente en la descomposición del problema en problemas más sencillos de resolver) o diseño modular (top-down design) (Metodología de diseño de programas, que consiste en dividir la solución a un problema en módulos más pequeños o subprogramas. Las soluciones de los módulos se unirán para obtener la solución general del problema). Este sistema se basa en el lema **divide y vencerás**.



Recuerda

El **diseño descendente** (divide y vencerás) es una de las estrategias más importantes en programación: descomponer un problema complejo en subproblemas más simples, y estos a su vez en otros más pequeños, hasta que cada pieza sea fácil de resolver.

2.1.3 Java

¿Qué y cómo es Java?

Java es un lenguaje sencillo de aprender, con una sintaxis parecida a la de C++, pero en la que se han eliminado elementos complicados y que pueden originar errores. Java es orientado a objetos, con lo que elimina muchas preocupaciones al programador y permite la utilización de gran cantidad de bibliotecas ya definidas, evitando reescribir código que ya existe. Es un lenguaje de programación creado para satisfacer nuevas necesidades que los lenguajes existentes hasta el momento no eran capaces de solventar.



Información

La principal virtud de Java es su **independencia de plataforma**: el código se compila a *bytecode*, que se ejecuta sobre la Máquina Virtual Java (JVM), permitiendo que un mismo programa funcione en cualquier sistema operativo. Lema: *"Write once, run everywhere"*.

Antes de que apareciera Java, el lenguaje C era uno de los más extendidos por su versatilidad. Pero cuando los programas escritos en C aumentaban de volumen, su manejo comenzaba a complicarse. Mediante las técnicas de programación estructurada y programación modular se conseguían reducir estas complicaciones, pero no era suficiente.

Fue entonces cuando la Programación Orientada a Objetos (POO) entra en escena, aproximando notablemente la construcción de programas al pensamiento humano y haciendo más sencillo todo el proceso. Los problemas se dividen en objetos que tienen propiedades e interactúan con otros objetos, de este modo, el programador puede centrarse en cada objeto para programar internamente los elementos y funciones que lo componen.



Resumen

Características clave de Java: independiente de plataforma, orientado a objetos, sintaxis similar a C/C++, distribuido (TCP/IP), amplias bibliotecas, robusto (comprobaciones en compilación y ejecución), y seguro.

Breve historia.

Java surgió en 1991 cuando un grupo de ingenieros de Sun Microsystems trataron de diseñar un nuevo lenguaje de programación destinado a programar pequeños dispositivos electrónicos. La dificultad de estos dispositivos es que cambian continuamente y para que un programa funcione en el siguiente dispositivo aparecido, hay que reescribir el código. Por eso la empresa Sun quería crear un lenguaje independiente del dispositivo.

Pero no fue hasta 1995 cuando pasó a llamarse Java, dándose a conocer al público como lenguaje de programación para computadores. Java pasa a ser un lenguaje totalmente independiente de la plataforma y a la vez potente y orientado a objetos. Esa filosofía y su facilidad para crear aplicaciones para redes TCP/IP ha hecho que sea uno de los lenguajes más utilizados en la actualidad.

El factor determinante para su expansión fue la incorporación de un intérprete Java en la versión 2.0 del navegador Web Netscape Navigator, lo que supuso una gran revuelo en Internet. A principios de 1997 apareció Java 1.1 que proporcionó sustanciales mejoras al lenguaje. Java 1.2, más tarde rebautizado como Java 2, nació a finales de 1998.

El principal objetivo del lenguaje Java es llegar a ser el nexo universal que conecte a los usuarios con la información, esté ésta situada en el ordenador local, en un servidor Web, en una base de datos o en cualquier otro lugar.

Para el desarrollo de programas en lenguaje Java es necesario utilizar un entorno de desarrollo denominado JDK (Java Development Kit), que provee de un compilador y un entorno de ejecución (JRE – Java RunEnvironment) para los bytecodes generados a partir del código fuente. Al igual que las diferentes versiones del lenguaje han incorporado mejoras, el entorno de desarrollo y ejecución también ha sido mejorado sucesivamente.

Java 2 es la tercera versión del lenguaje, pero es algo más que un lenguaje de programación, incluye los siguientes elementos:

- Un lenguaje de programación: Java.
- Un conjunto de bibliotecas estándar que vienen incluidas en la plataforma y que son necesarias en todo entorno Java. Es el Java Core.
- Un conjunto de herramientas para el desarrollo de programas, como es el compilador de bytecodes, el generador de documentación, un depurador, etc.
- Un entorno de ejecución que en definitiva es una máquina virtual que ejecuta los programas traducidos a bytecodes.

Compilar y ejecutar un programa Java . Uso de la consola.

Veamos los pasos para compilar e interpretar nuestro primer programa escrito en lenguaje Java.

ESTRUCTURA Y BLOQUES FUNDAMENTALES DE UN PROGRAMA.



Ejemplo Holamundo.java

Consulta el [Ejemplo01 — Hola Mundo](#) para ver el código completo.



Atención

En Java, el nombre del archivo `.java` debe coincidir **exactamente** con el nombre de la clase pública que contiene (respetando mayúsculas y minúsculas). Por ejemplo: la clase `public class Ejemplo` debe guardarse en `Ejemplo.java`.

```
1 public class Holamundo {
2     [...]
3 }
```

El código java en las clases se agrupa en funciones o métodos. Cuando java ejecuta el código de una clase busca la función o método `main()` para ejecutarla. Es público (`public`) estático (`static`) para llamarlo sin instanciar la clase. No devuelve ningún valor (`void`) y admite parámetros (`Strings[] args`) que en este caso no se han utilizado.

```
1 [...]
2 public static void main (String[] args)
3 {
4     [...]
5 }
6 [...]
```

El código de la función `main` se escribe entre las llaves. Por ejemplo:

```
1 [...]
2     System.out.println("Hola Mundo");
3 [...]
```

Muestra por pantalla el mensaje `Hola Mundo`, ya que la clase `System` tiene un atributo `out` con dos métodos: `print()` y `println()`. La diferencia es que `println` muestra mensaje e introduce un retorno de carro.

**Importante**

En Java, toda instrucción debe terminar con punto y coma (;). Las únicas excepciones son las llaves de apertura { y cierre } .

SANGRADO O TABULADO

**Recomendación**

Aplica sangrado (indentación) consistente de 2-4 espacios en cada nivel lógico. Para principiantes se recomiendan 4 espacios. No mezcles tabuladores con espacios; elige uno y sé consistente.

**Recomendación**

Limita las líneas de código a 70-90 caracteres. Si una línea es más larga, divídela: - Tras una coma - Antes de un operador (que pasará a la línea siguiente) - En una construcción de alto nivel (paréntesis) - La línea continuada debe alinearse con un sangrado lógico adicional

Unos pocos ejemplos, para comprender mejor:

Unos pocos ejemplos, para comprender mejor:

Dividir tras una coma:

```
1  funcion(expresionMuuuuyLarga1,
2      expresionMuuuyyyyLarga2,
3      expresionMuuuyyyLarga3);
```

Mantener la expresión entre paréntesis en la misma línea:

```
1  nombreLargo = nombreLargo2*
2      (nombreLargo3 + nombreLargo4)+
3      4*nombreLargo5;
```

Siempre hay excepciones. Puede resultar que al aplicar estas reglas, en operaciones muy largas, o expresiones lógicas enormes, el sangrado sea ilegible. En estos casos, el convenio se puede relajar.

PASO 1: CREACIÓN DEL CÓDIGO FUENTE

Abrimos un editor de texto (da igual cual sea, siempre que sea capaz de almacenar "texto sin formato" en código ASCII). Una vez abierto escribiremos nuestro primer programa, que mostrará un texto "Hola Mundo" en la consola. De momento no te preocupes si no entiendes lo que escribes, más adelante le daremos sentido. Ahora solo queremos ver si podemos ejecutar java en nuestro equipo.

El código de nuestro programa en Java será el siguiente (consulta el [Ejemplo02 — Compilación y ejecución](#) para ver el ciclo completo):

A continuación guardamos nuestro archivo y le ponemos como nombre `Ejemplo.java`. Debemos seguir una norma dictada por Java, hemos de hacer coincidir nombre del archivo y nombre del programa, tanto en mayúsculas como en minúsculas, y la extensión del archivo habrá de ser siempre `.java`.

```

~ : nano — Konsole
Fitxer  Edita  Visualitza  Adreces d'interès  Arranjament  Ajuda
GNU nano 4.8  Ejemplo.java  Modificat
/* Ejemplo Hola Mundo */
public class Ejemplo {
    public static void main(String[] arg) {
        System.out.println("Hola Mundo");
    }
}
^G Ajuda  ^O Desa  ^W On és  ^K Retalla  ^J Justifica  ^C Pos Act
^X Surt   ^R Llegeix  ^\ Reemplaça  ^U Enganxa te  ^T Ortografia  ^_ Vés a línia

```

Debemos recordar exactamente la ruta donde guardamos el archivo de ejemplo `Ejemplo.java`.

PASO 2: COMPILACIÓN DEL PROGRAMA

Vamos a proceder a compilar e interpretar este pequeño programa Java (no te preocupes si todavía no entiendes el significado de las palabras compilar e interpretar, lo veras en la asignatura de `Entornos de Desarrollo`). Para ello usaremos la consola. Una vez en la consola debemos colocarnos en la ruta donde previamente guardamos el archivo `Ejemplo.java`.

A continuación daremos la instrucción para que se realice **el proceso de compilación del programa**, para lo que escribiremos `javac Ejemplo.java`, donde `javac` es el nombre del compilador (`java c ompiler`) que transformará el programa que hemos escrito nosotros en lenguaje Java al lenguaje de la máquina virtual Java (`bytecode`), dando como resultado un nuevo archivo `Ejemplo.class` que se creará en este mismo directorio. Comprueba que no aparezca ningún error y que `javac` esté instalado en tu sistema (desde la consola lo puedes comprobar con el comando `javac --version` y debería aparecer el número de versión que tienes instalada). Si aparecen los dos archivos tanto `Ejemplo.java` (código fuente) como `Ejemplo.class` (bytecode creado por el compilador) puedes continuar.

Recuerda

El proceso de compilación (`javac`) transforma código fuente `.java` en bytecode `.class`. El proceso de ejecución (`java`) interpreta el bytecode en la JVM. Dos pasos separados: primero compilar, luego ejecutar.

PASO 3: EJECUCIÓN DEL PROGRAMA

Finalmente, vamos a pedirle al intérprete (JVM) que ejecute el programa, es decir, que transforme el código de la máquina virtual Java en código máquina interpretable por nuestro ordenador y lo ejecute. Para ello escribiremos en la ventana consola: `java Ejemplo`.

El resultado será que se nos muestra la cadena `Hola Mundo`. Si logramos visualizar este texto en pantalla, ya hemos desarrollado nuestro primer programa en Java.

Pregunta frecuente

¿Por qué no necesito compilar mi archivo `.java` antes de ejecutarlo y funciona directamente si me salto ese paso?

<https://stackoverflow.com/questions/54493058/running-a-java-program-without-compiling>

2.1.4 Componentes del lenguaje Java

Variables, identificadores, convenciones.

VARIABLES

Definición

Una **variable** es una posición de memoria identificada por un nombre, con un tipo de dato que determina qué clase de información puede almacenar y qué rango de valores admite.

Las variables vienen determinadas por: - un **nombre**, que permite al programa acceder al valor que contiene en memoria. Debe ser un identificador válido. - un **tipo de dato**, que especifica qué clase de información guarda la variable en esa zona de memoria - un **rango de valores** que puede admitir dicha variable.

Las variables declaradas dentro de un bloque `{ }` son accesibles solo dentro de ese bloque. Una variable local no puede ser declarada como `static`. Una variable no puede declararse fuera de la clase.

Importante

El ámbito (scope) de una variable es la zona del código donde es accesible. Las variables declaradas dentro de un bloque `{ }` solo existen y son visibles dentro de ese bloque. Fuera de él, no están disponibles.

Al nombre que le damos a la variable se le llama identificador. Los identificadores permiten nombrar los elementos que se están manejando en un programa. Vamos a ver con más detalle ciertos aspectos sobre los identificadores que debemos tener en cuenta.

IDENTIFICADORES

Definición

Un **identificador** es un nombre que damos a variables, clases, métodos, etc. Debe empezar por letra, `_` o `$`, y puede contener letras, dígitos, `_` y `$` (sin espacios). Java distingue mayúsculas de minúsculas. Por ejemplo, son válidos los siguientes identificadores:

- `x5`
- `αη`
- `NUM_MAX`
- `numCuenta`

Unicode es un código de caracteres o sistema de codificación, un alfabeto que recoge los caracteres de prácticamente todos los idiomas importantes del mundo. Además, el código Unicode es “compatible” con el código ASCII, ya que para los caracteres del código ASCII, Unicode asigna como código los mismos 8 bits, a los que les añade a la izquierda otros 8 bits todos a cero. La conversión de un carácter ASCII a Unicode es inmediata.

CONVENCIONES

Normas de estilo para nombrar variables

Recomendación

Sigue siempre las convenciones de nombres en Java:

- **Variables y métodos:** `lowerCamelCase` (ej. `numAlumnos`, `obtieneValor`)
- **Constantes:** `MAYÚSCULAS_CON_GUIONES` (ej. `TAM_MAX`, `PI`)
- **Clases:** `UpperCamelCase` (ej. `MiClase`, `String`)

Esto hace tu código más legible y profesional.

Identificador	Convención	Ejemplo
		<code>numAlumnos</code> , <code>suma</code>

Identificador	Convención	Ejemplo
nombre de variable	Comienza por letra minúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama <i>lowerCamelCase</i> .	
nombre de constante	En letras mayúsculas, separando las palabras con el guión bajo, por convenio el guión bajo no se utiliza en ningún otro sitio	TAM_MAX, PI
nombre de una clase	Comienza por letra mayúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama <i>UpperCamelCase</i> .	String, MiTipo
nombre de función	Comienza por letra minúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama <i>lowerCamelCase</i> .	modificaValor, obtieneValor

Puedes consultar estas y otras convenciones sobre código Java en este [enlace](#).

Palabras reservadas

Atención

Las **palabras reservadas** (`class`, `public`, `static`, `void`, `int`, etc.) NO pueden usarse como identificadores (nombres de variables, clases o métodos). Si intentas usarlas, el compilador producirá un error.

Las palabras reservadas en Java son:

```
1 abstract, continue, for, new, switch, assert, default, goto, package, synchronized, boolean, do, if, private, this, break, double, implements, protected, throw, byte, else, import, public, throws, case, enum, instanceof, return, transient, catch, extends, int, short, try, char, final, interface, static, void, class, finally, long, strictfp, volatile, const, float, native, super, while.
```

Tipos de datos.

Los tipos de datos se utilizan para declarar variables y el compilador sepa de antemano que tipo de información contendrá la variable.

Java dispone de los siguientes tipos de datos simples:

Tipo de dato	Representación	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
byte	Numérico Entero con signo	1	-128 a 127	0	Byte
short	Numérico Entero con signo	2	-32768 a 32767	0	Short
int	Numérico Entero con signo	4	-2147483648 a 2147483647	0	Integer
long	Numérico Entero con signo	8	-9223372036854775808 a 9223372036854775807	0	Long
float	Numérico en Coma flotante de precisión simple Norma IEEE 754	4	-3.4×10^{-38} a 3.4×10^{38}	0.0	Float
double	Numérico en Coma flotante de precisión doble Norma IEEE 754	8	-1.8×10^{-308} a 1.8×10^{308}	0.0	Double
char	Carácter Unicode	2	\u0000 a \uFFFF	\u0000	Character

Tipo de dato	Representación	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
boolean	Dato lógico	-	true ó false	false	Boolean
void	-	-	-	-	Void

Referencia

Sobre valores por defecto y inicialización de variables: <https://stackoverflow.com/questions/19131336/default-values-and-initialization-in-java>

Ejemplo de declaración y asignación de valores a variables:

Tipo de datos	código
byte	<code>byte a;</code>
short	<code>short b, c=3;</code>
int	<code>int d=-30;</code> <code>int e=0xC125; //la 0x significa Hexadecimal</code>
long	<code>long b=46240;</code> <code>long b=5L; // La L en este caso indica Long</code>
char	<code>char car1='c';</code> <code>char car2=99; //car1 y car2 son iguales, la c equivale al ascii 99</code> <code>char letra = '\u0061'; //código unicode del carácter "a"</code>
float	<code>float pi=3.1416;</code> <code>float pi=3.1416F; //La F significa float</code> <code>float medio=1/2; //0.5</code>
double	<code>double millon=1e6; // 1x10^6</code> <code>double medio=1/2D; //0.5, la D significa double</code> <code>double z=.123; //si la parte entera es 0 se puede omitir</code>
boolean	<code>boolean esPrimero;</code> <code>boolean esPar=false;</code>

Precaución

Ojo con los tipo float: <https://jvns.ca/blog/2023/01/13/examples-of-floating-point-problems/>

Tipos referenciados

A partir de los ocho tipos datos primitivos, se pueden construir otros tipos de datos. Estos tipos de datos se llaman tipos referenciados o referencias, porque se utilizan para almacenar la dirección de los datos en la memoria del ordenador.

```
1 int[] arrayDeEnteros;
2 Cuenta cuentaCliente;
```

En la primera instrucción declaramos una lista de números del mismo tipo, en este caso, enteros. En la segunda instrucción estamos declarando la variable u objeto `cuentaCliente` como una referencia de tipo `Cuenta`.

Cualquier aplicación de hoy en día necesita no perder de vista una cierta cantidad de datos. Cuando el conjunto de datos utilizado tiene características similares se suelen agrupar en estructuras para facilitar el acceso a los mismos, son los llamados datos estructurados.

Son datos estructurados los `arrays`, `listas`, `árboles`, etc. Pueden estar en la memoria del programa en ejecución, guardados en el disco como ficheros, o almacenados en una base de datos.

**Importante**

`String` en Java es un tipo referenciado (objeto), no un tipo primitivo. Sin embargo, Java le da un tratamiento especial que permite usarlo con una sintaxis simplificada, como si fuera un tipo básico. Lo estudiaremos en profundidad en la siguiente unidad.

```
1 String mensaje;
2 mensaje= "El primer programa";
```

Hemos visto qué son las variables, cómo se declaran y los tipos de datos que pueden adoptar. Anteriormente hemos visto un ejemplo de creación de variables, en esta ocasión vamos a crear más variables, pero de distintos tipos primitivos y los vamos a mostrar por pantalla. Los tipos referenciados los veremos en la siguiente unidad.

Para mostrar por pantalla un mensaje utilizamos `System.out`, conocido como la salida estándar del programa. Este método lo que hace es escribir un conjunto de caracteres a través de la línea de comandos. Podemos utilizar `System.out.print` o `System.out.println`. En el segundo caso lo que hace el método es que justo después de escribir el mensaje, sitúa el cursor al principio de la línea siguiente.

El texto en color gris que aparece entre caracteres `//` son comentarios que permiten documentar el código, pero no son tenidos en cuenta por el compilador y, por tanto, no afectan a la ejecución del programa.

Tipos enumerados**Definición**

Un **tipo enumerado** (`enum`) es un tipo de dato definido por el programador que solo puede tomar un conjunto restringido de valores predefinidos. Por ejemplo: `enum Dias {Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo}`.

Los tipos de datos enumerados son una forma de declarar una variable con un conjunto restringido de valores. Es como si definiéramos nuestro propio tipo de datos.

Al considerar Java este tipo de datos como si de una clase se tratara, no sólo podemos definir los valores de un tipo enumerado, sino que también podemos definir operaciones a realizar con él y otro tipo de elementos, lo que hace que este tipo de dato sea más versátil y potente que en otros lenguajes de programación.

**Tipos enumerados**

Consulta el [Ejemplo03 — Tipos enumerados](#) para ver el código completo y su salida.

Tenemos una variable `Dias` que almacena los días de la semana. Para acceder a cada elemento del tipo enumerado se utiliza el nombre de la variable seguido de un punto y el valor en la lista. Más tarde veremos que podemos añadir métodos y campos o variables en la declaración del tipo enumerado, ya que como hemos comentado un tipo enumerado en Java tiene el mismo tratamiento que las clases.

En este ejemplo hemos utilizado el método `System.out.print`. Como podrás comprobar si lo ejecutas, la instrucción `print` escribe el texto que tiene entre comillas pero no salta a la siguiente línea, por lo que la instrucción `println` escribe justo a continuación.

Sin embargo, también podemos escribir varias líneas usando una única sentencia. Así lo hacemos en la instrucción `println`, la cual imprime como resultado tres líneas de texto. Para ello hemos utilizado un carácter especial, llamado carácter escape (`\`). Este carácter sirve para darle ciertas órdenes al compilador, en lugar de que salga impreso en pantalla. Después del carácter de escape viene otro carácter que indica la orden a realizar, juntos reciben el nombre de secuencia de escape. La secuencia de escape `\n` recibe el nombre de carácter de nueva línea. Cada vez que el compilador se encuentra en un texto ese carácter, el resultado es que mueve el cursor al principio de la línea siguiente. En el próximo apartado vamos a ver algunas de las secuencias de escape más utilizadas.

Constantes y literales.**Definición**

Una **constante** es una variable cuyo valor no puede cambiar una vez asignado. En Java se declara con la palabra clave `final`:

```
1 final double IVA = 0.21;
```

Los **literales** pueden ser de tipo simple, null o string, como por ejemplo 230, null o "Java".

Respecto a los literales existen unos caracteres especiales que se representan utilizando secuencias de escape:

Secuencia de escape	Significado	Secuencia de escape	Significado
<code>\b</code>	Retroceso	<code>\r</code>	Retorno de carro
<code>\t</code>	Tabulador	<code>\"</code>	Carácter comillas dobles
<code>\n</code>	Salto de línea	<code>\'</code>	Carácter comillas simples
<code>\f</code>	Salto de página	<code>\</code>	Barra diagonal

Operadores y expresiones.

OPERADORES ARITMÉTICOS

Los **Operadores Aritméticos** permiten realizar operaciones matemáticas:

Operador	Uso	Operación
<code>+</code>	<code>A + B</code>	Suma
<code>-</code>	<code>A - B</code>	Resta
<code>*</code>	<code>A * B</code>	Multiplicación
<code>/</code>	<code>A / B</code>	División
<code>%</code>	<code>A % B</code>	Módulo o resto de una división entera

Operadores aritméticos

Consulta el [Ejemplo04 — Operadores aritméticos](#) para ver ejemplos de cada operación.

OPERADORES RELACIONALES

Los **Operadores Relacionales** permiten evaluar (la respuesta es un booleano: si o no) la igualdad de los operandos:

Operador	Uso	Operación
<code><</code>	<code>a < b</code>	a menor que b
<code>></code>	<code>a > b</code>	a mayor que b
<code><=</code>	<code>a <= b</code>	a menor o igual que b
<code>>=</code>	<code>a >= b</code>	a mayor o igual que b
<code>!=</code>	<code>a != b</code>	a distinto de b
<code>==</code>	<code>a == b</code>	a igual a b

Operadores relacionales

Consulta el [Ejemplo05 — Operadores relacionales](#) para ver ejemplos de cada operador.

OPERADORES LÓGICOS

Los **Operadores Lógicos** permiten realizar operaciones lógicas:

Operador	Uso	Operación
<code>&&</code> o <code>&</code>	<code>a&&b</code> o <code>a&b</code>	a AND b. El resultado será <i>true</i> si ambos operadores son <i>true</i> y <i>false</i> en caso contrario.
<code> </code> o <code> </code>	<code>a b</code> o <code>a b</code>	a OR b. El resultado será <i>false</i> si ambos operandos son <i>false</i> y <i>true</i> en caso contrario
<code>!</code>	<code>!a</code>	NOT a. Si el operando es <i>true</i> el resultado es <i>false</i> y si el operando es <i>false</i> el resultado es <i>true</i> .
<code>^</code>	<code>a^b</code>	a XOR b. El resultado será <i>true</i> si un operando es <i>true</i> y el otro <i>false</i> , y <i>false</i> en caso contrario.

**Operadores lógicos**

Consulta el [Ejemplo06 — Operadores lógicos](#) para ver ejemplos con AND, OR, NOT y XOR.

Para representar resultados de operadores Lógicos también se pueden usar tablas de verdad a las que conviene acostumbrarse:

a	b	a && b	a b	!a	a^b
false	false	false	false	true	false
true	false	false	true	false	true
false	true	false	true	true	true
true	true	true	true	false	false

OPERADORES UNARIOS O UNITARIOS

Los **Operadores Unarios** o **Unitarios** permiten realizar incrementos y decrementos:

Operador	Uso	Operación
<code>++</code>	<code>a++</code> o <code>++a</code>	Incremento de a
<code>--</code>	<code>a--</code> o <code>--a</code>	Decremento de a

**Operadores unarios**

Consulta el [Ejemplo07 — Operadores unarios](#) para ver la diferencia entre prefijo y sufijo.

OPERADORES DE ASIGNACIÓN

Los **Operadores de Asignación** permiten asignar valores:

Operador	Uso	Operación
<code>=</code>	<code>a = b</code>	Asignación (como ya hemos visto)
<code>*=</code>	<code>a *= b</code>	Multiplicación y asignación. La operación <code>a*=b</code> equivale a <code>a=a*b</code>
<code>/=</code>	<code>a /= b</code>	División y asignación. La operación <code>a/=b</code> equivale a <code>a=a/b</code>
<code>%=</code>	<code>a %= b</code>	Módulo y asignación. La operación <code>a%=b</code> equivale a <code>a=a%b</code>
<code>+=</code>	<code>a += b</code>	Suma y asignación. La operación <code>a+=b</code> equivale a <code>a=a+b</code>
<code>-=</code>	<code>a -= b</code>	Resta y asignación. La operación <code>a-=b</code> equivale a <code>a=a-b</code>



Operadores de asignación

Consulta el [Ejemplo08 — Operadores de asignación](#) para ver ejemplos.

OPERADORES DE DESPLAZAMIENTO

Los **Operadores de desplazamiento** permiten desplazar los bits de los valores:

Operador	Utilización	Resultado
<<	<code>a << b</code>	Desplazamiento de <code>a</code> a la izquierda en <code>b</code> posiciones. Multiplica por 2 el número <code>b</code> de veces.
>>	<code>a >> b</code>	Desplazamiento de <code>a</code> a la derecha en <code>b</code> posiciones, tiene en cuenta el signo. Divide por 2 el número <code>b</code> de veces.
>>>	<code>a >>> b</code>	Desplazamiento de <code>a</code> a la derecha en <code>b</code> posiciones, no tiene en cuenta el signo. (simplemente agrega ceros por la izquierda)
&	<code>a & b</code>	Operación AND a nivel de bits
	<code>a b</code>	Operación OR a nivel de bits
^	<code>a ^ b</code>	Operación XOR a nivel de bits
~	<code>~a</code>	Complemento de <code>A</code> a nivel de bits



Operadores de desplazamiento

Consulta el [Ejemplo09 — Operadores de desplazamiento](#) para ver ejemplos detallados con bits.

OPERADOR CONDICIONAL O TERNARIO ?:

El **operador condicional** `?:` sirve para evaluar una condición y devolver un resultado en función de si es verdadera o falsa dicha condición. Es el único operador ternario de Java, y como tal, necesita tres operandos para formar una expresión.

El primer operando se sitúa a la izquierda del símbolo de interrogación, y siempre será una expresión booleana, también llamada **condición**. El siguiente operando se sitúa a la derecha del símbolo de interrogación y antes de los dos puntos, y es el **valor** que devolverá el operador condicional **si la condición es verdadera**. El último operando, que aparece después de los dos puntos, es la expresión cuyo **resultado se devolverá si la condición evaluada es falsa**.



Operador ternario

Consulta el [Ejemplo10 — Operador ternario](#) para ver cómo funciona el operador `?:`.

El operador condicional se puede sustituir por la sentencia `if...then...else` que veremos más adelante.

PREVALENCIA DE OPERADORES

Los operadores tienen diferente **Prioridad** por lo que es interesante utilizar paréntesis para controlar las operaciones sin necesidad de depender de la prioridad de los operadores.

Prevalencia de operadores, ordenados de arriba a abajo de más a menos prioridad:

Descripción	Operadores
operadores posfijos	<code>op++ op--</code>
operadores unarios	<code>++op --op +op -op ~ !</code>
multiplicación y división	<code>* / %</code>
suma y resta	<code>+ -</code>
desplazamiento	<code><< >> >>></code>

Descripción	Operadores
operadores relacionales	< > <= >=
equivalencia	== !=
operador AND	&
operador XOR	^
operador OR	
AND booleano	&&
OR booleano	
condicional	?:
operadores de asignación	= += -= *= /= %= &= ^= \ = <<= >>= >>>=



Prevalencia de operadores

Consulta el [Ejemplo11 — Prevalencia de operadores](#) para ver la diferencia con y sin paréntesis.



Recuerda

"Los paréntesis son como las patatas fritas, cuantas más, mejor!" — Usa paréntesis para controlar explícitamente el orden de las operaciones y evitar depender de la tabla de precedencia, que es fácil de olvidar.

Conversiones de tipo.



Atención

Las conversiones de tipo pueden provocar **pérdida de información**. Una conversión implícita (automática) solo es segura cuando el tipo destino tiene mayor precisión. Las conversiones explícitas (cast) son forzadas por el programador y deben usarse con cuidado.

CONVERSIONES IMPLÍCITAS

Las **Conversiones Implícitas** se realizan de forma automática y requiere que la variable destino tenga más precisión que la variable origen para poder almacenar el valor.



Conversiones de tipo

Consulta el [Ejemplo12 — Conversiones de tipo](#) para ver ejemplos de conversión implícita y explícita.

Comentarios.

Los comentarios son muy importantes a la hora de describir qué hace un determinado programa. A lo largo de la unidad los hemos utilizado para documentar los ejemplos y mejorar la comprensión del código. Para lograr ese objetivo, es normal que cada programa comience con unas líneas de comentario que indiquen, al menos, una breve descripción del programa, el autor del mismo y la última fecha en que se ha modificado.



Recomendación

Documenta siempre tu código con comentarios. Cada programa debe incluir al menos: breve descripción de lo que hace, autor y fecha de última modificación. Java soporta tres tipos: `//` (una línea), `/* ... */` (multilínea), y `/** ... */` (Javadoc para documentación automática).

Todos los lenguajes de programación disponen de alguna forma de introducir comentarios en el código. En el caso de Java, nos podemos encontrar los siguientes tipos de comentarios:

- Comentarios de **una sola línea**. Utilizaremos el delimitador `//` para introducir comentarios de sólo una línea. Consulta el [Ejemplo13 — Comentarios](#) para ver ejemplos de cada tipo.

2.1.5 Herramientas útiles para empezar

Generar números aleatorios.

Podemos generar números aleatorios entre 0 y 1 utilizando el método `random` de la clase `Math`.



Generación de números aleatorios

Consulta el [Ejemplo14 — Números aleatorios](#) para ver el código completo.

Introducir un texto desde el teclado.



Importante

`System.console().readLine()` **no funciona en la mayoría de IDEs** (Eclipse, IntelliJ, NetBeans). Para entrada de datos en entornos de desarrollo, usa la clase `Scanner` (`java.util.Scanner`), que veremos en la siguiente unidad.

Podemos introducir texto desde el teclado utilizando `System.console().readLine();`



Introducción de datos por teclado

Consulta el [Ejemplo15 — Entrada por teclado](#) para ver el código completo.

2.1.6 Ejemplos UD01

[Descarga el código fuente completo de los ejemplos](#)

Ejemplo01

Programa mínimo en Java que muestra un mensaje por consola (Hola Mundo).

```
1 public class Holamundo {
2     // programa Hola Mundo
3     public static void main(String[] args) {
4         /* lo único que hace este programa es mostrar
5          la cadena "Hola Mundo!" por pantalla */
6         System.out.println("Hola Mundo!");
7     }
8 }
```

Compilación y ejecución:

```
1 $ javac Holamundo.java
2 $ java Holamundo
3 Hola Mundo!
```

[↑ Volver a teoría](#)

Ejemplo02

Ciclo completo de creación, compilación y ejecución de un programa Java.

```

1  /* Ejemplo Hola Mundo */
2  public class Ejemplo {
3      public static void main(String[] args) {
4          System.out.println("Hola Mundo");
5      }
6  }

```

Compilación:

```

1  $ javac Ejemplo.java

```

Ejecución:

```

1  $ java Ejemplo
2  Hola Mundo

```

[↑ Volver a teoría](#)

Ejemplo03

Declaración y uso de un tipo `enum`.

```

1  public class tiposEnumerados {
2      public enum dias {Lunes, Martes, Miercoles, Jueves, Viernes, Sábado, Domingo};
3
4      public static void main(String[] args) {
5          dias diaActual = dias.Martes;
6          dias diaSiguiente = dias.Miercoles;
7
8          System.out.print("Hoy es: ");
9          System.out.println(diaActual);
10         System.out.println("Mañana\nes\n"+diaSiguiente);
11     }
12 }

```

Salida:

```

1  Hoy es: Martes
2  Mañana
3  es
4  Miercoles

```

[↑ Volver a teoría](#)

Ejemplo04

Operaciones aritméticas básicas con `double`.

```

1  double num1, num2, suma, resta, producto, division, resto;
2  num1 = 8;
3  num2 = 5;
4  suma = num1 + num2;      // 13
5  resta = num1 - num2;     // 3
6  producto = num1 * num2;  // 40
7  division = num1 / num2;  // 1.6
8  resto = num1 % num2;     // 3

```

[↑ Volver a teoría](#)

Ejemplo05

Comparaciones entre valores enteros.

```

1  int valor1 = 10;
2  int valor2 = 3;
3  boolean compara;
4  compara = valor1 > valor2; // true
5  compara = valor1 < valor2; // false
6  compara = valor1 >= valor2; // true
7  compara = valor1 <= valor2; // false
8  compara = valor1 == valor2; // false
9  compara = valor1 != valor2; // true

```

[↑ Volver a teoría](#)

Ejemplo06

Operaciones lógicas con `&&`, `||`, `!` y `^`.

```

1  double sueldo = 1400;
2  int edad = 34;
3  boolean logica;
4  logica = (sueldo>1000 & edad<40); //true
5  logica = (sueldo>1000 && edad >40); //false
6  logica = (sueldo>1000 | edad>40); //true
7  logica = (sueldo<1000 || edad >40); //false
8  logica = !(edad <40); //false
9  logica = (sueldo>1000 ^ edad>40); //true
10 logica = (sueldo<1000 ^ edad>40); //false

```

[↑ Volver a teoría](#)

Ejemplo07

Incremento y decremento con prefijo y sufijo.

```

1  int m = 5, n = 3;
2  m++; // 6
3  n--; // 2
4
5  int a = 1, b;
6  b = ++a; // a vale 2 y b vale 2
7  b = a++; // a vale 3 y b vale 2

```

[↑ Volver a teoría](#)

Ejemplo08

Operadores compuestos de asignación.

```

1  int dato1 = 10, dato2 = 2, dato;
2  dato = dato1; // dato vale 10
3  dato2 *= dato1; // dato2 vale 20
4  dato2 /= dato1; // dato2 vale 2
5  dato2 += dato1; // dato2 vale 12
6  dato2 -= dato1; // dato2 vale 2
7  dato1 %= dato2; // dato1 vale 0

```

[↑ Volver a teoría](#)

Ejemplo09

Desplazamiento de bits a nivel binario.

```

1 int j = 33;
2 int k = j << 2;
3 // 000000000000000000000000000000000000000000000000000 : j = 33
4 // 000000000000000000000000000000000000000000000000000 : k = 33 << 2 ; k = 132
5
6 int o = 132;
7 int p = o >> 2;
8 // 000000000000000000000000000000000000000000000000000 : o = 132
9 // 000000000000000000000000000000000000000000000000000 : p = 132 >> 2 ; p = 33
10
11 int x = -1;
12 int y = x >>> 2;
13 // 111111111111111111111111111111111111111111111111111 : x = -1
14 // 001111111111111111111111111111111111111111111111111 : y = x >>> 2; y = 1073741823
15
16 int q = 132; // q: 000000000000000000000000000000000000000000000000000
17 int r = 144; // r: 000000000000000000000000000000000000000000000000000
18
19 int s = q & r; // s: 000000000000000000000000000000000000000000000000000 = 128
20 int t = q | r; // t: 000000000000000000000000000000000000000000000000000 = 148
21 int u = q ^ r; // u: 000000000000000000000000000000000000000000000000000 = 20
22 int v = ~q; // v: 111111111111111111111111111111111111111111111111111 = -133

```

[↑ Volver a teoría](#)

Ejemplo10

Uso del operador condicional `?:` para elegir entre dos valores.

```
1 int mayor, exp1 = 15, exp2 = 25;
2 mayor = (exp1 > exp2) ? exp1 : exp2;
3 // mayor valdrá 25
```

[↑ Volver a teoría](#)

Ejemplo11

Diferencia en el resultado según el uso de paréntesis.

```
1  int x, y1 = 6, y2 = 2, y3 = 8;
2  x = y1 + y2 * y3; // 22 (por prevalencia de *)
3  x = (y1 + y2) * y3; // 64
```

[↑ Volver a teoría](#)

Ejemplo12

Conversión implícita (automática) y explícita (cast).

```
1 // Conversi3n Impl3cita
2 byte origen = 5;
3 short destino;
4 destino = origen; // 5
5
6 // Conversi3n Expl3cita
7 short origen2 = 3;
8 byte destino2;
9 destino2 = (byte) origen2; // 3
```

[↑ Volver a teoría](#)

Ejemplo13

Los tres tipos de comentarios en Java.

```
1 // comentario de una sola línea
2 byte estoEsUnByte = 1;
3
4 /* Esto es un
5 comentario
6 de varias líneas */
7
8 /** Comentario de documentación.
9  * Javadoc extrae los comentarios del código y
10  * genera un archivo html a partir de este tipo de comentarios
11  */
```

[↑ Volver a teoría](#)

Ejemplo14

Generación de números aleatorios con `Math.random()`.

```
1 double numero;
2 int entero;
3 numero = Math.random();
4 System.out.println("El número es: " + numero); //entre 0 y 0.999...
5
6 numero = Math.random() * 100;
7 System.out.println("El número es: " + numero); //entre 0 y 99.999...
8
9 entero = (int)(Math.random() * 100);
10 System.out.println("El número sin decimales es: " + entero); //entre 0 y 99
11
12 int lado = ((int)(Math.random() * 6)) + 1;
13 char letra = (char)((Math.random() * 26) + 65); //65..90
14 System.out.println(letra); //A..Z
```

[↑ Volver a teoría](#)

Ejemplo15

Lectura de datos desde la consola con `System.console().readLine()`.

```
1 // Ejemplo 1: Introducción de texto
2 String texto;
3 System.out.print("Introduce un texto: ");
4 texto = System.console().readLine();
5 System.out.println("El texto introducido es: " + texto);
6
7 // Ejemplo 2: Introducción de un número entero
8 String texto2;
9 int entero2;
10 System.out.print("Introduce un número: ");
11 texto2 = System.console().readLine();
12 entero2 = Integer.parseInt(texto2);
13 System.out.println("El número introducido es: " + entero2);
14
15 // Ejemplo 3: Introducción de un número decimal
16 String texto3;
17 double doble3;
18 System.out.print("Introduce un número decimal: ");
19 texto3 = System.console().readLine();
20 doble3 = Double.parseDouble(texto3);
21 System.out.println("Número decimal introducido es: " + doble3);
```

[↑ Volver a teoría](#)

2.1.7 Resumen — Conceptos clave

Concepto	Definición
Problema	Situación que requiere una solución mediante métodos y estrategias
Algoritmo	Conjunto de pasos finitos y ordenados para resolver un problema
Programa	Algoritmo codificado en un lenguaje de programación
JVM	Máquina Virtual Java que ejecuta el bytecode

Concepto	Definición
Variable	Zona de memoria con nombre que almacena un valor
Tipo de dato	Categoría de valor que puede tomar una variable
Operador	Símbolo que realiza una operación sobre uno o más operandos

2.1.8 Autoevaluación

- ✓ Comprendo qué es un problema, un algoritmo y un programa
- ✓ Sé compilar y ejecutar un programa Java
- ✓ Conozco los tipos de datos primitivos de Java
- ✓ Puedo declarar variables y constantes
- ✓ Utilizo correctamente los operadores aritméticos, relacionales y lógicos
- ✓ Entiendo la diferencia entre conversión implícita y explícita

2.1.9 Vídeos recomendados

Canal	Vídeo	Contenido
Píldoras Informáticas	Curso Java 2026 — Presentación (vídeo 1)	Presentación del curso completo
Píldoras Informáticas	Curso Java — Estructuras principales VII — Clase Math II (vídeo 10)	Clase Math y operaciones matemáticas
Programación ATS	Playlist completa del curso Java	Vídeos 1-9: introducción, Hola Mundo, tipos de datos, entrada/salida
MoureDev	Curso Completo de Java desde Cero	8 horas — cubre todos los temas del curso
DiscoDurodeRoer	Curso Java SE — playlist	Instalación, Hola Mundo, variables, tipos

🕒 21 de julio de 2026

2.2 Ejercicios de la UD01

2.2.1 Retos

1. (Reto1) Haga un programa que evalúe una expresión que contenga literales de los cuatro tipos de datos (booleano, entero, real y carácter) y la muestre por pantalla.
2. (Reto2) En su entorno de trabajo, cree el programa siguiente. Obsérvese que pasa exactamente. Entonces, intente arreglar el problema.

```
1 // Un programa que usa un entero muuuuy grande
2 public class TresMilMillions {
3     public static void main (String [] args) {
4         System.out.println (3000000000);
5     }
6 }
```

3. (Reto3) Haga un programa con dos variables que, sin usar ningún literal ninguna parte excepto para inicializar estas variables, vaya estimando e imprimiendo sucesivamente los 5 primeros valores de la tabla de multiplicar del 4. Puede usar operadores aritméticos y de asignación, si desea.
4. (Reto4) Haga dos programas, uno que muestre por pantalla la tabla de multiplicar del 3, y otro, la del 5. Los dos deben ser exactamente iguales, letra por letra, excepto en un único literal dentro de todo el código.
5. (Reto5) Experimente qué pasa si en el siguiente programa inicializa la variable `reallLargo` con un valor con varios decimales. El programa continúa compilando? ¿Qué resultado da? Después inténtelo asignando un valor superior al rango de los enteros (por ejemplo, 3000000000.0).

```
1 public class ConversionExplicita {
2     public static void main (String[] args) {
3         double reallLargo = 3000000000.0;
4         // Asignación incorrecta. ¿Un real tiene decimales, no?
5         long enterLargo = (long) reallLargo;
6         // Asignación incorrecta. ¿Un entero largo tiene un rango mayor que un entero, no?
7         int enter = (int) enterLargo;
8         System.out.println (enter);
9     }
10 }
```

6. (Reto6) Haga un programa que muestre en pantalla de forma tabulada la tabla de verdad de una expresión de disyunción entre dos variables booleanas.
7. (Reto7) Haga un programa que muestre por pantalla la multiplicación de tres números reales entrados por teclado.

2.2.2 Ejercicios

Importante

`Scanner` solo se puede usar en esta actividad porque no se ha explicado en profundidad en este tema. No lo confundas con `System.console().readLine()`, que no funciona en la mayoría de IDEs.

1. (Ejs1) Probar la E/S elemental: Escribe el pequeño programa que aparece a continuación.

```
1 import java.util.*;
2 public class EntradaSalida {
3     public static void main (String arg[]){
4         Scanner tec = new Scanner(System.in);
5         int a, b;
6         System.out.println("Introduce un número entero");
7         a = tec.nextInt();
8         System.out.println("Introduce otro número entero");
9         b = tec.nextInt();
10        System.out.println("Los números introducidos son " + a + " y " + b);
11    }
12 }
```

Ejecútalo para ver como se comporta el programa.

¿Qué ocurre si cuando nos pide un número entero le damos un número real? ¿Y si le damos un carácter no numérico?

¿Qué ocurre si eliminamos la instrucción `import java.util.*;`

2. (Ejs2) Averigua mediante pruebas:

- ¿Es posible escribir dos instrucciones en la misma línea de un programa?
- ¿Se puede "romper" una instrucción entre varias líneas?
- Algunos lenguajes de programación dan un valor por defecto a las variables cuando las declaramos sin inicializarlas. Otros no permiten usar el contenido de una variable que no haya sido previamente inicializada. ¿Cuál es comportamiento de Java?

3. (Esj3) ¿Cuáles de los siguientes identificadores son válidos y cuales no? Pruébalos cuando tengas duda

- n
- MiProblema
- MiJuego
- Mi Juego
- Int
- Jose&Co
- A b
- 1rApellido
- aaaaaaaaaaaa
- NombreApellidos
- Saldo-actual
- Universidad Alicante
- Juan=Rubio
- Edad5
- _5Java
- true
- _false
- f_false

4. (Por2) Escribir un programa que lea un entero desde teclado, lo multiplique por 2, y a continuación escriba el resultado en la pantalla:

Ejemplo de ejecución:

```
1  Escribe un número:
2  3
3  El doble de 3 es 6
```

5. (Intercambio) Escribir un programa que ...

- Lea desde teclado dos valores de texto. Llama a las variables s1 y s2.
- Muestre los valores introducidos por el usuario
- Intercambie el valor de s1 y s2 (s1 pasa a valer lo que valía s2 y viceversa)
- Muestre de nuevo los valores, ahora con su valor intercambiado

Ejemplo de ejecución:

```
1  Escribe un texto para s1: David
2  Escribe un texto para s2: Maria
3  Antes de intercambiar    s1: David y    s2: Maria
4  Después de intercambiar  s1: Maria y    s2: David
```

6. (ExpresionesMatematicas) Escribir las siguientes expresiones siguiendo la sintaxis de Java.

- $\frac{x}{y} + 1$
- $\frac{x+y}{x-y}$
- $\left[\frac{b}{c+d} \right]$
- $(a + b)^2$
- $\frac{x + \frac{y}{z}}{x - \frac{y}{z}}$
- $\frac{xy}{1-4zx}$

g. $(a + b) \frac{c}{d}$

h. $\frac{xy}{mn}$

7. (Superficie) Escribir un programa que solicite al usuario la longitud y la anchura de una habitación y a continuación muestre su superficie (longitud por anchura).
8. (Medidas) Escribir un programa que convierta una medida dada en pies a sus equivalentes en yardas, pulgadas, centímetros y metros, sabiendo que 1 pie = 12 pulgadas, 1 yarda = 3 pies, 1 pulgada = 2.54 cm, 1 m = 100 cm.
9. (Segundos) Escribir un programa que, dada una cantidad de segundos, introducida por teclado, la desglose en días, horas, minutos y segundos.

Ejemplo de ejecución:

```
1 Introduce cantidad de segundos: 3661
2 3661 segundos son:
3 0 días
4 1 horas
5 1 minutos
6 1 segundos
```

10. (Fuerza) La fuerza de atracción entre dos masas m_1 y m_2 separadas por una distancia d , está dada por la fórmula:

$$F = \frac{(G \cdot m_1 \cdot m_2)}{d^2}$$

donde G es la constante de gravitación universal $G = 6.67430 \cdot 10^{-11}$.

Escribir un programa que lea la masa de dos cuerpos y la distancia entre ellos y a continuación obtenga su fuerza de atracción.

11. (Círculo) Escribir un programa que calcule la longitud de la circunferencia y el área del círculo para un valor del radio introducido por teclado.
12. (Dados) Escribir un programa que simula el lanzamiento de dos dados.

```
1 Dado 1 : 5
2 Dado 2 : 4
3 Puntuación total: 9
```

13. (UltimaCifra) Escribir un programa que muestre la última cifra de un número entero que introduce el usuario por teclado.

Pista: ¿Qué devuelve `a%10` ?

```
1 Introduce un número entero: 3761
2 La última cifra de 3761 es 1
```

14. (PenultimaCifra) Escribir un programa que muestre la penúltima cifra de un número entero que introduce el usuario por teclado.

```
1 Introduce un número entero: 3761
2 La penúltima cifra de 3761 es 6
```

Una vez hayas comprobado que el programa funciona correctamente, prueba qué ocurre si el usuario introduce un valor de una sola cifra (por ejemplo 4). Explica el resultado mostrado por el programa.

15. (Redondear1) `Math.round(x)` redondea x de manera que este queda sin decimales. (`Math.round(35.5289)` da como resultado 36)

Trata de escribir un programa en el que el usuario introduzca un número real y a continuación se muestre redondeado a un decimal. Pista : combinar productos, divisiones y `Math.round()`

Ejemplo de ejecución:

```
1 Introduce un número real: 35.5289
2 El número 35.5289, redondeado a un decimal es 35.5
```

16. (ExpresionesAritmeticas) 16. Cuál es el valor resultante de dada una de las siguientes expresiones

a. $5 * 4 - 3 * 6$

b. $4 * 5 * 2$

c. $(24 + 2 * 6) / 4$

d. $8 / 2 / 2 * 5$

- e. `3 + 4 * (8 * (4 - (9 + 3) / 6))`
- f. `4 * 3 * 5 + 8 * 4 * 2`
- g. `4 - 40 % 5`
- h. `4 * 3 / 2`
- i. `4 / 2 * 3`
- j. `213 / 100`

17. (Einstein) La famosa ecuación de Einstein para la conversión de una masa m en energía viene dada por la fórmula $E=mc^2$, donde c es la velocidad de la luz que vale $2.997925 \cdot 10^8$ m/s. Escribir un programa que lea el valor de la masa y obtenga la energía correspondiente según la anterior fórmula.
18. (FragmentosCodigo) Indica cuales serán los valores de las variables después de ejecutar cada uno de los siguientes fragmentos de código. Resuelve el ejercicio sin escribir los programas correspondientes y probarlos.

- a. `java int a=3, b = 2; a = b + b; b = a + a;`
- b. `java int a=3,b=0; b = b - 1; a = a + b;`
- c. `java int a, b=5; b++; ++b; a= b+1;`
- d. `java int a = 5,b; b = a++;`
- e. `java int a = 5,b; b = ++a;`
- f. `java int a=2, b=3; b+=a;`
- g. `java int a=2, b=3; b-=a; a=-b;`
- h. `java int a=2, b=3; b%=a;`
- i. `java int a=2,b=3,c=4; a = --b + c++; b+=a;`

2.2.3 Expresiones Lógicas

1. Sean 4 variables enteras:

```
1 int m, j, p, v ;
```

que contienen respectivamente la edad de Miguel, Julio, Pablo y Vicente.

Expresar las siguientes afirmaciones utilizando operadores lógicos y relacionales

Ejemplo: Miguel es mayor de edad.

Solución: `m >= 18`

- a. (Logica1) Miguel es menor de edad.
- b. (Logica2) Miguel es mayor que Julio
- c. (Logica3) Miguel es el más viejo.
- d. (Logica4) Miguel es el más joven.
- e. (Logica5) Miguel no es el más joven.
- f. (Logica6) Miguel no es el más viejo.
- g. (Logica7) Alguno de ellos es mayor de edad.
- h. (Logica8) Miguel y Julio son los más jóvenes.
- i. (Logica9) Entre todos tienen más de 100 años.
- j. (Logica10) Entre Miguel y Julio suman más edad que Pablo.
- k. (Logica11) Entre Miguel y Julio suman más edad que Pablo y Vicente juntos.
- l. (Logica12) Si los ordenamos por edades de menor a mayor, Julio es el segundo.
- m. (Logica13) Si los ordenamos por edades de menor a mayor, Julio es el segundo y Pablo el tercero.
- n. (Logica14) Al menos uno de ellos es menor de edad.
- o. (Logica15) Al menos dos de ellos son menores de edad.
- p. (Logica16) Todos son menores de edad.
- q. (Logica17) Solo dos de ellos son menores de edad.
- r. (Logica18) Al menos dos de ellos nacieron el mismo año.
- s. (Logica19) Solo dos de ellos nacieron el mismo año.

- t. (Logica20) Al menos uno de ellos es menor que Julio
- u. (Logica21) Solo uno de ellos es menor que Julio
- v. (Logica22) Miguel es mayor de edad y alguno de los otros es menor de edad.

2.2.4 Actividades

1. (Actividad1) Realiza un conversor de euros a pesetas. La cantidad de euros que se quiere convertir debe ser introducida por teclado.
2. (Actividad2) Realiza un conversor de pesetas a euros. La cantidad de pesetas que se quiere convertir debe ser introducida por teclado.
3. (Actividad3) Escribe un programa que calcule el área de un rectángulo. ($\text{area} = \text{base} * \text{altura}$)
4. (Actividad4) Escribe un programa que calcule el área de un triángulo. ($\text{area} = (\text{base} * \text{altura}) / 2$)
5. (Actividad5) Escribe un programa que calcule el salario semanal de un empleado en base a las horas trabajadas, a razón de 12 euros la hora.
6. (Actividad6) Realiza un conversor de MiB a KiB. [Ayuda](#)
7. (Actividad7) Realiza un conversor de Kib a Mib. [Ayuda](#)
8. (Actividad8) Realiza un programa en Java que genere letras de forma aleatoria.
9. (Actividad9) Realiza un programa en Java que genere el número premiado del Cupón de la ONCE.
10. (Actividad10) Modificar el siguiente programa para que compile y funcione:

```

1 public class Activ10 {
2     public static void main(String[] args) {
3         int n1 = 50, int n2 = 30,
4         boolean suma = 0;
5         suma = n1 + n2;
6         System.out.println("LA SUMA ES: " + suma);
7     }
8 }

```

11. (Actividad11) Modificar el siguiente programa para que compile y funcione:

```

1 public class Activ11 {
2     public static void main(String[] args) {
3         int numero = 2;
4         cuad = numero * número;
5         System.out.println("EL CUADRADO DE "+NUMERO+" ES: "+cuad);
6     }
7 }

```

12. (Actividad12) Indicar que valor devolverá la ejecución del siguiente programa:

```

1 public class Activ12 {
2     public static void main(String[] args) {
3         int num = 5;
4         num += num - 1 * 4 + 1;
5         System.out.println(num);
6     }
7 }

```

13. (Actividad13) Indicar que valor devolverá la ejecución del siguiente programa:

```

1 public class Activ13 {
2     public static void main(String[] args) {
3         int num = 4;
4         num %= 7 * num % 3 * 3;
5         System.out.println(num);
6     }
7 }

```

14. (Actividad14) Realizar un programa que muestre por pantalla respetando los saltos de carro el siguiente texto (con un solo `println`):

```

1 Me gusta la programación
2 cada día más

```

15. (Actividad15) Realiza un programa en Java que tenga las variables edad, nivel de estudios e ingresos y almacene en una variable llamada jasp el valor verdadero si la edad es menor o igual a 28 y el nivel de estudios es mayor a 3, o bien la edad es menor de 30 y los ingresos superiores a 28000. En caso contrario almacenar el valor falso.
16. (Actividad16) Realizar un programa que realice el cálculo del precio de un producto teniendo en cuenta que el producto vale 120 €, tiene un descuento del 15% y el IVA que se le aplica es del 21%.
17. (Actividad17) Realiza un programa que calcule la nota que hace falta sacar en el segundo examen de la asignatura Programación para obtener la media deseada. Hay que tener en cuenta que la nota del primer examen cuenta el 40% y la del segundo examen un 60%. Ejemplo 1:

```
1 Introduce la nota del primer examen: 7
2 ¿Qué nota quieres sacar en el trimestre? 8.5
3 Para tener un 8.5 en el trimestre necesitas sacar un 9.5 en el segundo examen.
```

Ejemplo 2:

```
1 Introduce la nota del primer examen: 8
2 ¿Qué nota quieres sacar en el trimestre? 7
3 Para tener un 7 en el trimestre necesitas sacar un 6.333333333 en el segundo examen.
```

18. (Actividad18) Realizar un programa que dado un importe en euros nos indique el mínimo número de billetes y la cantidad sobrante de euros. Debes usar el operador condicional `?:`

```
1 ¿Cuántos euros tienes?: 232
2 1 billete de 200 €
3 1 billete de 20 €
4 1 billete de 10 €
5 Sobran 2 €
```

🕒 19 de julio de 2026

2.3 Talleres

2.3.1 Taller UD01_01: Instalar NoMachine para el control remoto

¿Qué es NoMachine ?

Conéctese a cualquier computadora de forma remota a la velocidad de la luz. Gracias a nuestra tecnología NX, NoMachine es el escritorio remoto más rápido y de mayor calidad que jamás haya probado. Conecta con tu ordenador al otro lado del mundo con solo unos pocos clics. Vé donde esté tu escritorio, podrás acceder a él desde cualquier otro dispositivo y compartirlo con quien quieras. NoMachine es tu servidor personal, privado y seguro. Además, es gratis.

<https://www.nomachine.com/>

DESCARGA E INSTALA LA APLICACIÓN

Desde la página de descargas:

<https://downloads.nomachine.com/>

E instala la aplicación en tu PC.

Permisos al profesor

Debemos conceder permisos para que el profesor se pueda conectar a nuestro PC mientras estemos en el instituto sin necesidad de contraseña. Esto solo será posible cuando estemos conectados a la red del instituto, y el profesor no podrá acceder cuando estemos en casa.

Agrega la clave SSH pública (es un fichero `authorized.crt` que te proporcionará el profesor a través de AULES) en tu ordenador

- Debes colocarla en la carpeta `<Inicio del usuario>/.nx/config`.
- Cree este directorio si no existe.
- En Linux y macOS, ejecute en una terminal: `mkdir $HOME/.nx/config`
- En Windows, créelo en (`C:\Users\username\.nx\config`) usando las herramientas del sistema (Explorador de archivos).
- Si la carpeta de configuración ya existe, copia el fichero `authorized.crt` en ella.

!!! tip "Consejo" Ten en cuenta que los navegadores pueden cambiar las extensiones de los archivos, es conveniente tener las opciones de "ver extensiones de archivos" y "ver archivos ocultos" en nuestro gestor de archivos habitual

Tarea

Debes enviar un archivo `*.pdf` a la plataforma de AULES con una simple captura que demuestre que el profesor se ha podido conectar a tu PC.



Obligatorio

Debes mantener NoMachine instalado y permitir las conexiones automáticas por parte del profesor para pedir ayuda y consultar dudas en clase, para corregir las tareas diarias, y para realizar los exámenes.

🕒 19 de julio de 2026

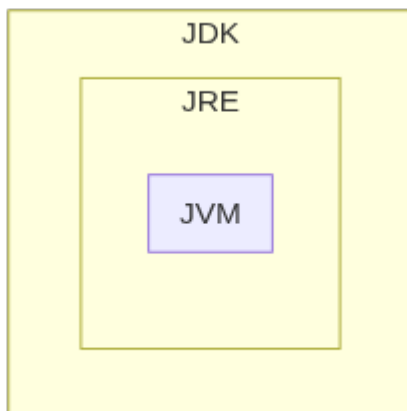
2.3.2 Taller UD01_02: Instalación y uso de entornos de desarrollo

Java

Cada software y cada entorno de desarrollo tiene unas características y funcionalidades específicas. Esto también se verá reflejado en la instalación y configuración del software. Dependiendo de la plataforma, entorno o sistema operativo en el que se vaya a instalar el software, se utilizará un paquete de instalación u otro, y habrá que tener en cuenta unas opciones u otras en su configuración. A continuación se muestra cómo instalar una herramienta de desarrollo de software integrada, como Eclipse. Pero también podrás observar los procedimientos para instalar otras herramientas necesarias o recomendadas para trabajar con el lenguaje de programación JAVA, como Tomcat o la Máquina Virtual de Java. Debes tener en cuenta los siguientes conceptos:

i Definición

- **JVM:** Máquina Virtual Java — ejecuta el bytecode.
- **JRE:** Java Runtime Environment — incluye la JVM + bibliotecas para ejecutar aplicaciones.
- **JDK:** Java Development Kit — incluye el JRE + herramientas de desarrollo (compilador `javac`, etc.).



El proceso de instalación consta de los siguientes pasos: 1. Descargue, instale y configure el JDK. 2. Descargue e instale un servidor web o de aplicaciones. 3. Descargue, instale y configure el IDE (Netbeans o Eclipse). 4. Configurar JDK con IDE. 5. Configure el servidor web o de aplicaciones con el IDE instalado. 6. Si es necesario, instalación de conectores. 7. Si es necesario, instale un nuevo software.

DESCARGUE E INSTALE EL JDK

Podemos diferenciar entre:

- Java SE (Java Standard Edition): es la versión estándar de la plataforma, siendo esta plataforma la base para todos los entornos de desarrollo Java ya sea de aplicaciones cliente, de escritorio o web.
- Java EE (Java Enterprise Edition): esta es la versión más grande de Java y generalmente se utiliza para crear grandes aplicaciones cliente/servidor y para el desarrollo de servicios web.

En este curso se utilizarán las funcionalidades de Java SE. El archivo es diferente según el sistema operativo donde se tenga que instalar. Así:

- Para los sistemas operativos Windows y Mac OS hay un archivo instalable.
- Para los sistemas operativos GNU/Linux que admiten paquetes `.rpm` o `.deb`, también están disponibles paquetes de este tipo.
- Para el resto de sistemas operativos GNU/Linux existe un archivo comprimido (terminado en `.tar.gz`).

En los dos primeros casos, simplemente hay que seguir el procedimiento de instalación habitual del sistema operativo con el que estamos trabajando. En este último caso, sin embargo, hay que descomprimir el archivo y copiarlo en la carpeta donde se desea instalar. Normalmente, todos los usuarios tendrán permisos de lectura y ejecución en esta carpeta.

**Atención**

Desde JDK 11, Oracle cambió la licencia. Para evitar restricciones, usa **OpenJDK** (descargable desde <https://adoptium.net/>), que es gratuito y de código abierto.

Una alternativa es utilizar <https://adoptium.net/> antes conocido como adoptOpenJDK, que ahora se ha integrado en la fundación Eclipse. Desde allí podemos descargar los binarios de la versión openJDK para nuestra plataforma sin restricciones. [Noticia completa] (<https://es.wikipedia.org/wiki/OpenJDK>).

**GNU/Linux**

En GNU/Linux podemos utilizar los comandos:

- `sudo apt install default-jdk` para instalar el jdk predeterminado.
- `java --version` para ver las versiones disponibles en nuestro sistema.
- `sudo update-alternatives --config java` para elegir cuál de las versiones instaladas queremos usar por defecto o incluso ver la ruta de las diferentes versiones que tenemos instaladas.

CONFIGURAR LAS VARIABLES DE ENTORNO "JAVA_HOME" Y "PATH"

Una vez descargado e instalado el JDK, debes configurar algunas variables de entorno:

- La variable `JAVA_HOME`: indica la carpeta donde se ha instalado el JDK. No es obligatorio definirla, pero es muy cómodo hacerlo, ya que muchos programas buscan en ella la ubicación del JDK. Además, resulta muy fácil definir las dos variables siguientes.
- La variable `PATH`. Debe apuntar al directorio que contiene el ejecutable de la máquina virtual. Suele ser la subcarpeta `bin` del directorio donde hemos instalado el JDK.

Variable CLASSPATH Otra variable que tiene en cuenta el JDK es la variable `CLASSPATH`, que apunta a las carpetas donde se encuentran las librerías de la aplicación que se quiere ejecutar con el comando `java`. Es preferible, no obstante, indicar la ubicación de estas carpetas con la opción `-cp` del mismo comando `java`, ya que cada aplicación puede tener diferentes librerías y las variables de entorno afectan a todo el sistema. Establecer la variable `PATH` es esencial para que el sistema operativo encuentre los comandos JDK y pueda ejecutarlos.

Eclipse

Eclipse es una aplicación de código abierto desarrollada actualmente por Eclipse Foundation, una organización independiente, sin fines de lucro, que fomenta una comunidad de código abierto y el uso de un conjunto de productos, servicios, capacidades y complementos para la divulgación del uso de código abierto en el desarrollo de aplicaciones informáticas. Eclipse fue desarrollado originalmente por IBM como sucesor de VisualAge. Como Eclipse está desarrollado en Java, es necesario, para su ejecución, tener un JRE (Java Runtime Environment) previamente instalado en el sistema. Para saber si tienes este JRE instalado, puedes hacer el test en la web oficial de Java, en la sección ¿Tengo Java? Si vamos a desarrollar con Java, como es nuestro caso, deberemos tener instalado el JDK (recordemos que es un superconjunto del JRE).

INSTALACIÓN

Las versiones actuales del entorno Eclipse se instalan con un instalador. Este, básicamente, se encarga de descomprimir, solucionar algunas dependencias y crear los accesos directos. Este instalador se puede obtener descargándolo directamente desde la página oficial del Proyecto Eclipse www.eclipse.org. Podrás encontrar las versiones para los diferentes sistemas operativos e instrucciones para su uso. No son nada complejas. En el caso de GNU/Linux y MAC OS, el archivo es un archivo comprimido, por lo que hay que descomprimirlo y luego ejecutar el instalador. Se trata del archivo `eclipse-inst`, dentro de la carpeta `eclipse`, que es una subcarpeta del resultado de descomprimir el archivo anterior. Si sólo el usuario actual va a utilizar el IDE, la instalación se puede realizar sin utilizar privilegios de administrador o root y seleccionando para la instalación una carpeta perteneciente a este usuario. Si se desea compartir la instalación entre distintos usuarios, se debe indicar al instalador una carpeta sobre la que todos estos usuarios tengan permisos de lectura y ejecución.

Al iniciar el instalador veremos una pantalla similar a esta:



El instalador nos preguntará qué versión queremos instalar. La versión que utilizaremos es “Eclipse IDE for Java EE Developers”.

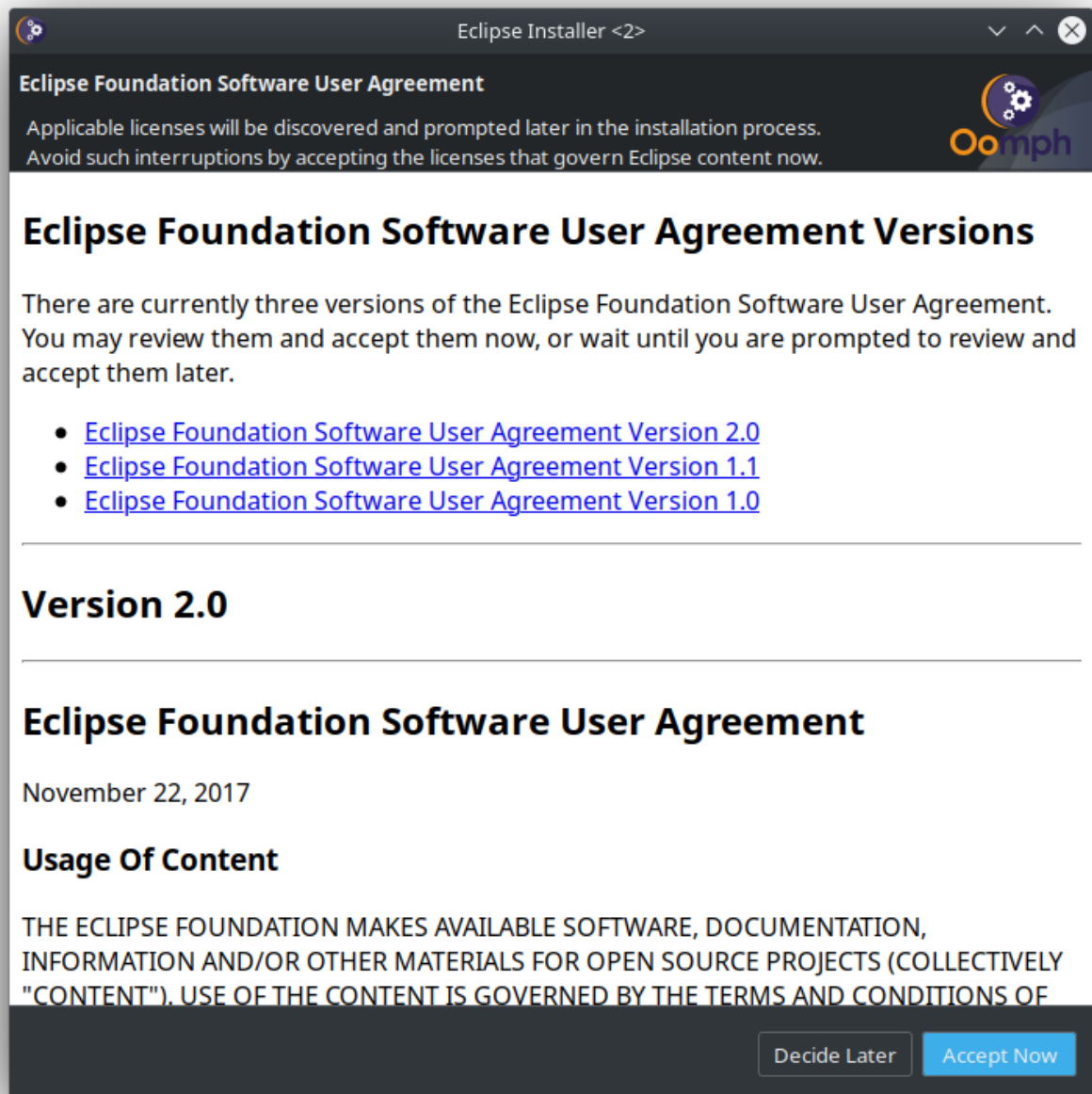


Luego nos pedirá la versión de JDK/JRE que vamos a utilizar (en la captura aparece con letras blancas). También nos pide la carpeta donde la instalaremos. Y dos check boxes para indicar si queremos que nos cree el acceso directo al menú de aplicaciones ya en el escritorio.

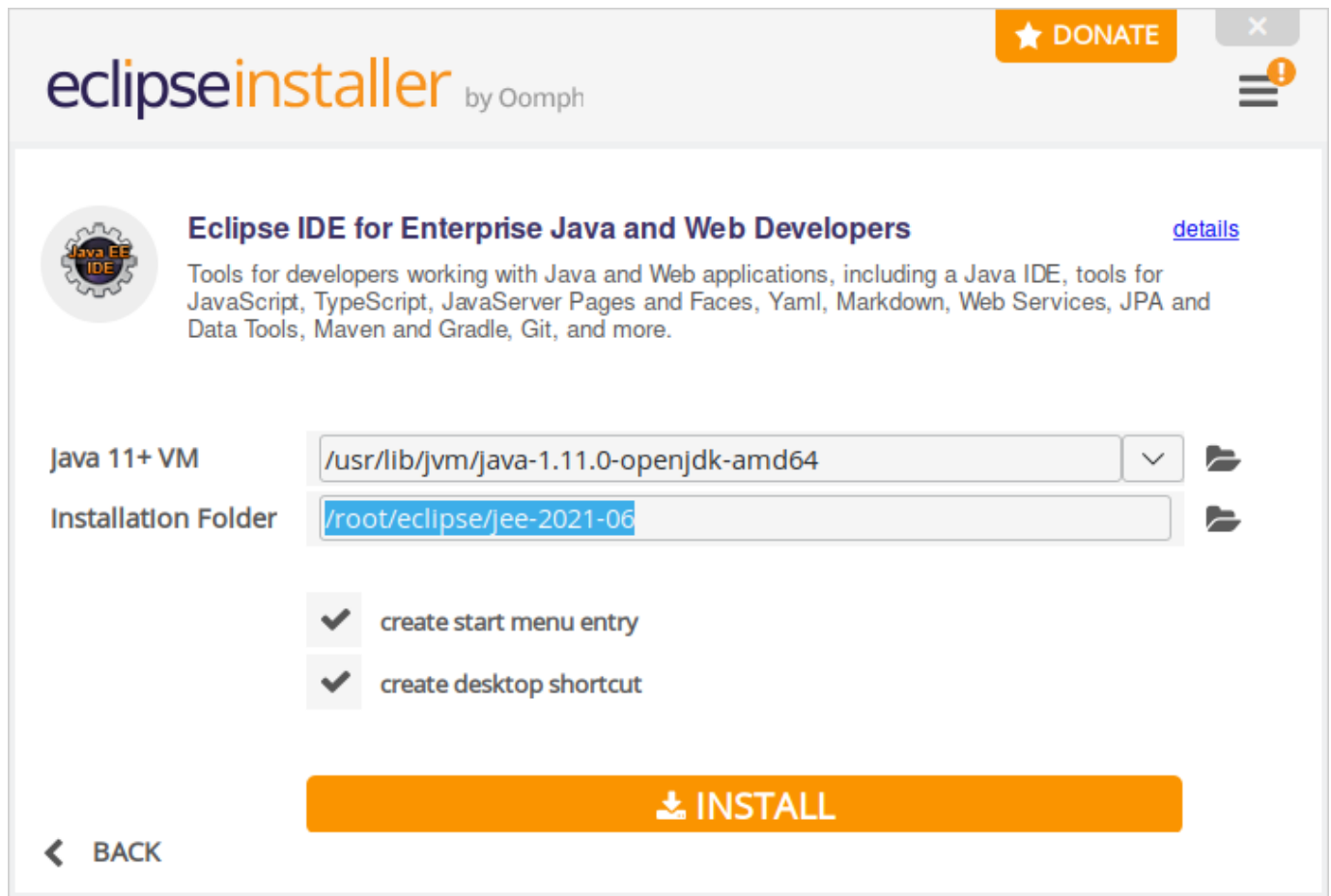


Para seleccionar la carpeta correcta hay que tener en cuenta qué usuarios van a utilizar el entorno. Todos ellos deben tener permisos de lectura y ejecución sobre la carpeta en cuestión. Una vez introducida la carpeta podemos pulsar el botón INSTALAR para iniciar la instalación.

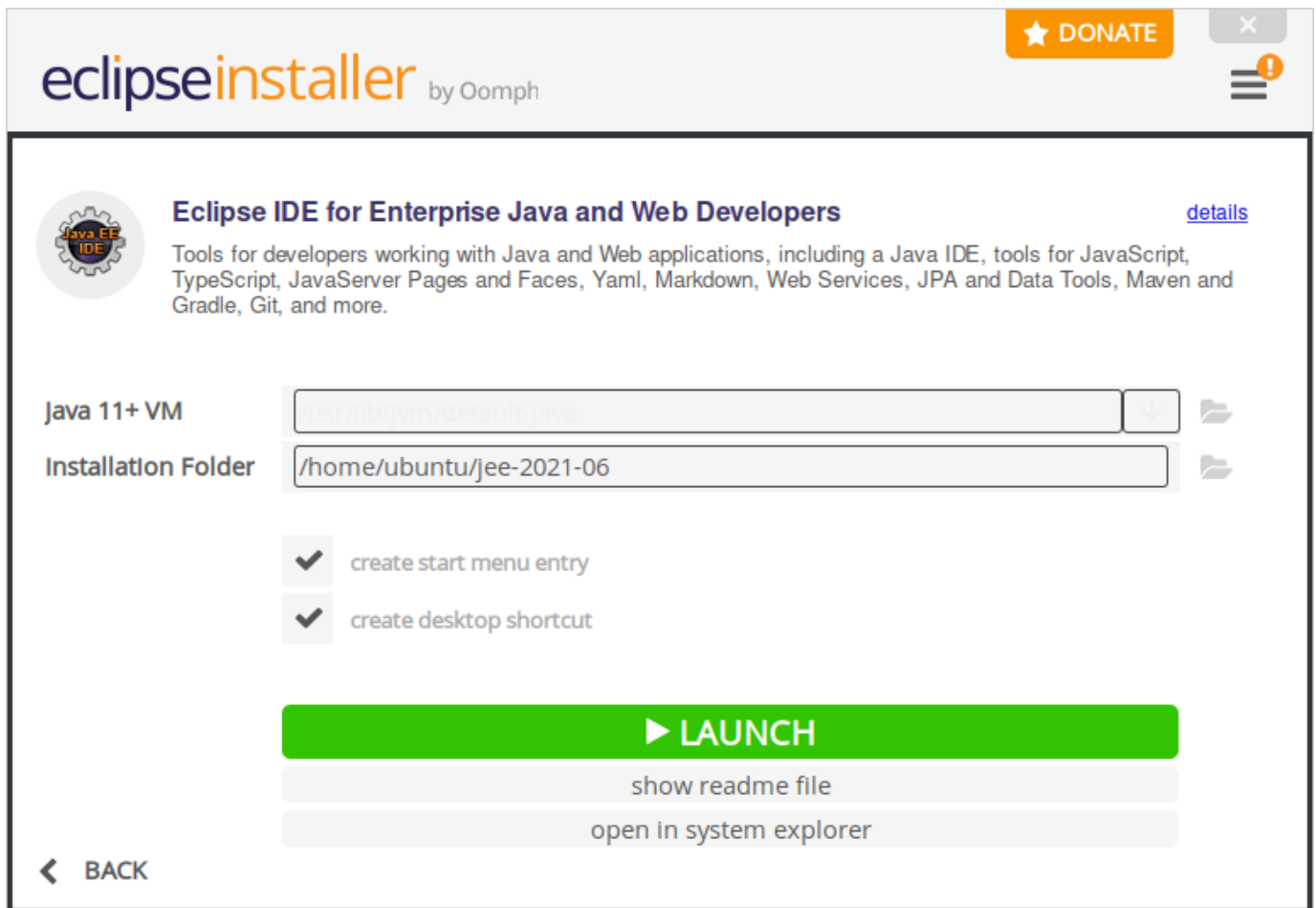
También se nos pedirá que aceptemos las licencias del software a instalar, como muestra la captura de pantalla:



Durante la instalación veremos una pantalla de progreso como la que se muestra a continuación:



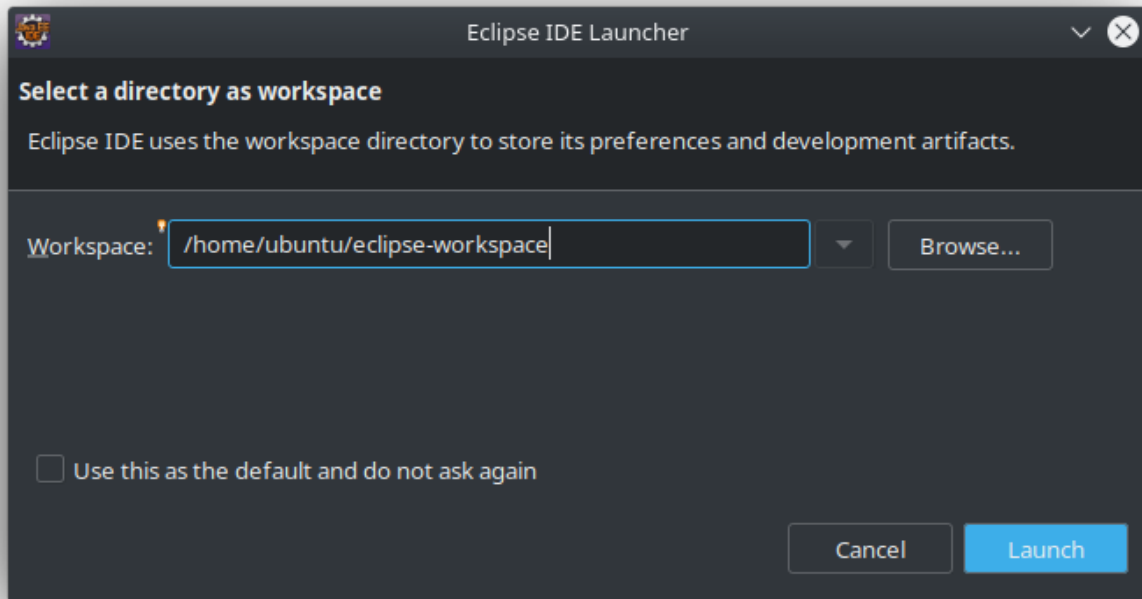
Una vez finalizada la instalación, se nos muestra una pantalla que nos invita a ejecutar directamente el entorno.



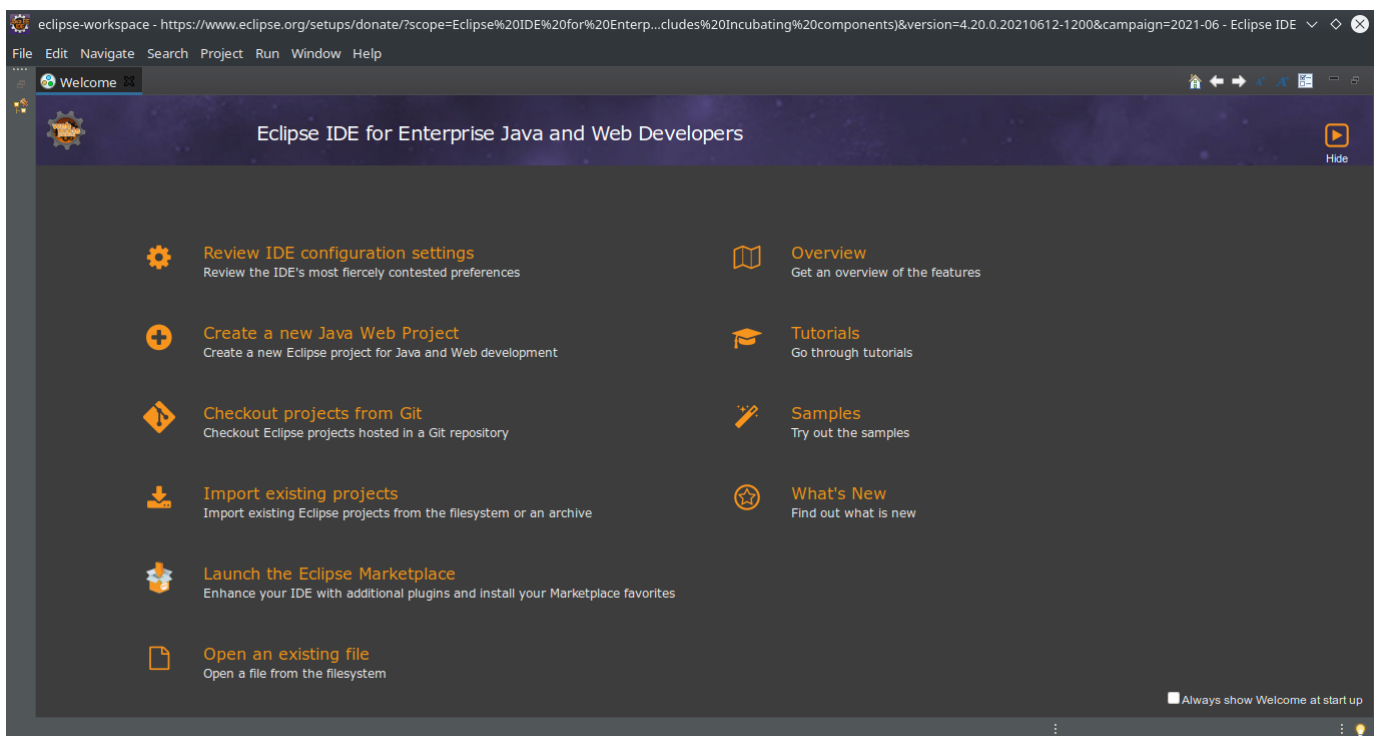
Esta primera vez podremos ejecutar el entorno Eclipse pulsando el botón LAUNCH. El resto de las veces será necesario invocarlo desde los accesos directos o lanzadores, si se han creado o, en caso contrario, invocando directamente el ejecutable. Este se llama eclipse y lo encontrarás en una subcarpeta de la carpeta de instalación también llamada eclipse. La ruta exacta puede variar de una versión a otra. Si en el futuro es necesario desinstalarlo, sólo se debe borrar la carpeta donde ha sido instalado ya que la instalación de Eclipse no aparece en el repositorio de GNU/Linux ni en el panel de control en Windows. Cuando ejecutamos el entorno nos aparecerá una pantalla como la siguiente:



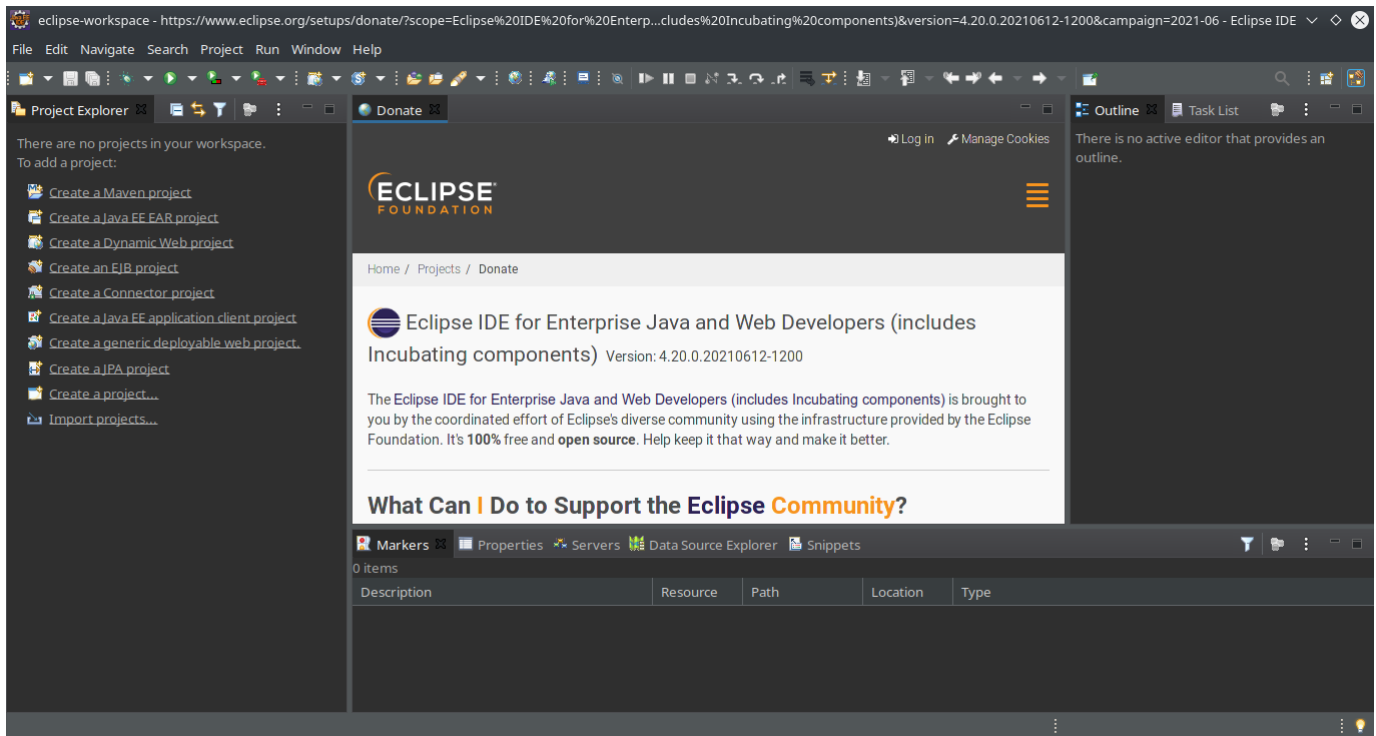
Inmediatamente se nos preguntará en qué carpeta se ubicará el workspace. Podemos pedirle que lo recuerde para el resto de ejecuciones activando la opción “Usar esto como predeterminado y no volver a preguntar”.



La primera vez que lo ejecutemos se mostrará la pestaña de bienvenida. Podemos pedirle que no nos la muestre más desactivando la opción “Mostrar siempre la bienvenida al iniciar”.



Una vez cerrada esta pestaña, el entorno de trabajo será similar a esto:



Nota

Por defecto Eclipse nos ofrece la descarga del instalador más ligero que descargará de Internet los paquetes necesarios para completar la instalación según nuestras elecciones. Si esta instalación nos da problemas, podemos descargar la versión "package" en la que previamente deberemos elegir el paquete de instalación que queramos, ocupará bastante más, pero descargará todos los paquetes necesarios. Después solo tendremos que descomprimir el archivo descargado en una carpeta de nuestra elección y ya tendremos eclipse instalado. Tendremos que crear nuestro propio menú de inicio e iconos del escritorio (podéis seguir esta [guía] (<https://www.donovanbrown.com/post/Añadir-Eclipse-al-Launcher-en-Ubuntu-1604>) cambiando la ruta donde habéis descomprimido vuestra versión de eclipse).

CONFIGURACIÓN

Versión Java

Por defecto Eclipse intenta utilizar las nuevas características del JDK 16, pero en nuestro caso por ejemplo tenemos la versión 11. Podemos personalizar estas opciones en el apartado `Ventana/Preferencias/Java/Compilador` y elegir en el campo `Nivel de conformidad del compilador` la versión correcta, en nuestro caso la 11.

Además, si lo necesitamos, podemos configurar los JDKs que están disponibles, añadirlos o eliminarlos desde la opción `Ventana/Preferencias/Java/JRE instalados`.

Perspectiva

Eclipse llama a la distribución de los paneles en la ventana Perspectiva, hay unos cuantos predefinidos y podemos configurar los nuestros, a nuestro gusto en la sección `Ventana/Perspectiva`.

Apariencia

Eclipse nos permite personalizar cualquier aspecto de la apariencia de nuestro entorno, cambiar tanto el tema del IDE como el tamaño de fuente y los colores para el coloreado del código fuente. Todas estas opciones están disponibles en `Ventana/Apariencia`.

MÓDULOS

Las opciones y funcionalidades de Eclipse se pueden ampliar añadiendo módulos desde su "store" de plugins. En `Help/Eclipse Marketplace...` podemos por ejemplo buscar por texto, o buscar en la pestaña de populares. Eso nos mostrará todos los complementos que contengan la palabra buscada, o los complementos más descargados del marketplace. Podemos instalar, por ejemplo, `SonarLint 6.0` que nos ayuda a mantener nuestro código limpio de errores comunes, para ello simplemente tenemos que

pulsar el botón `INSTALAR` que aparece a su lado en el listado, aceptar la licencia de uso y automáticamente nos pedirá que reiniciemos el `IDE`.

USO BÁSICO ("¡HOLA MUNDO!")

Eclipse proporciona información sobre su uso en la sección de `Ayuda`, y podemos aprender a crear nuestro primer proyecto en Java (el típico "¡Hola Mundo!"). Para ello debemos abrir la ventana de `Bienvenido`, que es la que nos aparece cuando abrimos eclipse por primera vez, o bien podemos abrirla desde `Ayuda/Bienvenido`, desde esta ventana podemos elegir la sección de `Tutoriales`, y dentro de la sección de Desarrollo Java, elegir el primer ítem "Crear una aplicación Hola Mundo", y el propio Eclipse nos irá guiando paso a paso para crear y ejecutar nuestro primer proyecto Java en Eclipse.

ACTUALIZACIÓN Y MANTENIMIENTO

En la misma sección `Ayuda` Eclipse nos proporciona las opciones para actualizar el propio Eclipse o los complementos que tengamos instalados `Ayuda/Buscar actualizaciones`.

Podemos personalizar el comportamiento respecto a las actualizaciones en la sección `Ventana/Preferencias/Instalar/Actualizar/Actualizaciones automáticas`.

Netbeans

NetBeans es una herramienta de entorno de desarrollo integrado (IDE) muy potente que se utiliza principalmente para el desarrollo en Java y C/C++. Permite desarrollar fácilmente aplicaciones web, de escritorio y móviles desde su marco modular. Puede agregar soporte para otros lenguajes de programación como PHP, HTML, JavaScript, C, C++, Ajax, JSP, Ruby on Rails, etc. mediante extensiones.

Se ha lanzado NetBeans IDE 12 con soporte para Java JDK 11. También incluye las siguientes características:

- Soporte para PHP 7.0 a 7.3, PHPStan y Twig.
- Incluir módulos en el clúster "webcommon". Es decir, todas las funciones de JavaScript en Apache NetBeans GitHub son parte de Apache NetBeans 10.
- Los módulos de clúster "groovy" están incluidos en Apache NetBeans 10.
- OpenJDK puede detectar automáticamente JTReg desde la configuración de OpenJDK y registrar el JDK expandido como una plataforma Java.
- Soporte para JUnit 5.3.1

INSTALACIÓN

Podemos instalar NetBeans de tres maneras:

Instalar desde binarios

Paso 1: Descargue el archivo NetBeans

Descargue el archivo binario de NetBeans 12 `netbeans-12.4-bin.zip`.

Paso 2: Extraer el archivo

Espere a que finalice la descarga y luego extraígalas.

```
1 $ unzip netbeans-12.4-bin.zip
```

Confirme el contenido del archivo de directorio creado:

```
1 $ ls netbeans
2 apisupport      enterprise  groovy     javafx     netbeans.css  profiler
3 bin             ergonomics harness     LICENSE    NOTICE      README.html
4 cpplite         etc        ide        licenses   php           webcommon
5 DEPENDENCIES   extide     java       nb         platform     websvccommon
```

Step 3: Move the `netbeans` folder to `/opt`

Ahora movamos la carpeta `netbeans/` a `/opt`

```
1 $ sudo mv netbeans/ /opt/
```

Paso 4: Ruta de configuración

El binario ejecutable de Netbeans se encuentra en `/opt/netbeans/bin/netbeans`. Necesitamos agregar su directorio principal a nuestro `$PATH` para poder iniciar el programa sin especificar la ruta absoluta al archivo binario. Abra su archivo `~/.bashrc` o `~/.zshrc`.

```
1 $ nano ~/.bashrc
```

Añade la siguiente línea al final

```
1 export PATH = "$PATH:/opt/netbeans/bin/"
```

Obtenga el archivo para iniciar Netbeans sin reiniciar el shell.

```
1 $ source ~/.bashrc
```

Paso 5: Crear el iniciador de escritorio NetBeans IDE (opcional)

Cree un nuevo archivo en `/usr/share/applications/netbeans.desktop`.

```
1 $ sudo nano /usr/share/applications/netbeans.desktop
```

Añade los siguientes datos.

```
1 [Desktop Entry]
2 Name=Netbeans IDE
3 Comment=Netbeans IDE
4 Type=Application
5 Encoding=UTF-8
6 Exec=/opt/netbeans/bin/netbeans
7 Icon=/opt/netbeans/nb/netbeans.png
8 Categories=GNOME;Application;Development;
9 Terminal=false
10 StartupNotify=true
```

Para desinstalar NetBeans debemos eliminar la carpeta `netbeans/` que está dentro de la carpeta `/opt/`, podemos utilizar el comando:

```
1 $ sudo rm /opt/netbeans -rf
```

Paso 6: Configurar correctamente el JDK (opcional)

En el fichero `/opt/netbeans/etc/netbeans.conf` debemos especificar correctamente la ruta de nuestro JDK en la variable `netbeans_jdkhome`. En GNU/Linux podemos saber los JDK disponibles con el comando `sudo update-alternatives --config java` que nos mostrará un resultado similar a este:

```
1 Hi ha 3 possibilitats per a l'alternativa java (que proveeix /usr/bin/java).
2
3 Selecció Camí Prioritat Estat
4 -----
5 * 0 /usr/lib/jvm/java-14-openjdk-amd64/bin/java 1411 mode automàtic
6 1 /usr/lib/jvm/java-11-openjdk-amd64/bin/java 1111 mode manual
7 2 /usr/lib/jvm/java-14-openjdk-amd64/bin/java 1411 mode manual
8 3 /usr/lib/jvm/java-8-openjdk-amd64/jre/bin/java 1081 mode manual
9
10 Premu retorn per a mantenir l'opció per defecte[*], o introduïu un número de selecció:
```

En la configuración de netbeans no es necesario especificar el final de la ruta `bin/java`

```
1 netbeans_jdkhome="/usr/lib/jvm/java-11-openjdk-amd64/"
```

Instalar desde script

Paso 1: Descargue el archivo NetBeans

También puede instalar Netbeans 12.4 en GNU/Linux desde un script proporcionado para descargar `Apache-NetBeans-12.4-bin-linux-x64.sh`.

Paso 2: Ejecutar el script

Debes ejecutar el script de instalación

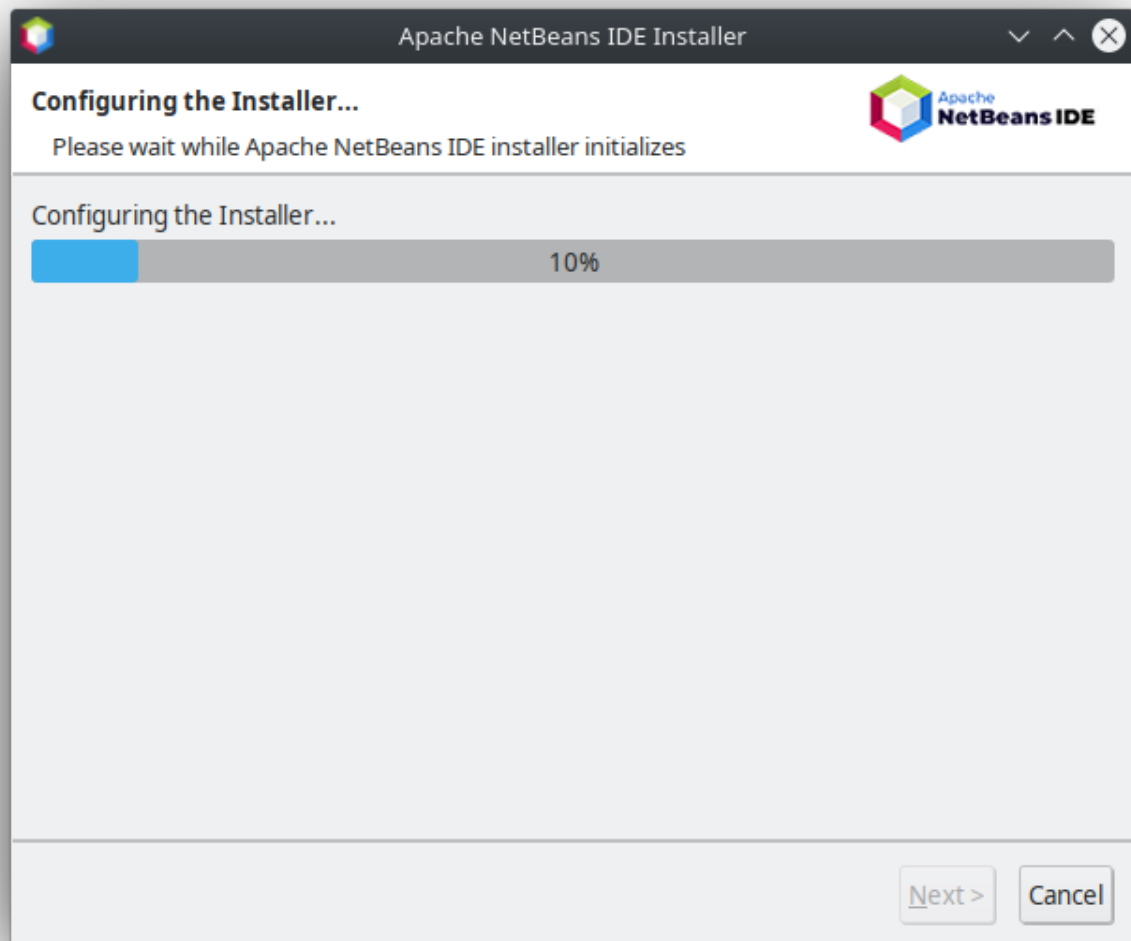
```
1 $ sudo sh ./Apache-NetBeans-12.4-bin-linux-x64.sh
```



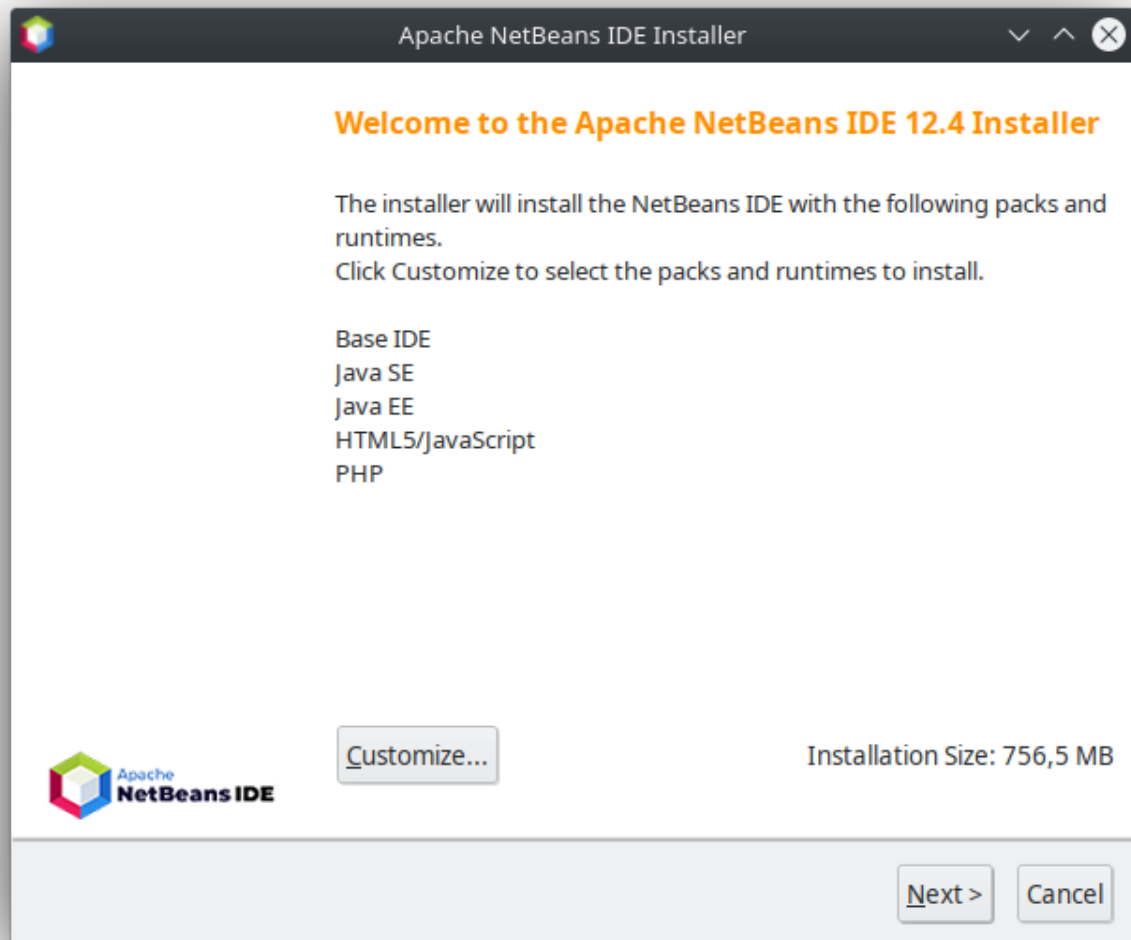
Instalación

Si ejecuta el script como `root` (`sudo`) Netbeans estará disponible para todos los usuarios. Por el contrario, si ejecuta el usuario sin `sudo` , solo estará disponible para su usuario.

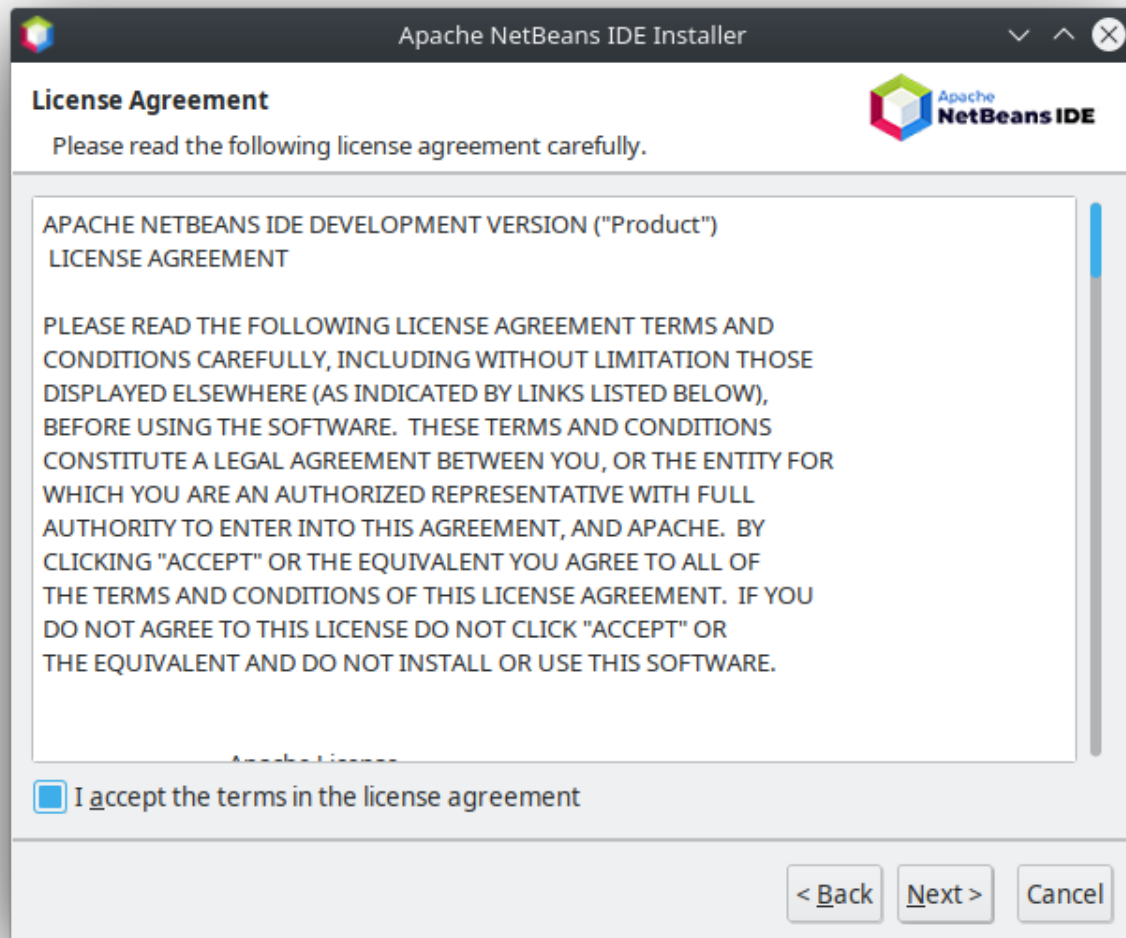
Aparecerá una barra de progreso como esta:



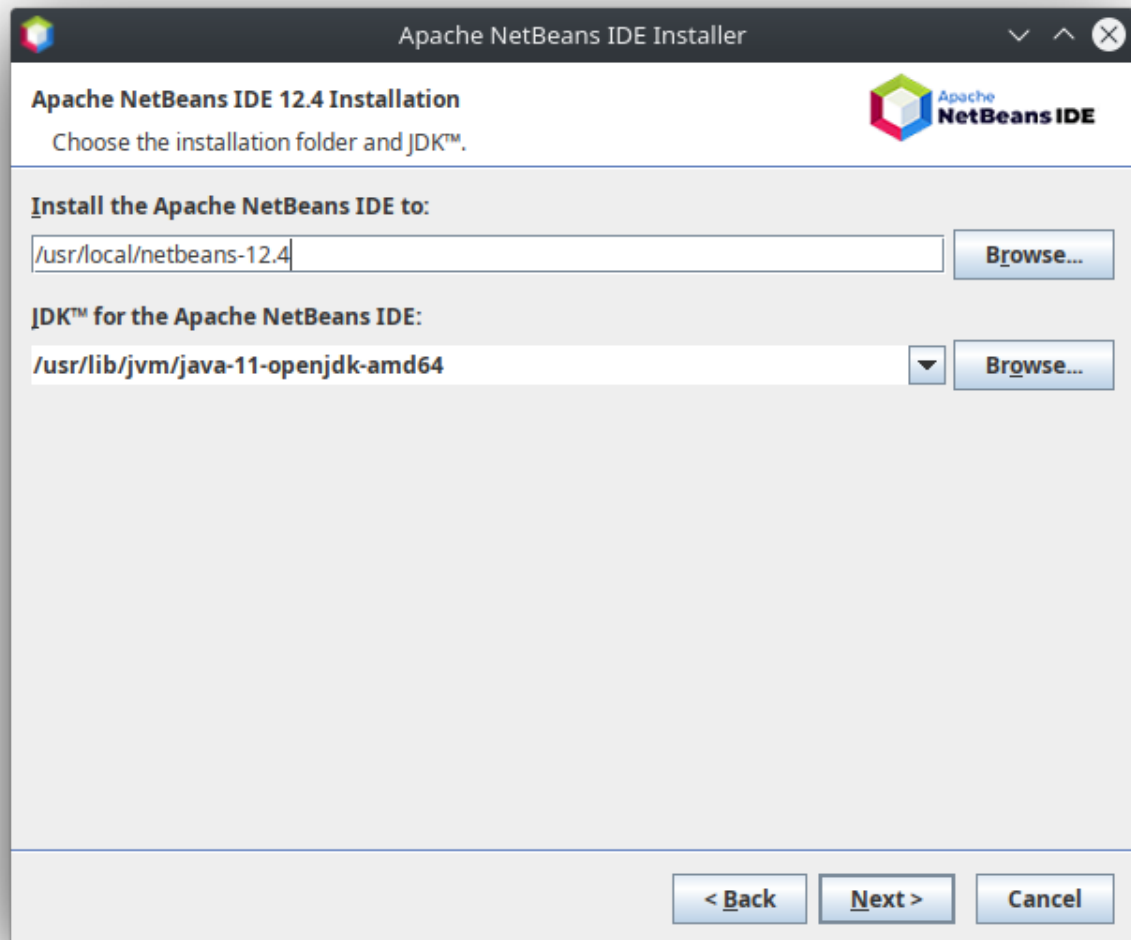
Ahora podemos elegir los componentes que queremos instalar con el IDE de Netbeans, lo dejaremos por defecto y pulsaremos el botón siguiente.

**Paso 3: Aceptar la licencia**

Luego debemos aceptar el acuerdo de licencia de uso marcando la casilla y presionando el botón siguiente.

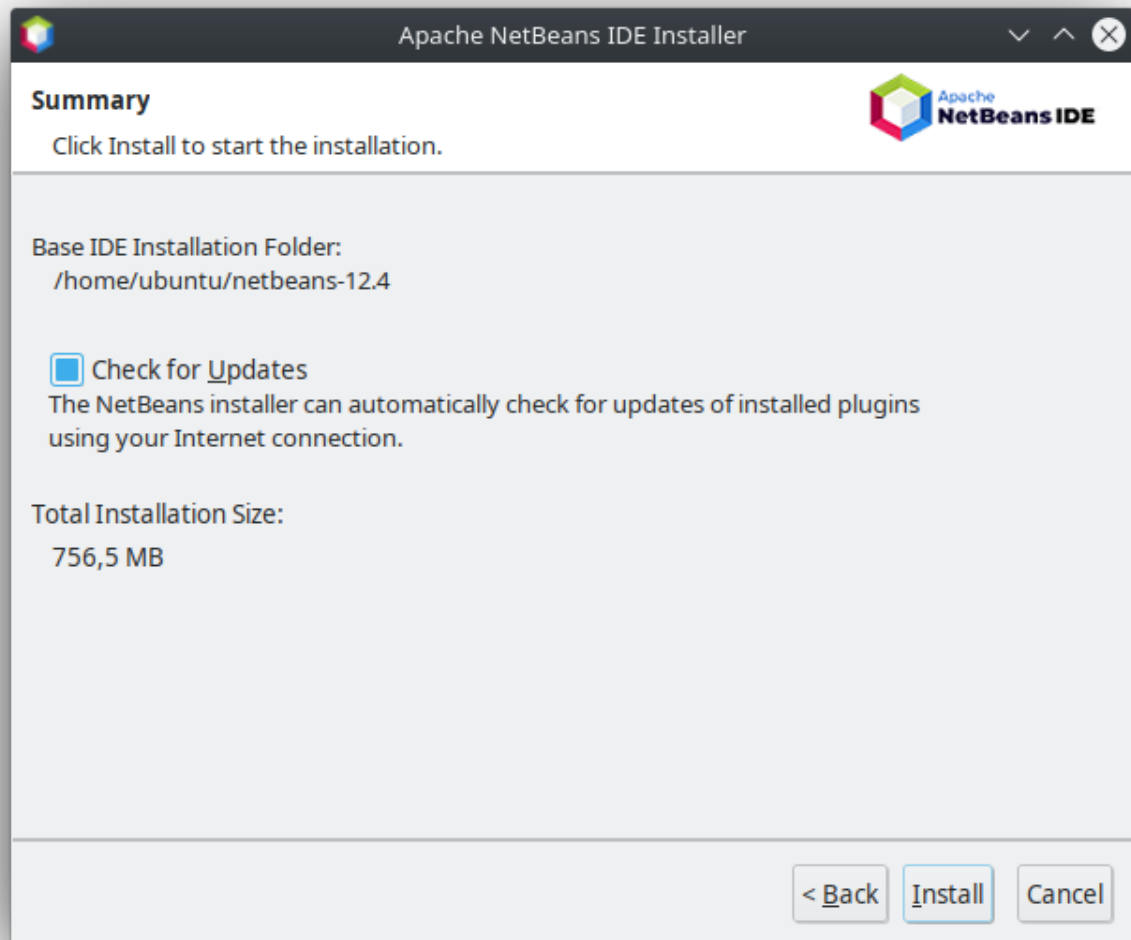
**Paso 4: Elija la ruta de instalación y el JDK**

Ahora debemos elegir la ruta donde se instalará Netbeans 12.4. Y debemos elegir la ruta donde se encuentra el JDK (por defecto indica `/usr`, pero debemos especificar la ubicación como por ejemplo `/usr/lib/jvm/java-11-openjdk-amd64`).

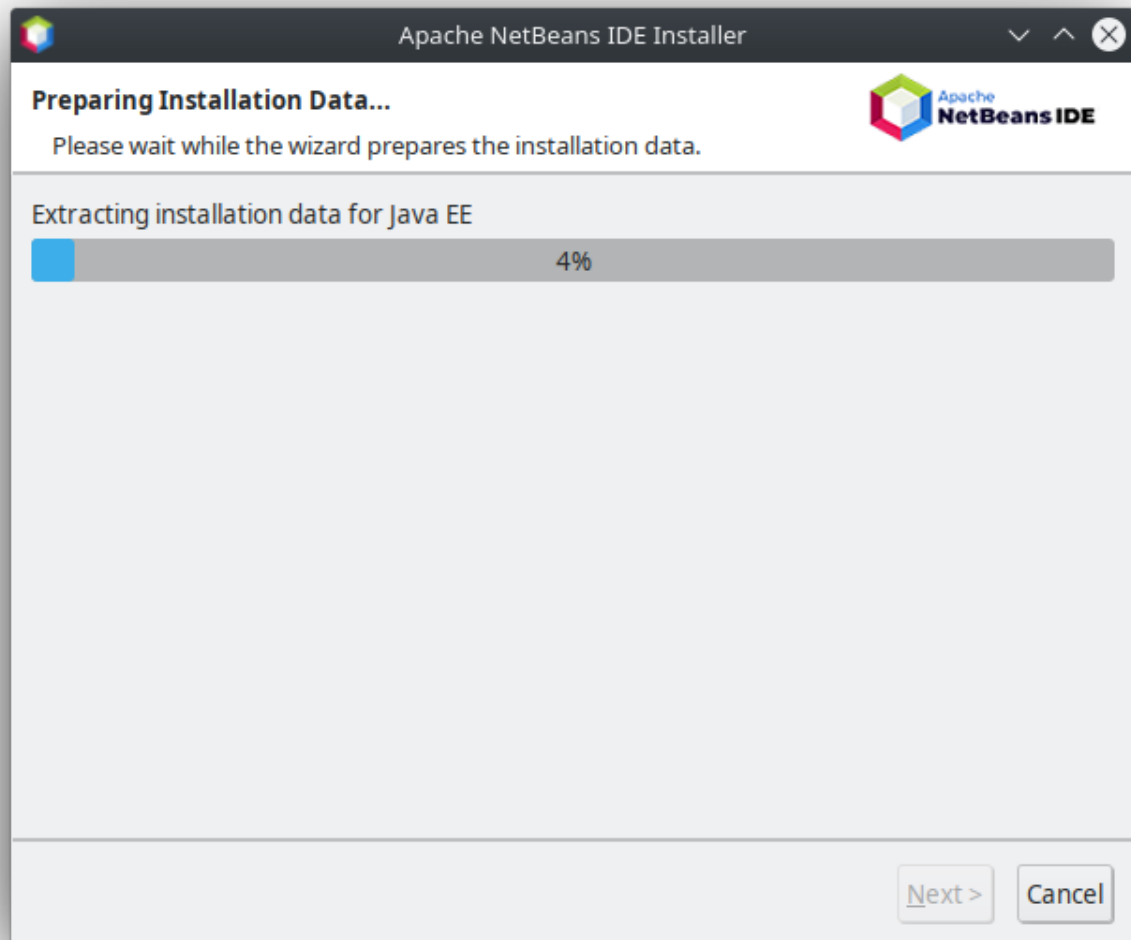


Paso 5: Actualizaciones automáticas

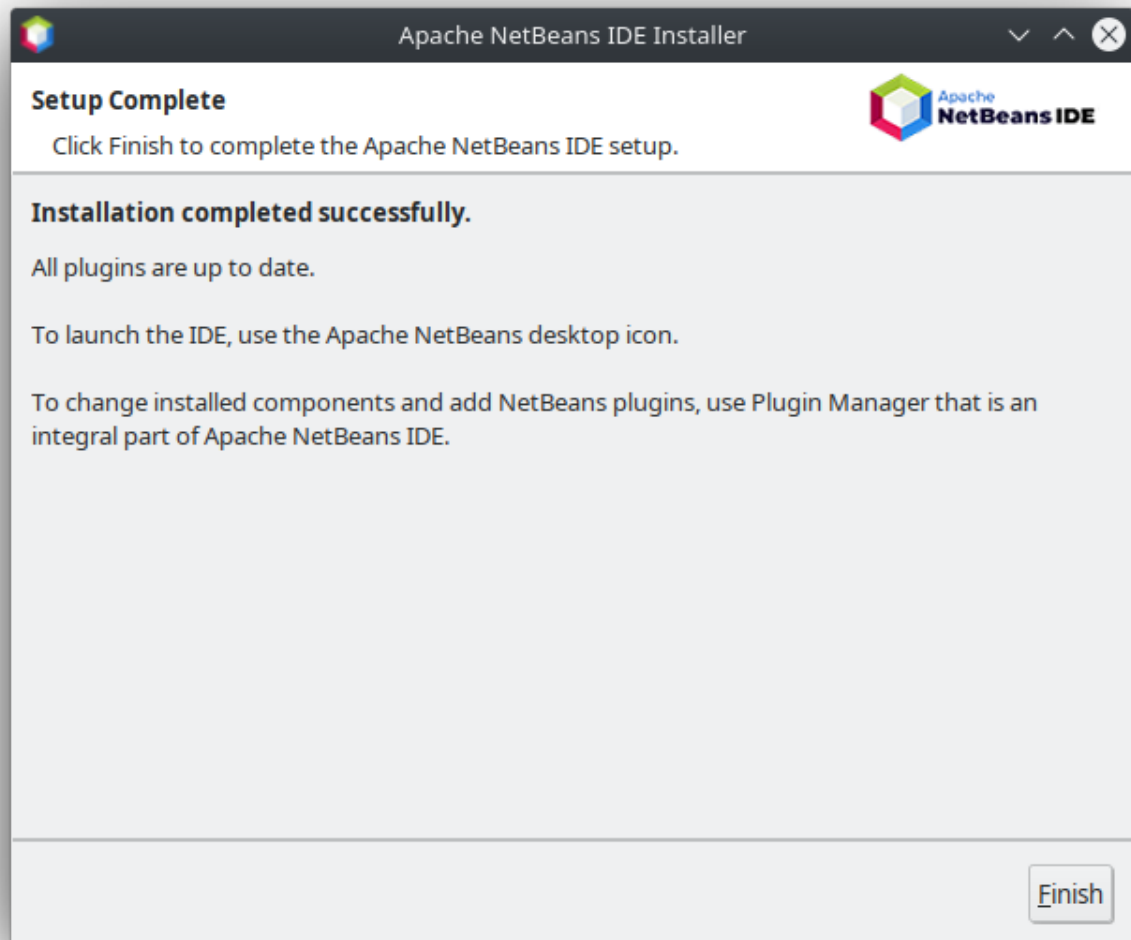
En este punto se muestra un resumen de la instalación, y podemos elegir si queremos que NetBeans busque e instale actualizaciones desde Internet, y pulsar el botón instalar.

**Paso 6: Instalación**

Aparecerá una barra de progreso.

**Paso 7: Paso final**

Al terminar, aparecerá una pantalla con las acciones realizadas por el instalador y ya tendremos los launchers creados en el menú de aplicaciones.



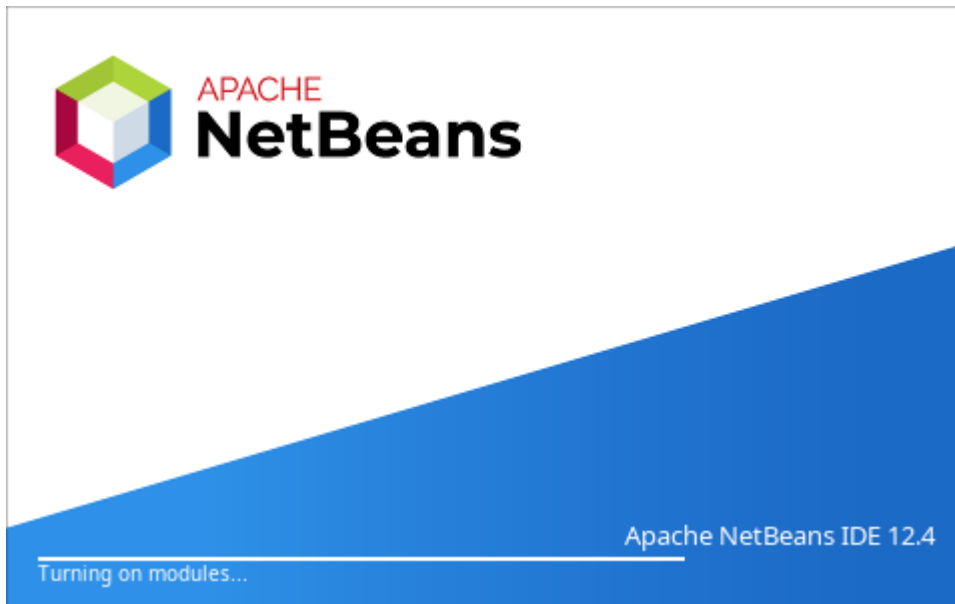
Instalar mediante snap

Quizás una forma más sencilla de instalar la última versión de Netbeans en nuestro sistema GNU/Linux es a través de snap :

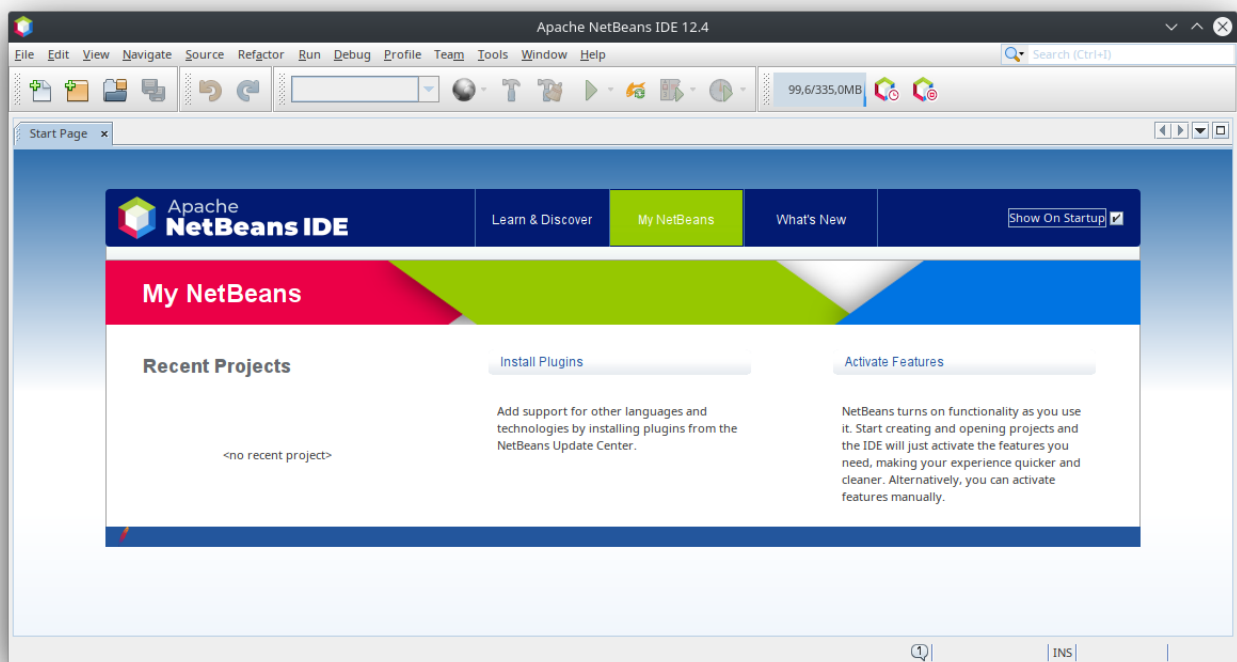
```
1 $ sudo snap install netbeans --classic
```

Primera ejecución

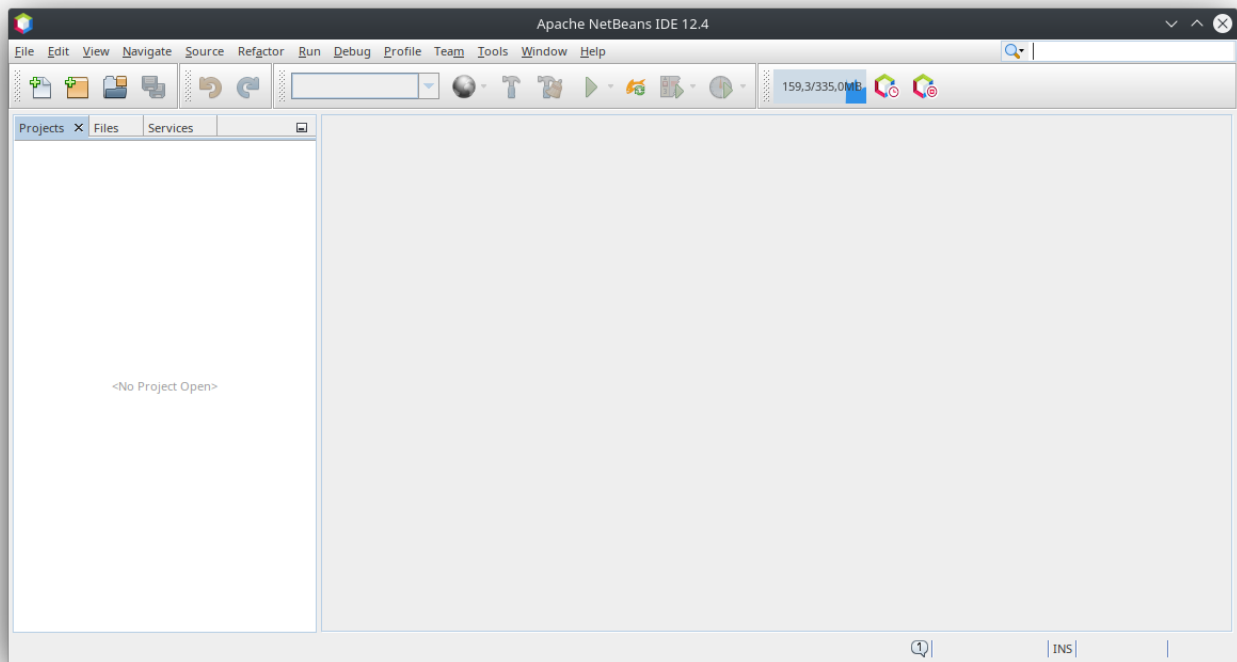
Cuando ejecutamos el entorno nos aparecerá una pantalla como la siguiente:



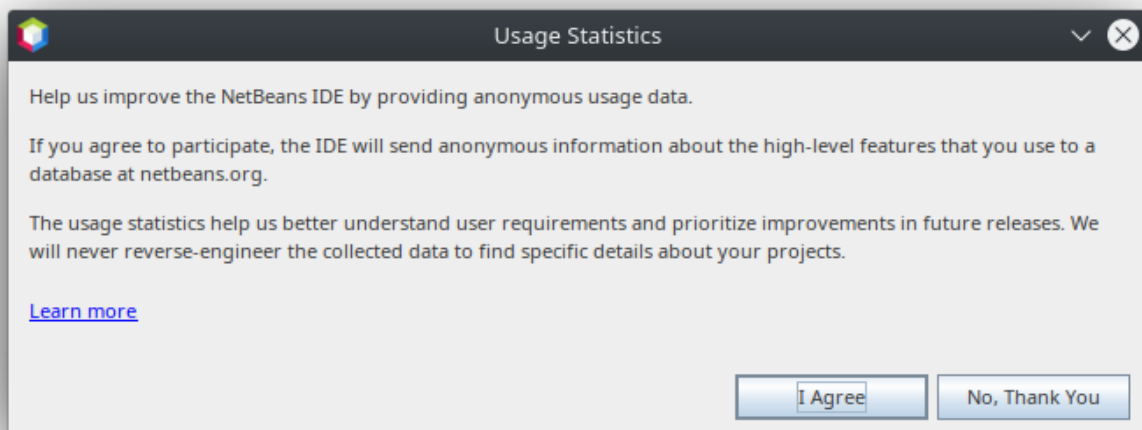
La primera vez que lo ejecutemos se mostrará la pestaña de bienvenida. Podemos pedir que no se nos muestre más desactivando la opción “Mostrar al iniciar”.



Una vez cerrada esta pestaña, el entorno de trabajo será similar a esto:



NetBeans puede solicitarnos permiso para utilizar nuestra información a nivel estadístico, elegimos el comportamiento deseado y aceptamos.



Para desinstalar NetBeans en este caso debemos ejecutar el archivo `uninstall.sh` que se encuentra en la carpeta de instalación.

CONFIGURACIÓN

Activar módulos

Por defecto Netbeans tiene los módulos desactivados y será la primera vez que los necesitemos cuando pasen a estar activos y disponibles. Por ejemplo, si creamos un nuevo proyecto y elegimos `Java Application` dentro de la categoría `Java with Ant`, veremos en la parte inferior que Netbeans nos avisa de que el módulo necesario no está activo y que debemos pulsar `Next` para

que esté disponible. Lo hacemos, y a continuación nos pedirá que activemos el módulo `nb-javac Impl`, dejamos el check marcado y pulsamos el botón `Activate`, y nos aparecerá el asistente para crear nuestro primer proyecto Java.

Versión Java

Dentro del menú `Herramientas/Plataformas Java` podemos cambiar o ver la ubicación de nuestra instalación JDK.

Perspectiva

En Netbeans las perspectivas no son necesarias, el entorno de Netbeans, aunque es personalizable, se adapta automáticamente a las tareas que estás realizando en cada momento.

Apariencia

Netbeans nos permite personalizar cualquier aspecto de la apariencia de nuestro entorno, cambiar el tema del IDE así como el tamaño de fuente y los colores para el coloreado del código fuente. Todas estas opciones están disponibles en `Herramientas/Opciones`, y dentro de esta ventana elegimos la tercera pestaña `Fuente y Colores` y la penúltima pestaña `Apariencia`.

Configuración de exportación/importación

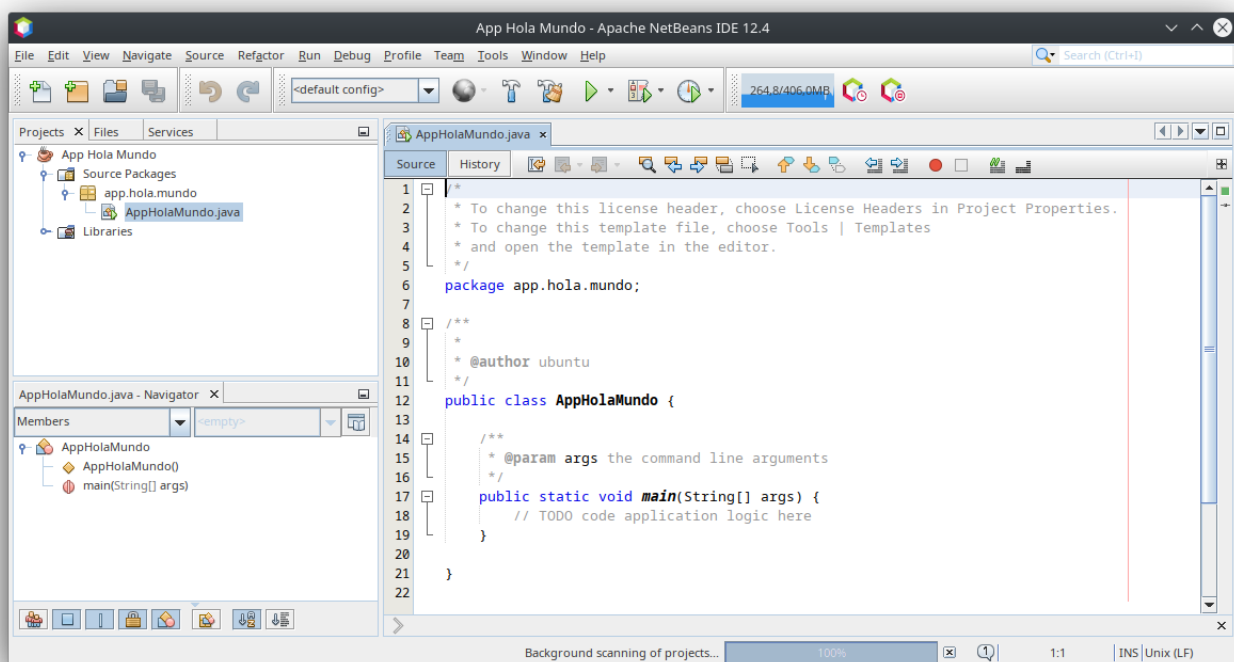
Una opción muy interesante de Netbeans es que nos permite exportar o importar configuraciones y compartirlas con otros compañeros o incluso entre nuestros equipos o diferentes instalaciones. La opción está disponible en `Herramientas/Opciones`, abajo a la izquierda encontramos los botones `Exportar...` e `Importar...`.

MÓDULOS

Las opciones y funcionalidades de Netbeans se pueden ampliar añadiendo módulos desde su sección de plugins. En `Tools/Plugins` podemos por ejemplo buscar por texto, o buscar en la pestaña de plugins disponibles. Eso nos mostrará todos los plugins que contienen la palabra buscada, o los plugins disponibles. Podemos instalar por ejemplo `sonarlint4netbeans` que nos ayuda a mantener nuestro código limpio de errores comunes, para ello simplemente tenemos que marcar la casilla delante del nombre del plugin, y pulsar el botón `INSTALAR` que aparece más abajo, pulsar siguiente, aceptar la licencia de uso e instalar. Cuando termine la instalación nos pedirá que reiniciemos el IDE.

USO BÁSICO ("¡HOLA MUNDO!")

Para crear nuestra primera aplicación en Netbeans, debemos crear una aplicación Java, desde el menú `Archivo/Nuevo Proyecto...` debemos elegir `Aplicación Java` dentro de la categoría `Java con Ant`. A continuación debemos especificar el nombre del proyecto, por ejemplo "App Hello World", y nos aseguramos de dejar marcada la opción `Crear clase principal` `app.hello.world.AppHolaWorld` y nos debería aparecer algo como esto:



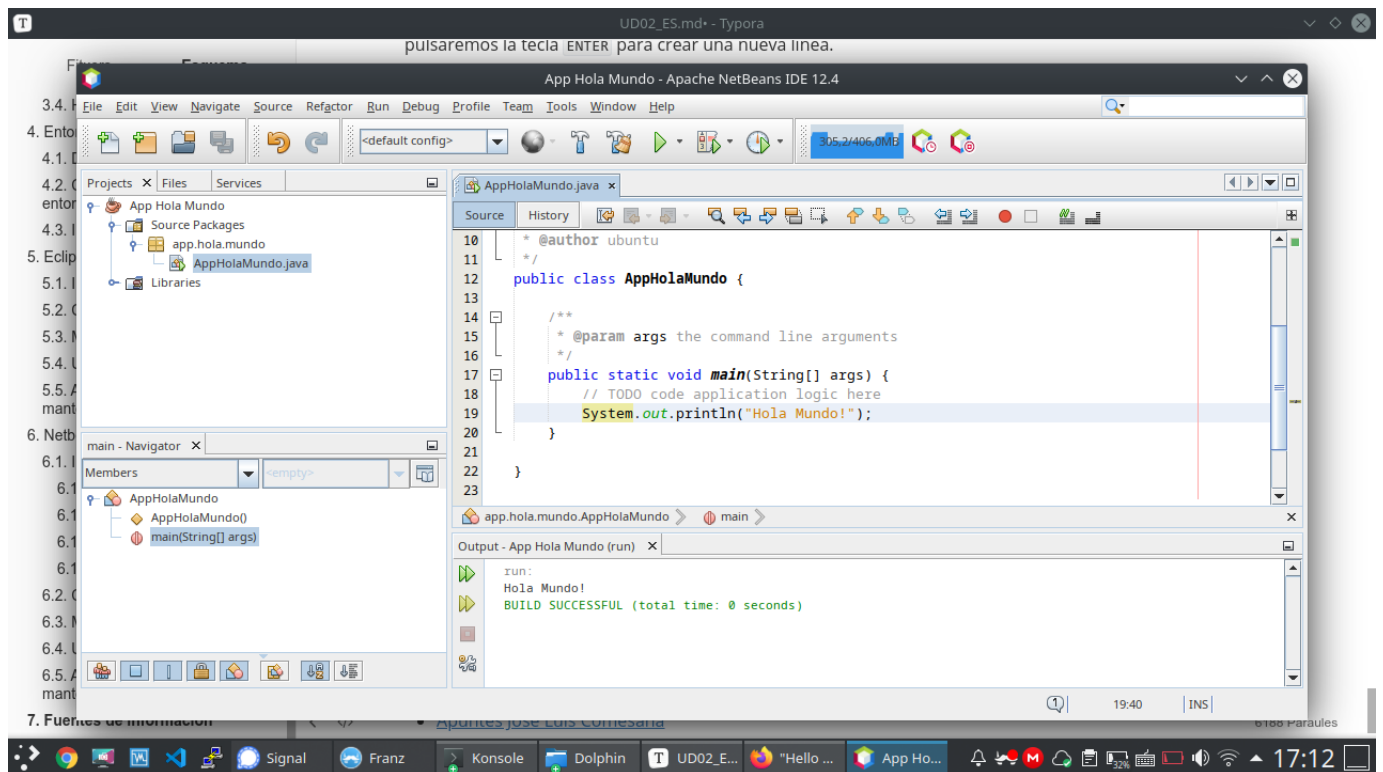
En este punto, sólo nos queda incluir la línea de código necesaria para imprimir el mensaje de texto en pantalla. Para ello, nos dirigiremos al final de la línea `// TODO code application logic here` y pulsaremos la tecla `ENTER` para crear una nueva línea.

Una vez situados en el lugar adecuado utilizaremos una de las funcionalidades más interesantes de Netbeans, que son las plantillas de código. Tecleamos la palabra "sout" y luego pulsamos la tecla `TAB` y Netbeans la sustituirá por el código correcto: `System.out.println ("");`.

Ahora debemos escribir entre las dos comillas dobles el mensaje de texto que debe aparecer en pantalla, y debe quedar así:

```
1 System.out.println("Hola Mundo!");
```

Luego podemos presionar el botón superior con un triángulo verde (Ejecutar proyecto) o presionar la tecla `F6` del teclado:



Aparecerá una nueva sección en la ventana (en la parte inferior) llamada **Salida** en la que podremos visualizar el resultado de la ejecución de nuestro primer programa.

ACTUALIZACIÓN Y MANTENIMIENTO

En la sección **Ayuda**, Netbeans nos proporciona las opciones para actualizar el propio Netbeans con la opción **Ayuda/Buscar actualizaciones**.

IntelliJ (recomendado)

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) escrito en Java para desarrollar software informático escrito en Java, Kotlin, Groovy y otros lenguajes basados en JVM. Está desarrollado por JetBrains (antes conocido como IntelliJ) y está disponible como una edición comunitaria con licencia Apache 2 y en una edición comercial propietaria. Ambas se pueden utilizar para el desarrollo comercial.

Nuestra institución dispone de licencias para nuestros alumnos mientras tengáis correo electrónico @ieseduardoprime.es.

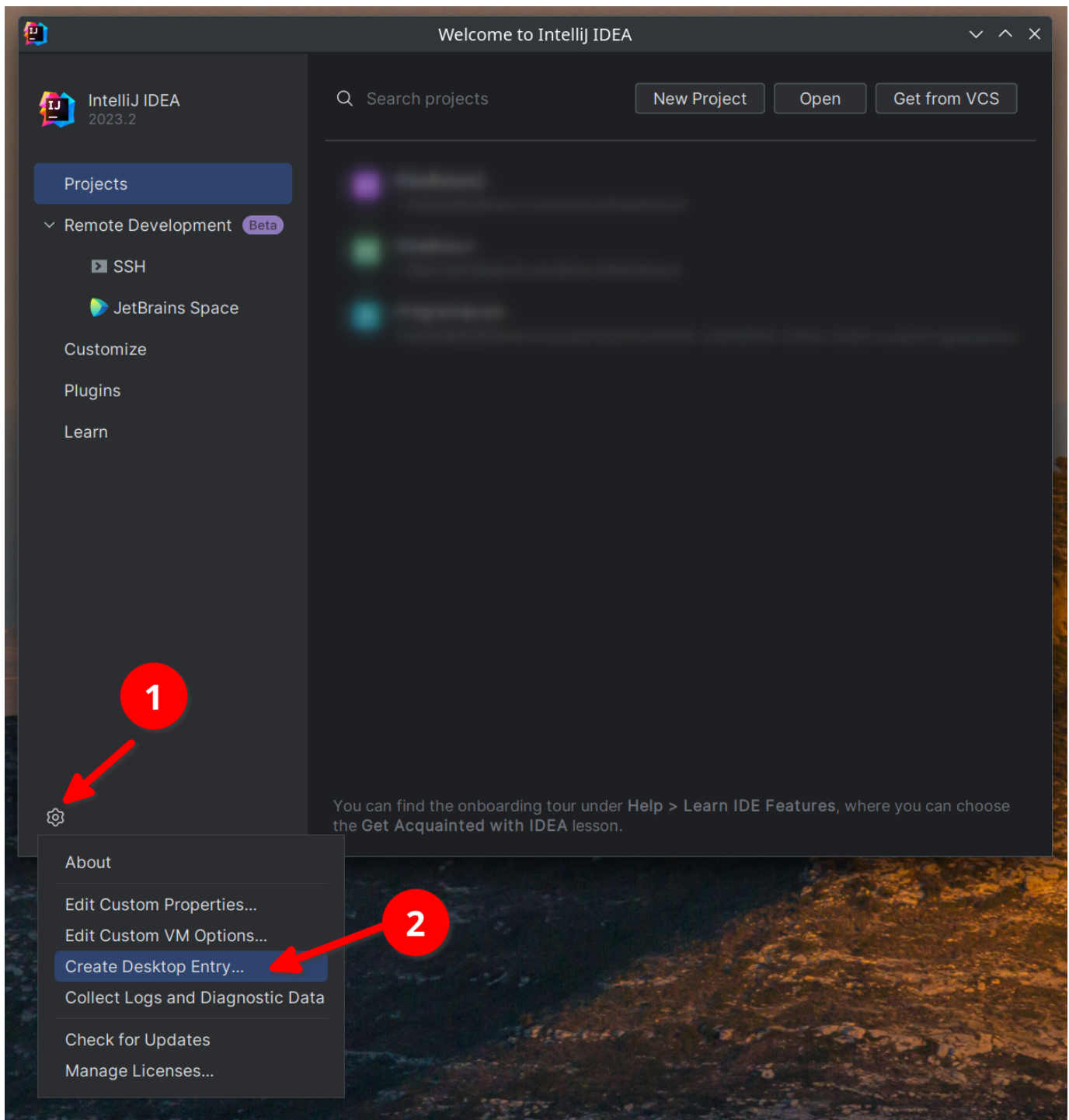
INSTALACIÓN

Descargue desde <https://www.jetbrains.com/idea/> la versión de la herramienta toolbox correspondiente a su sistema operativo.

Siga las instrucciones para su sistema operativo desde <https://www.jetbrains.com/help/idea/installation-guide.html#toolbox>

Una vez instalada la caja de herramientas, puede elegir instalar todos los productos de JetBrains.

Una vez instalada la Idea (IDE) puedes crear una entrada de escritorio desde la pantalla inicial:



Y en la opción Administrar licencias debes seguir estas instrucciones: https://www.jetbrains.com/help/license_server/Activating_license.html

La dirección del servidor es: <https://iesepm.fls.jetbrains.com/>

AJUSTES

Documentos para configurar su IDE: <https://www.jetbrains.com/help/idea/configuring-project-and-ide-settings.html>

MÓDULOS

Puedes agregar complementos siguiendo estas instrucciones:

<https://www.jetbrains.com/help/idea/managing-plugins.html>

USO BÁSICO ("¡HOLA MUNDO!")

Los documentos te ayudan con tu primer programa en Java: <https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html>

Mucha más información:

- Si vienes de Eclipse: <https://www.jetbrains.com/help/idea/migrating-from-eclipse-to-intellij-idea.html>
- Si estuvieras en NetBeans: <https://www.jetbrains.com/help/idea/netbeans.html>
- Si quieres aprender por tu cuenta: <https://www.jetbrains.com/help/idea/product-educational-tools.html>

Por qué debería elegir IntelliJ en lugar de VSCode para la codificación en Java

IDEA INTELLIJ:

Ventajas:

1. **Entorno integrado completo:** IntelliJ IDEA está diseñado específicamente para el desarrollo de Java y ofrece un conjunto completo de herramientas y características optimizadas para esta tarea.
2. **Análisis estático avanzado:** Proporciona un análisis de código en profundidad que detecta errores y problemas potenciales antes de la compilación.
3. **Depuración avanzada:** ofrece un potente conjunto de herramientas de depuración que ayudan a identificar y resolver problemas en el código.
4. **Refactorización guiada:** Proporciona herramientas para reorganizar y optimizar el código de forma segura, promoviendo buenas prácticas de programación.
5. **Compatibilidad con marcos y tecnologías Java:** Integración nativa con muchos marcos y tecnologías utilizados en el desarrollo Java, lo que facilita la creación de aplicaciones completas.
6. **Generación automática de código:** ayuda a los programadores a generar automáticamente fragmentos de código repetitivos, como captadores y definidores.
7. **Integración con herramientas de compilación:** facilita la integración con herramientas de compilación como Maven y Gradle.
8. **Soporte para pruebas unitarias:** Ofrece integración con marcos de prueba como JUnit para el desarrollo basado en pruebas.
9. **Facilidad de configuración:** Proporciona asistentes guiados para configurar de manera eficiente proyectos Java.

Contras:

1. **Mayor consumo de recursos:** Debido a su naturaleza integral y rica en funciones, IntelliJ IDEA puede consumir más recursos del sistema en comparación con IDE más livianos.
2. **Curva de aprendizaje:** Dado que ofrece una amplia gama de funciones, los principiantes pueden tardar un tiempo en familiarizarse con todas las herramientas disponibles.

VISUAL STUDIO CODE (VSCODE):

Ventajas:

1. **Ligero y rápido:** VSCode es un editor de código liviano y rápido, lo que lo hace ideal para proyectos más pequeños o para aquellos que prefieren una experiencia más ágil.
2. **Amplia gama de extensiones:** Tiene una amplia comunidad que desarrolla extensiones para diversas tecnologías y lenguajes, incluido Java.
3. **Versatilidad:** Si bien no está diseñado específicamente para Java, se puede personalizar para que funcione con Java a través de extensiones.
4. **Integración de control de versiones:** ofrece integración nativa con sistemas de control de versiones como Git.
5. **Curva de aprendizaje rápida:** Debido a su enfoque más ligero, puede resultar más sencillo para los principiantes comenzar a trabajar con él.

Contras:

1. **Funcionalidad limitada de Java:** Aunque existen extensiones de Java, VSCode no ofrece el mismo conjunto completo de herramientas optimizadas para Java que IntelliJ IDEA.

2. **Análisis menos profundo:** Las capacidades de análisis estático y corrección de código podrían no ser tan avanzadas como las de IntelliJ IDEA.
3. **Depuración limitada:** si bien ofrece depuración, es posible que no sea tan avanzada o completa como la de IntelliJ IDEA.
4. **Configuración manual del proyecto:** La configuración de proyectos Java puede requerir más pasos y configuración manual en comparación con IntelliJ IDEA.

Tarea

Debes entregar un documento `*.pdf` explicando que IDE has elegido para empezar a programar (más adelante lo puedes cambiar si quieres), justificando porqué lo has elegido.

Además envía una captura de pantalla en la que se vea el resultado del comando:

```
1 java --version
```

Y por último capturas de pantalla donde se pueda ver que editas el fichero fuente (HolaMundo.java), lo compilas y lo ejecutas dentro del IDE que has elegido (explica los pasos que has seguido)

🕒 19 de julio de 2026

2.3.3 Taller UD01_03: Crear cuenta en GitHub

Qué es GitHub

GitHub es una plataforma en la nube basada en Git que permite a los desarrolladores almacenar, gestionar y colaborar en proyectos de código. Es el portafolio universal de los programadores.

Crear una cuenta es esencial para quien aprende o busca trabajar en programación porque: sirve como tu currículum técnico, donde muestras tus proyectos y evolución; te permite colaborar en proyectos open source para ganar experiencia real; y es una herramienta fundamental para el control de versiones y trabajo en equipo, usada por prácticamente todas las empresas tech.

Creas tu cuenta

Accede a la plataforma GitHub: <https://github.com/>

Pulsa sobre el botón [Sign Up] y sigue las instrucciones para crear tu cuenta.

Una vez creada tu cuenta, entra en tu página principal, por ejemplo la mía es esta: <https://github.com/martinezpenya> (martinezpenya es mi usuario de github) y realiza una captura de pantalla.

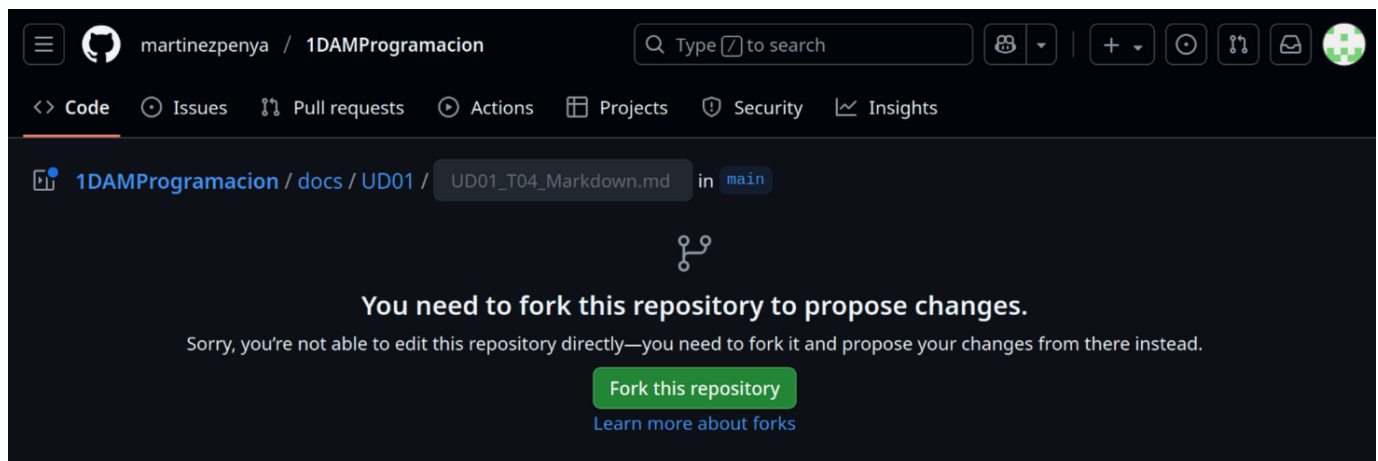
Solicitar corrección de los apuntes

Ahora, para probar nuestra nueva cuenta y colaborar con algún proyecto, no hay nada mejor que ayudar a mejorar los apuntes del profesor de Programación 😊.

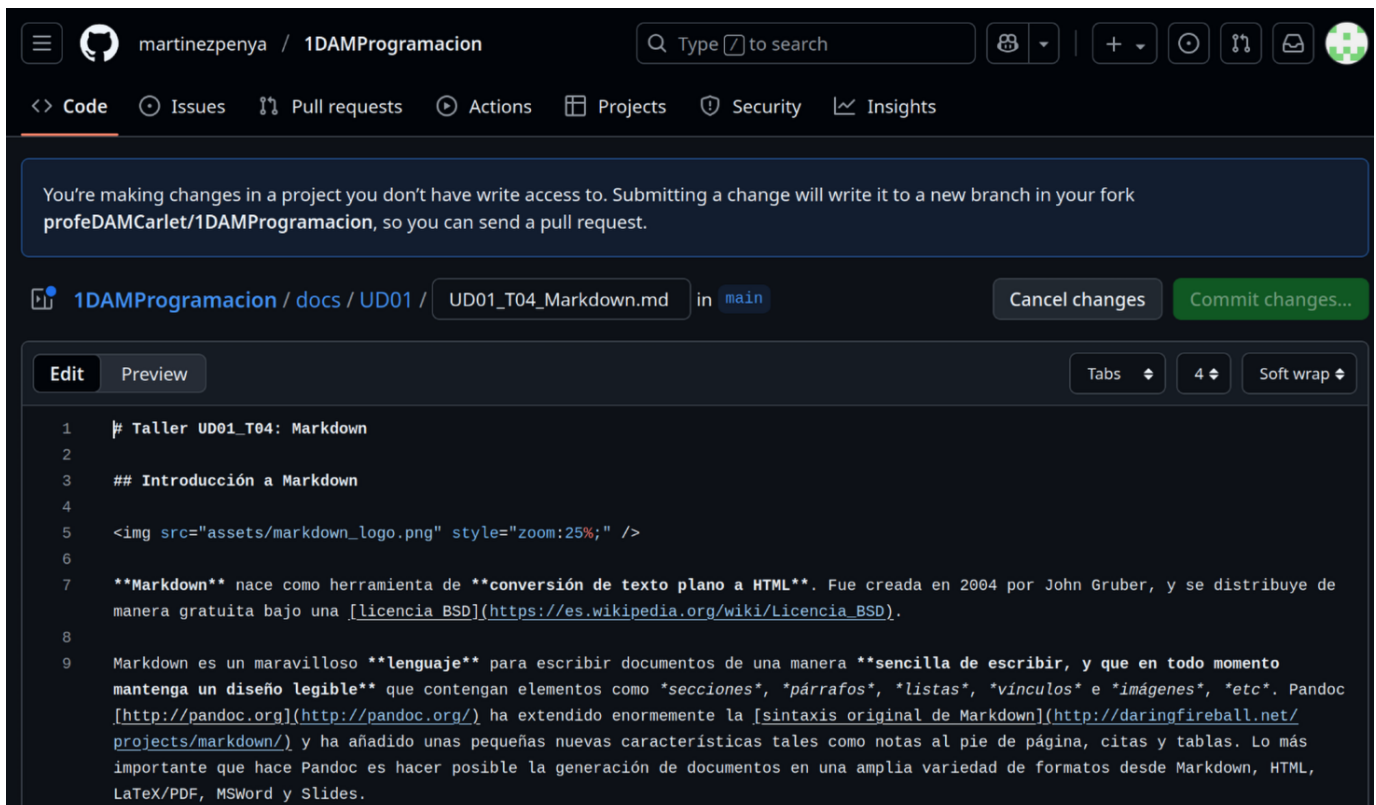
Accedemos a la página de los apuntes en la que hemos detectado el error o queremos sugerir un cambio y en la parte superior derecha debe aparecer el icono:



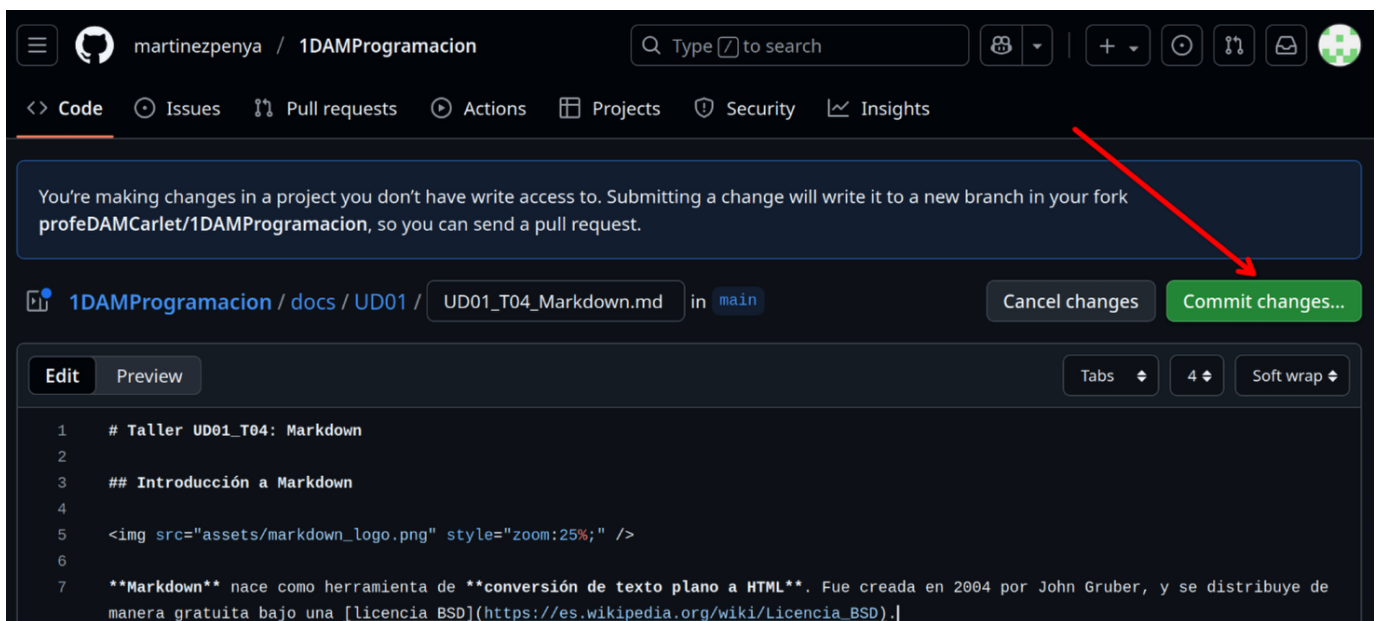
Esto nos llevará a crear un Fork del repositorio (este concepto lo aprenderás más adelante en el módulo de Entornos de Desarrollo):



Ahora debemos pulsar el botón **[Fork this repository]**, y a continuación veremos el código de la página en nuestro fork que es `MarkDown` (Puedes aprender más sobre `MarkDown` en el Taller 4):



Ahora debemos buscar el texto a modificar y una vez hayamos cambiado algo del documento se activará el botón [Commit changes...]:



Ahora debes explicar cual ha sido la modificación que hemos realizado y pulsar el botón [Propose changes]:

Propose changes

Commit message

Update UD01_T04_Markdown.md

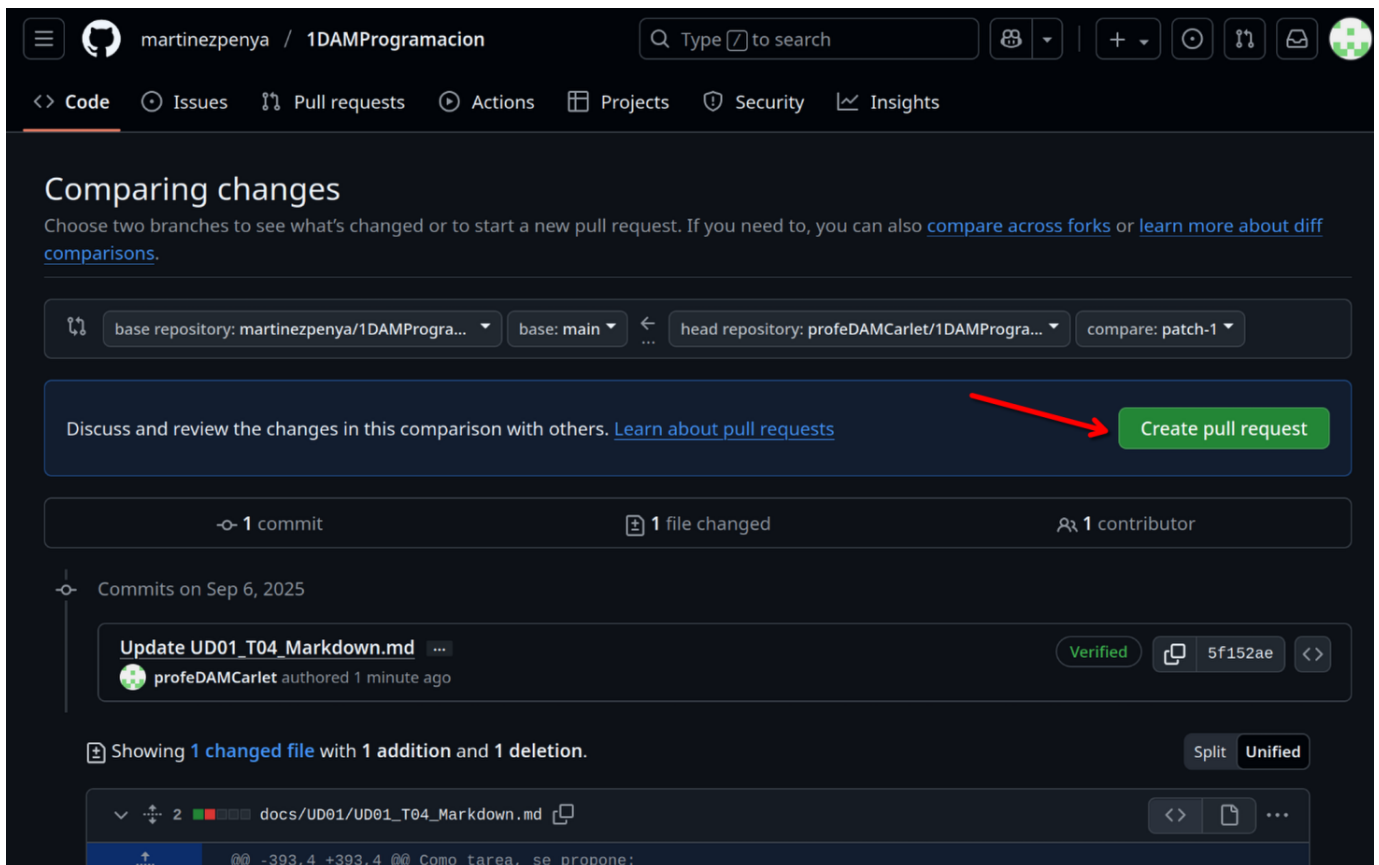
Extended description

He cambiado el formato de las extensiones de los documentos
a entregar.

Cancel

Propose changes

Todavía no hemos terminado! ahora hay que comunicar los cambios propuestos en nuestro Fork al propietario del repositorio, para que los visualice y valore si los quiere incluir en la página de documentación. Para ello debemos pulsar el botón [Create pull request]:



martinezpenya / 1DAMProgramacion

Q Type to search

<> Code Issues Pull requests Actions Projects Security Insights

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) **Create pull request**

1 commit 1 file changed 1 contributor

Commits on Sep 6, 2025

Update UD01_T04_Markdown.md ... Verified 5f152ae <>
profeDAMCarlet authored 1 minute ago

Showing 1 changed file with 1 addition and 1 deletion. Split Unified

docs/UD01/UD01_T04_Markdown.md <> ...

-393,4 +393,4 @ @ Como tarea, se propone:

Ahora podemos modificar el mensaje (pero no hace falta), directamente pulsamos sobre el botón [Create pull request]:

martinezpenya / 1DAMProgramacion

<> Code Issues Pull requests Actions Projects Security Insights

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Add a title

Update UD01_T04_Markdown.md

Add a description

Write Preview H B I ...

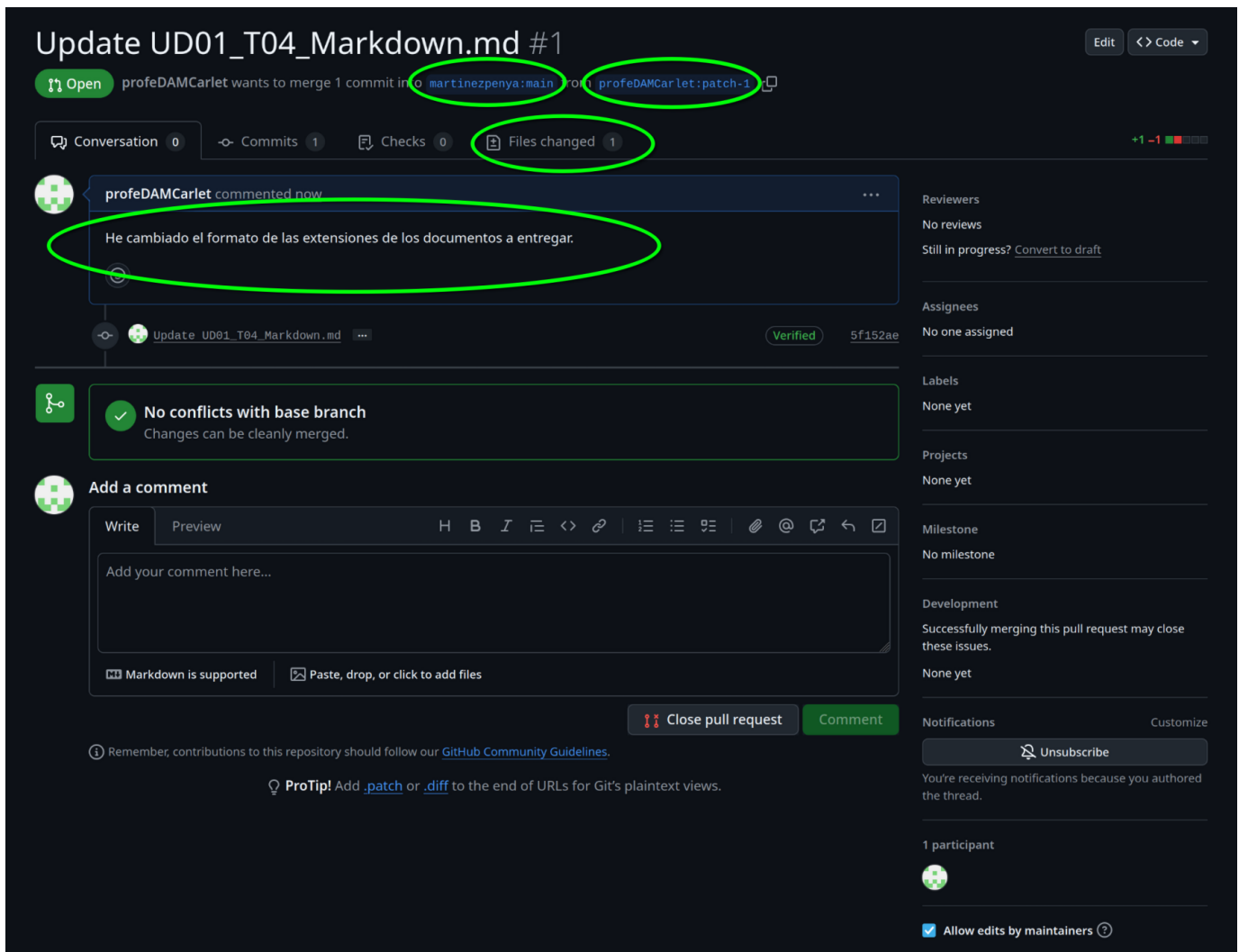
He cambiado el formato de las extensiones de los documentos a entregar.

Markdown is supported Paste, drop, or click to add files

☒ Allow edits by maintainers **Create pull request**

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Ahora si, deberías ver una página similar a la siguiente, de la que también deberás obtener una captura y adjuntarla al .pdf, y además explicar los 4 campos que hay redondeados:



Como resumen:

1. Hemos creado un fork de un repositorio.
2. Hemos modificado un archivo en nuestro fork.
3. Hemos comparado nuestro fork con el original y hemos creado un pull request con las diferencias.

Ahora pueden pasar dos cosas, que el propietario del repositorio original acepte nuestros cambios, y por tanto pasaremos a ser colaboradores del repositorio original.

O bien, que el cambio no sea aceptado.

En cualquiera de los dos casos, si adjuntas las capturas y explicas los campos la actividad estará correcta.

En este caso concreto se ha aceptado la modificación:

Update UD01_T04_Markdown.md #1

Merged martinezpenya merged 1 commit into martinezpenya:main from profeDAMCarlet:patch-1 1 minute ago

Conversation 1 Commits 1 Checks 0 Files changed 1

profeDAMCarlet commented 10 minutes ago Contributor ...

He cambiado el formato de las extensiones de los documentos a entregar.

Update UD01_T04_Markdown.md Verified 5f152ae

martinezpenya commented 1 minute ago Owner ...

Perfecto, gracias!

martinezpenya closed this 1 minute ago

martinezpenya merged commit 8052475 into martinezpenya:main 1 minute ago Revert

Tarea

- Crea un documento `.pdf` donde debes adjuntar la captura de tu perfil de github.
- Añade una **captura** de pantalla donde se vea que has solicitado el **pull request** y que estás esperando a que se integre en el repositorio original.
- Además, **explica** que significan cada uno de los **4 apartados** señalados en la captura.

Adjunta el documento `.pdf` con las capturas y las explicaciones a la tarea de AULES.

🕒 29 de septiembre de 2025

2.3.4 Taller UD01_T03: Markdown

Introducción a Markdown



Definición

Markdown es un lenguaje de marcado ligero que permite escribir texto con formato (negritas, cursivas, enlaces, imágenes, código, tablas, etc.) usando texto plano, que luego puede convertirse a HTML, PDF y otros formatos.

Markdown es un maravilloso **lenguaje** para escribir documentos de una manera **sencilla de escribir, y que en todo momento mantenga un diseño legible** que contengan elementos como *secciones, párrafos, listas, vínculos e imágenes, etc.* Pandoc <http://pandoc.org> ha extendido enormemente la [sintaxis original de Markdown](#) y ha añadido unas pequeñas nuevas características tales como notas al pie de página, citas y tablas. Lo más importante que hace Pandoc es hacer posible la generación de documentos en una amplia variedad de formatos desde Markdown, HTML, LaTeX/PDF, MSWord y Slides.

Este método te permitirá añadir formatos tales como **negritas**, *cursivas* o [enlaces](#), utilizando texto plano, lo que permitirá hacer de tu escritura algo más simple y eficiente al evitar distracciones.

Con Markdown **no vas a reemplazar todo**, sino cubrir las funcionalidades más comunes que se requieren para escribir un documento relativamente complicado.

Para qué sirve Markdown

Markdown será perfecto para ti sobre todo **si publicas de manera constante en Internet**, donde el lenguaje HTML está más que presente: WordPress, Squarespace, Jekyll...

Pero no estoy hablando solo de [blogs](#) o páginas web. **Servicios** como Trello o **foros** como Stackoverflow también soportan este lenguaje, y con el paso del tiempo encontrarás aún más lugares que lo utilicen.

Además, Markdown está cada vez más extendido en el **mundo “offline”**. Nada te impedirá utilizar este lenguaje para **tomar notas y apuntes** de tus clases o reuniones en una determinada [aplicación](#) (incluso podrías **escribir un libro con él**, ya que puedes exportar fácilmente el resultado final a un formato ePub).

Gracias a la simplicidad de su sintaxis podrás utilizarlo siempre que necesites escribir y dar formato rápidamente, sobre todo si quieres hacerlo desde dispositivos móviles.

Por qué utilizar Markdown

VENTAJAS

- **Markdown para todo.** Para crear apuntes, documentos, notas, sitios web, libros, documentación técnica, etc. de forma off-line.
- **Markdown transportable.** Este tipo de formato siempre será **compatible con todas las plataformas** que utilices, así que utilizar Markdown es una manera de mantener todo tu contenido siempre accesible desde cualquier dispositivo (smartphones, ordenadores de escritorio, tablets...), ya que en cualquiera de ellas siempre encontrarás [las aplicaciones adecuadas](#) para leer y editar este tipo de contenido.
- Ideal para escribir un libro, pues permite la exportación fácil en ePub, PDF...

Si en el futuro Microsoft Word desapareciese perderías acceso a todo el contenido que has creado durante años utilizando dicho procesador. Así que lo más inteligente para evitar eso es **generar tu contenido de la manera más sencilla posible**: utilizando texto plano.

DESVENTAJAS

- No tiene muchas funcionalidades (esto es lo que lo hace muy compatible).
- Al no tener todas las opciones de un procesador de textos a veces tendrás que combinar Markdown con HTML para lograr ciertos formatos.

Editores para Markdown

OFF-LINE

- **Typora**
- MarkdownPad
- HarooPad
- Markdown Monster
- ...

ONLINE

- Dillinger
- GitHub
- ...

Párrafos y saltos de línea

Si queremos generar un nuevo párrafo en Markdown simplemente separa el texto mediante una línea en blanco (**pulsando dos veces intro**).



Importante

Markdown ignora las líneas en blanco múltiples: dos o más líneas en blanco consecutivas se reducen a una sola al procesarse.

Para realizar un salto de línea y empezar **una frase en una línea siguiente dentro del mismo párrafo**, tendrás que pulsar **dos veces la barra espaciadora antes de pulsar una vez intro**.

Por ejemplo si quisieses escribir un poema quedaría tal que así:

«La tierra estaba seca, No había ríos ni fuentes. Y brotó de tus ojos.

Donde cada verso tiene **dos espacios en blanco al final**.

Encabezados

Las **# almohadillas** son uno de los métodos utilizados en Markdown para crear encabezados. Debes usarlos añadiendo **uno por cada nivel**.

Es decir,

```
1 # Encabezado 1
2 ## Encabezado 2
3 ### Encabezado 3
4 #### Encabezado 4
5 ##### Encabezado 5
6 ##### Encabezado 6
```

Se corresponde con:

1. Encapçalament 1

1.1. Encapçalament 2

1.1.1. Encapçalament 3

1.1.1.1. Encapçalament 4

1.1.1.1.0.1. Encapçalament 5

1.1.1.1.0.1. Encapçalament 6

También puedes cerrar los encabezados con el mismo número de almohadillas, por ejemplo escribiendo `### Encabezado 3 ###`. Pero la única finalidad de esto es un **motivo estético**.

Texto básico

Un párrafo no requiere sintaxis especial.

Para aplicar **negrita** al texto, se escribe entre dos asteriscos.

Para aplicar *cursiva* al texto, se escribe entre un solo asterisco.

Para tachar el texto, se escribirá dos virgulillas antes y dos después de éste.

```
1 Este texto es en **negrita**.
2 Este texto es en *itálica*.
3 Este texto está ~tachado~.
4 Este texto es en ambos ***negrita e itàlica***.
```

Se corresponde a:

Este texto es en ****negrita****.

Este texto es en *itálica*.

Este texto está ~~tachado~~.

Este texto es en ambos *****negrita e itàlica*****.

⚠ Atención

Markdown no soporta subrayado de forma nativa. Para subrayar texto debes usar HTML: `<u>texto subrayado</u>`.

Para **ignorar los caracteres** de formato de Markdown, ponga `\` antes del carácter:

Citas

Las citas se generan utilizando el carácter *mayor que* `>` al comienzo del bloque de texto.

```
1 > No hay que ir para atrás ni para darse impulso. — Lao Tsé.
```

No hay que ir para atrás ni para darse impulso. — Lao Tsé.

Si la cita en cuestión se compone de **varios párrafos**, deberás añadir el mismo símbolo `>` al comienzo de cada uno de ellos.

Listas

LISTAS ORDENADAS

Para crear **listas numeradas**, empieza una línea con `1.` or `1)`.



Truco

En listas ordenadas, puedes poner `1.` en todos los elementos y Markdown los numerará automáticamente de forma correcta. Esto facilita reordenar elementos sin tener que reenumerar.

```
1 1. Ítem 1 de la lista.
2 1. Siguiendo ítem de la lista.
3 1. Siguiendo ítem, el tercero, de la lista.
```

Se corresponde con:

1. Ítem 1 de la lista.
2. Siguiendo ítem de la lista.
3. Siguiendo ítem, el tercero, de la lista.

LISTAS NO ORDENADAS

Para crear listas no numeradas, o de viñetas, empieza una línea con `*`, `-` o `+`, pero no mezcles los formatos dentro de la misma lista. (No mezclar formatos de viñetas, como `*` y `+` por ejemplo, dentro del mismo documento).

```
1 * Ítem 1 de la lista.
2 * Siguiendo ítem de la lista.
3 * Siguiendo ítem, el tercero, de la lista.
```

Se corresponde con:

- Ítem 1 de la lista.
- Siguiendo ítem de la lista.
- Siguiendo ítem, el tercero, de la lista.

También podremos combinar ambos tipos de listas. Como por ejemplo:

1. element de llista 2 - element de llista 2.2
 - element de llista 2.2.1
 - element de llista 2.2.2

LISTAS DE TAREAS

Para crear listas de tareas basta con que empiece la línea con `- [ ]`, si queremos que no esté el check marcado, y `- [x]`, si queremos que esté el check marcado.

```
1 - [x] regar plantas.
2 - [ ] realizar ejercicios de programación.
```

Se corresponde con:

- ☒ regar plantas.
- ☐ realizar ejercicios de programación.

Tablas

Las tablas no forman parte de la especificación principal de Markdown, pero Adobe, en cierta forma, las admite.

Para generar una tabla utiliza la barra vertical `|` para generar filas y columnas.

Si insertamos guiones `---` dentro de una celda crearemos el encabezado de la tabla.

```
1 | encabezado1 | encabezado2 | encabezado3 |
2 | --- | --- | --- |
3 | celda 1.1 | celda 1.2 | celda 1.3 |
4 | celda 2.1 | celda 2.2 | celda 2.3 |
```

Quedaría:

encabezado1	encabezado2	encabezado3
celda 1.1	celda 1.2	celda 1.3
celda 2.1	celda 2.2	celda 2.3

Si queremos una **celda con más de una línea** de texto podremos insertar `\` (o **Shift+Intro**) al final de ésta.

Enlaces

Para generar un enlace en Markdown se debe poner un código con dos partes:

- `[texto del enlace]`, que es el texto que se va a mostrar,
- Y después `(nombrefichero.md)`, que es la URL o el nombre de archivo al que se va a vincular.

```
1 [link text](file-name.md)
```

Un ejemplo:

[enlace a web del centro](https://iesmre.com)

La visualización del ejemplo anterior:

[enlace a web del centro](https://iesmre.com)

Imágenes

Para insertar una imagen se debe poner un código con dos partes:

- `![texto alternativo]`, que es el texto que se va a mostrar si la imagen no pudiera visualizarse,
- Seguido de `(nombrefichero.extension)`, que es el archivo imagen (con su dirección).

```
1 ![texto alternativo](file-name.md)
```

Un ejemplo:

[logo markdown](assets/markdown_logo.png)

La visualización de la imagen anterior:



Código de bloque

Uno de los puntos más útiles de Markdown a la hora de crear un documento con texto específico de informática es que admite la colocación de bloques de código tanto en línea como en un bloque "delimitado" independiente entre frases.

Para ello utilizaremos:

- Dos comillas invertidas ```` si queremos escribir código dentro de la misma línea de texto del párrafo.
- Si queremos crear un bloque de código multilínea, con un lenguaje específico, pondremos ````` seguido del nombre del lenguaje del bloque.

Unos ejemplos:

- En la misma línea:

...estamos escribiendo un párrafo ```insertar el bloque` y seguimos escribiendo...

- Un bloque de código:

````javascript` y escribimos el código.

```
1 function holamundo(){
2 console.log ("hola mundo web");
3 }
```

### Línea horizontal

Para crear una línea horizontal, de separación de contenido por ejemplo, se añaden tres guiones: `---`

Visualización:

---

### Insertar emojis

Para insertar emojis basta con utilizar `:` seguido del nombre del emoji y cerrar con otro `:`.

Podemos observar, que en algunos editores markdown, al escribir, por ejemplo, `:a` nos muestra todos los emojis con la inicial **a**.

Por ejemplo: `: star :`

Visualización: 🌟

### Crear diagrama de flujo

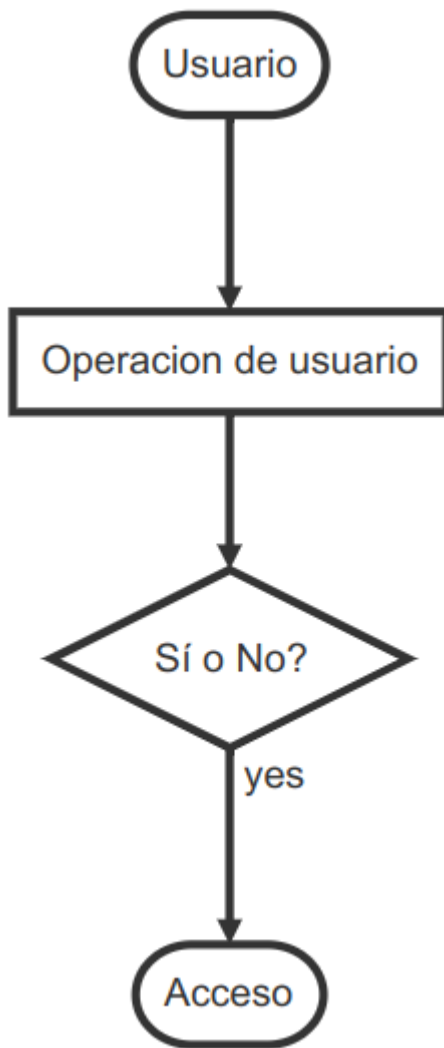
Cuando queremos crear documentos con elementos gráficos como diagramas de flujo, debemos generar una especie de *código* para construirlos.

- Por eso, comenzaremos introduciendo la línea de inicio: ````flow`
- Es conveniente asignar un nombre (por ejemplo: st, op, cond, e...) a cada elemento que conforma el diagrama; así, después podremos unir todos estos.
- Forma de inicio: `st=>start: Nombre`
- Forma de fin: `e=>end: Nombre`
- Rectángulo: `op=>operation: texto de nombre`
- Condición: `cond=>condition: texto de la condición (Si o No?)`
- Subrutina: `sub1=>subroutine: nombre subtarea`
- EntradaSalida: `io1=>inputoutput: nombre elemento entrada/salida`
- Líneas: `st->op->cond`
- Caminos de condiciones: `cond(yes)**->e` y `cond(no)->op`
- Línea de cierre: `````

Ejemplo:

```
1 ```flow
2 st=>start: Usuario
3 e=>end: Acceso
4 op=>operation: Operacion de usuario
5 cond=>condition: Si o No?
6 st->op->cond
7 cond(yes)->e
8 cond(no)->op
9 ```
```

Visualización:



### ? Actividad

Intenta realizar un diagrama para "programar" un almuerzo. En él, deberás dar los **buenos días**, indicar que **es hora del descanso**, y preguntar si **alguién quiere almorzar**. Si no hay nadie que quiera almorzar contigo, debes **ir a otro grupo de amigos** y volver a indicar que **es hora del descanso**. Si alguien sí quiere almorzar **escribe en la pizarra que os vais a almorzar y sal al patio**.

### Crear secuencias

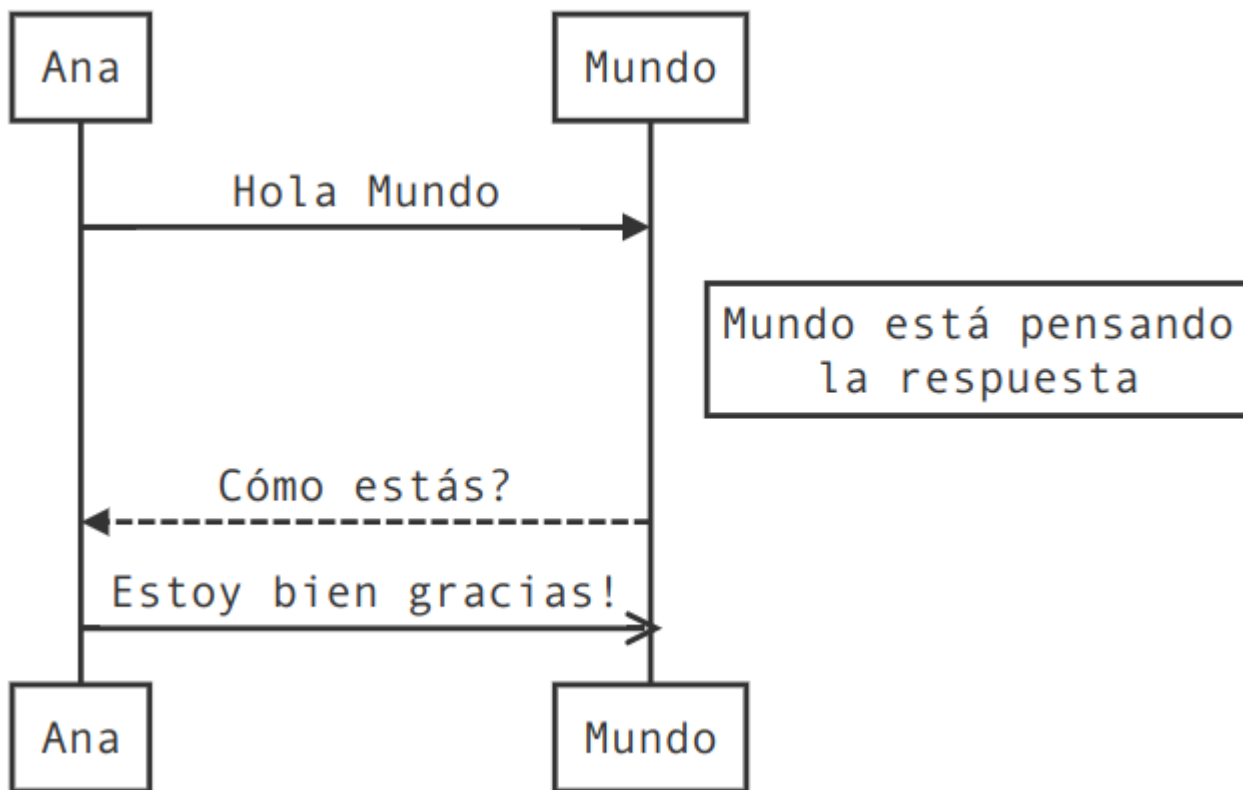
En la secuenciación podemos observar que es bastante parecido a la creación de diagramas; pero la primera línea (crear un bloque de código) no será **flow** sino **sequence**.

```

1 ```sequence
2 Ana->Mundo: Hola Mundo
3 Note right of Mundo: Mundo está pensando\nla respuesta
4 Mundo-->Ana: Cómo estás?
5 Ana->>Mundo: Estoy bien gracias!
6 ```

```

Visualización:



#### Crear índice

Para crear el índice a partir de los encabezados creados debemos insertar `[TOC]` .

#### Resumen

##### Resumen

Markdown te permite escribir documentos con formato usando texto plano. Sus elementos principales son: encabezados (`#`), énfasis (`**negrita**`, `*cursiva*`), listas, enlaces, imágenes, bloques de código, tablas, citas (`>`) y diagramas. Es ideal para documentación técnica, apuntes y páginas web estáticas.

#### Tarea

Como tarea, se propone:

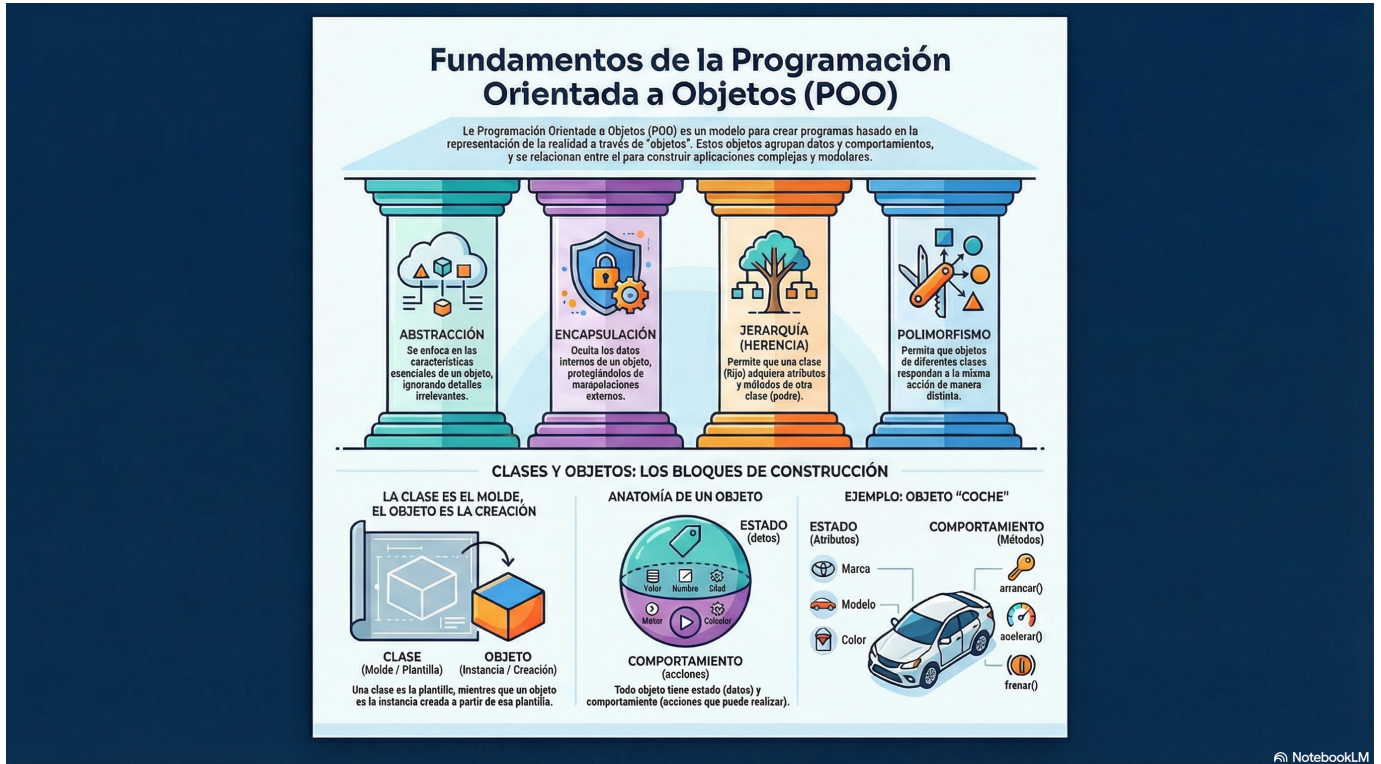
- Crear un documento markdown en tu editor markdown favorito (por ejemplo Typora o VSCode) que documente información acerca de tí mismo.
- En dicho documento crear título, índice.
- Añadir 4 encabezados principales (y otros encabezados secundarios dentro de éstos) en el que hables por ejemplo de: *Tus datos*, *Currículum*, *Aficiones* y *Otros datos de interés*. No hace falta que indiques información personal relevante. (O te la puedes inventar)
- Se valorará la inclusión de distintos elementos como: negrita-cursiva-subrayado, listas ordenadas-desordenadas-tareas, enlaces, imágenes, citas, código, etc.
- Si te atreves con ello, crea un diagrama de flujo en el que indiques los pasos que realizas un sábado por la mañana.
- Exporta el documento a pdf.

Subir a la plataforma **AULES** un documento Markdown (*.md*) y otro documento PDF (*.pdf*) que sea la exportación del primero.

 19 de julio de 2026

## 3. UD02

### 3.1 Utilización de Objetos y Clases



NotebookLM



**Al finalizar esta unidad serás capaz de...**

- ✓ Comprender los fundamentos de la Programación Orientada a Objetos
- ✓ Identificar las características de los objetos: identidad, estado y comportamiento
- ✓ Definir clases con atributos y métodos
- ✓ Instanciar objetos a partir de clases predefinidas
- ✓ Utilizar métodos y propiedades de los objetos
- ✓ Diferenciar entre parámetros y argumentos
- ✓ Conocer el uso de constructores

#### 3.1.1 Introducción a la POO

**Orientado a objetos** hace referencia a una forma diferente de acometer la tarea del desarrollo de software, frente a otros modelos como el de la programación imperativa, la programación funcional o la programación lógica. Supone una reconsideración de los métodos de programación, de la forma de estructurar la información y, ante todo, de la forma de pensar en la resolución de problemas.

La **programación orientada a objetos (POO)** es un modelo para la elaboración de programas que ha impuesto en los últimos años. Este auge se debe, en parte, a que esta forma de programar está fuertemente basada en la representación de la realidad; pero también a que refuerza el uso de buenos criterios aplicables al desarrollo de programas.



#### Atención

La orientación a objetos no es un tipo de lenguaje de programación. Es una metodología de trabajo para crear programas.

En POO, un programa es una colección de objetos que se relacionan entre sí de distintas formas.

### 3.1.2 Características de la POO

Cuando hablamos de Programación Orientada a Objetos, existen una serie de características que se deben cumplir. Cualquier lenguaje de programación orientado a objetos las debe contemplar. Las características más importantes del paradigma de la programación orientada a objetos son:

- **Abstracción.** Es el proceso por el cual definimos las características más importantes de un objeto, sin preocuparnos de cómo se escribirán en el código del programa, simplemente lo definimos de forma general. En la Programación Orientada a Objetos la herramienta más importante para soportar la abstracción es la clase. Básicamente, una clase es un tipo de dato que agrupa las características comunes de un conjunto de objetos. Poder ver los objetos del mundo real que deseamos trasladar a nuestros programas, en términos abstractos, resulta de gran utilidad para un buen diseño del software, ya que nos ayuda a comprender mejor el problema y a tener una visión global de todo el conjunto. Por ejemplo, si pensamos en una clase Vehículo que agrupa las características comunes de todos ellos, a partir de dicha clase podríamos crear objetos como Coche y Camión. Entonces se dice que Vehículo es una abstracción de Coche y de Camión.
- **Modularidad.** Una vez que hemos representado el escenario del problema en nuestra aplicación, tenemos como resultado un conjunto de objetos software a utilizar. Este conjunto de objetos se crean a partir de una o varias clases. Cada clase se encuentra en un archivo diferente, por lo que la modularidad nos permite modificar las características de la clase que define un objeto, sin que esto afecte al resto de clases de la aplicación.
- **Encapsulación.** También llamada "ocultamiento de la información". La encapsulación o encapsulamiento es el mecanismo básico para ocultar la información de las partes internas de un objeto a los demás objetos de la aplicación. Con la encapsulación un objeto puede ocultar la información que contiene al mundo exterior, o bien restringir el acceso a la misma para evitar ser manipulado de forma inadecuada. Por ejemplo, pensemos en un programa con dos objetos, un objeto Persona y otro Coche. Persona se comunica con el objeto Coche para llegar a su destino, utilizando para ello las acciones que Coche tenga definidas como por ejemplo conducir. Es decir, Persona utiliza Coche pero no sabe cómo funciona internamente, sólo sabe utilizar sus métodos o acciones.
- **Jerarquía.** Mediante esta propiedad podemos definir relaciones de jerarquías entre clases y objetos. Las dos jerarquías más importantes son la jerarquía "es un" llamada generalización o especialización y la jerarquía "es parte de", llamada agregación. Conviene detallar algunos aspectos:
  - La generalización o especialización, también conocida como herencia, permite crear una clase nueva en términos de una clase ya existente (herencia simple) o de varias clases ya existentes (herencia múltiple). Por ejemplo, podemos crear la clase CochedeCarreras a partir de la clase Coche, y así sólo tendremos que definir las nuevas características que tenga.
  - La agregación, también conocida como inclusión, permite agrupar objetos relacionados entre sí dentro de una clase. Así, un Coche está formado por Motor, Ruedas, Frenos y Ventanas. Se dice que Coche es una agregación y Motor, Ruedas, Frenos y Ventanas son agregados de Coche.
- **Polimorfismo.** Esta propiedad indica la capacidad de que varias clases creadas a partir de una antecesora realicen una misma acción de forma diferente. Por ejemplo, pensemos en la clase `Animal` y la acción de expresarse. Nos encontramos que cada tipo de `Animal` puede hacerlo de manera distinta, los Perros ladran, los Gatos maullan, las Personas hablamos, etc. Dicho de otra manera, el polimorfismo indica la posibilidad de tomar un objeto (de tipo `Animal`, por ejemplo), e indicarle que realice la acción de expresarse, esta acción será diferente según el tipo de mamífero del que se trate.

### 3.1.3 Objetos y Clases

#### Características de los objetos

En este contexto, un objeto de software es una representación de un objeto del mundo real, compuesto de una serie de características y un comportamiento específico. Pero ¿qué es más concretamente un objeto en Programación Orientada a Objetos? Veámoslo.

#### Definición

Un objeto es un conjunto de datos con las operaciones definidas para ellos. Los objetos tienen un estado y un comportamiento.

Por tanto, estudiando los objetos que están presentes en un problema podemos dar con la solución a dicho problema. Los objetos tienen unas características fundamentales que los distinguen:

- **Identidad.** Es la característica que permite diferenciar un objeto de otro. De esta manera, aunque dos objetos sean exactamente iguales en sus atributos, son distintos entre sí. Puede ser una dirección de memoria, el nombre del objeto o cualquier otro elemento que utilice el lenguaje para distinguirlos. Por ejemplo, dos vehículos que hayan salido de la misma cadena de fabricación y sean iguales aparentemente, son distintos porque tienen un código que los identifica.

- **Estado.** El estado de un objeto viene determinado por una serie de parámetros o atributos que lo describen, y los valores de éstos. Por ejemplo, si tenemos un objeto `Coche`, el estado estaría definido por atributos como `Marca`, `Modelo`, `Color`, `Cilindrada`, etc.
- **Comportamiento.** Son las acciones que se pueden realizar sobre el objeto. En otras palabras, son los métodos o procedimientos que realiza el objeto. Siguiendo con el ejemplo del objeto `Coche`, el comportamiento serían acciones como: `arrancar()`, `parar()`, `acelerar()`, `frenar()`, etc. Definición de clases.

Una clase java se escribe en un fichero con extensión `.java` que tiene el mismo nombre que la clase. Por ejemplo la clase `Vehículo` se escribiría en el fichero `Vehiculo.java`.

Cuando la clase se compila se obtiene un fichero con el mismo nombre que la clase y extensión `.class`. Ej.: `Vehiculo.class`.



#### Atención

Los identificadores de clase siguen las mismas reglas que otros identificadores de Java (contienen carácter alfanuméricos y especiales, no pueden comenzar por un dígito, no pueden coincidir con una palabra reservada, etc.). Por convenio los identificadores de las clases comienzan por mayúsculas.

### Propiedades y métodos de los objetos

Como acabamos de ver todo objeto tiene un estado y un comportamiento. Concretando un poco más, las partes de un objeto son:

- **Campos, Atributos o Propiedades:** Parte del objeto que almacena los datos. También se les denomina Variables Miembro. Estos datos pueden ser de cualquier tipo primitivo (`boolean`, `char`, `int`, `double`, etc) o ser a su vez otro objeto. Por ejemplo, un objeto de la clase `Coche` puede tener un objeto de la clase `Ruedas` (o más concretamente cuatro).
- **Métodos o Funciones Miembro:** Parte del objeto que lleva a cabo las operaciones sobre los atributos definidos para ese objeto.

La idea principal es que el objeto reúne en una sola entidad los datos y las operaciones, y para acceder a los datos privados del objeto debemos utilizar los métodos que hay definidos para ese objeto.

La única forma de manipular la información del objeto es a través de sus métodos. Es decir, si queremos saber el valor de algún atributo, tenemos que utilizar el método que nos muestre el valor de ese atributo. De esta forma, evitamos que métodos externos puedan alterar los datos del objeto de manera inadecuada. Se dice que los datos y los métodos están encapsulados dentro del objeto.

#### ATRIBUTOS

Los atributos representan la información que almacenan los objetos de la clase. Los atributos son declaraciones de variables dentro de la clase.

Se sigue la siguiente sintaxis (los corchetes indican opcionalidad):

```
1 [ámbito] tipo nombreDelAtributo;
2 [ámbito] tipo nombreDelAtributo1, nombreDelAtributo2, ...;
```

donde ...

- **ámbito** permite indicar desde qué clases es accesible el atributo.
- **tipo** indica el tipo de dato del atributo.
- **nombreDelAtributo** es el identificador del atributo.

#### MÉTODOS

Los métodos determinan qué puede hacer un objeto de la clase, es decir, su comportamiento.

Los métodos realizan algún tipo de acción o tarea y, en ocasiones, devuelven un resultado.

Para realizar su trabajo puede ser necesario que pasemos al método cierta información. Por ejemplo, cuando llamamos al método `round` de la clase `Math`, para redondear un número real, debemos indicar al método cual es el número que queremos redondear. A esa información que pasamos a los métodos se le llama **parámetros o argumentos**.



### Ejemplo

Consulta el [Ejemplo01](#) para ver la sintaxis de declaración de métodos.

En la definición de un método se distinguen dos partes

- **La cabecera**, en la cual se indica información relevante sobre el método.
- **El cuerpo**, que contiene las instrucciones mediante las cuales el método realiza su tarea.

Para definirlos, se sigue la siguiente sintaxis (los corchetes indican opcionalidad):



### Ejemplo

Consulta el [Ejemplo01](#) para ver la sintaxis de declaración de métodos. donde...

- **ámbito** permite indicar desde qué clases es accesible el método.
- **static**, cuando aparece, indica que el método es estático.
- **tipoDevuelto** indica el tipo de dato que devuelve el método. La palabra reservada *void* (que no es ningún tipo de dato), indicaría que el método no devuelve nada.
- **nombreDelMetodo** es el identificador del método
- **parámetros** es una lista, separada por comas, de los parámetros que recibe el método. De cada parámetro se indica el **tipo** y un **identificador**.

### Interacción entre objetos

Dentro de un programa los objetos se comunican llamando unos a otros a sus métodos. Los métodos están dentro de los objetos y describen el comportamiento de un objeto cuando recibe una llamada a uno de sus métodos. En otras palabras, cuando un objeto, `objeto1`, quiere actuar sobre otro, `objeto2`, tiene que ejecutar uno de sus métodos. Entonces se dice que el `objeto2` recibe un mensaje del `objeto1`.

Un mensaje es la acción que realiza un objeto. Un método es la función o procedimiento al que se llama para actuar sobre un objeto.

Los distintos mensajes que puede recibir un objeto o a los que puede responder reciben el nombre de **protocolo** de ese objeto.

El proceso de interacción entre objetos se suele resumir diciendo que se ha "enviado un mensaje" (hecho una petición) a un objeto, y el objeto determina "qué hacer con el mensaje" (ejecuta el código del método). Cuando se ejecuta un programa se producen las siguientes acciones:

- **Creación** de los objetos a medida que se necesitan.
- **Comunicación** entre los objetos mediante el envío de mensajes unos a otros, o el usuario a los objetos.
- **Eliminación** de los objetos cuando no son necesarios para dejar espacio libre en la memoria del computador.



### Recuerda

Los objetos se pueden comunicar entre ellos invocando a los métodos de los otros objetos.

### Clases

Hasta ahora hemos visto lo que son los objetos. Un programa informático se compone de muchos objetos, algunos de los cuales comparten la misma estructura y comportamiento. Si tuviéramos que definir su estructura y comportamiento del objeto cada vez que queremos crear un objeto, estaríamos utilizando mucho código redundante. Por ello lo que se hace es crear una clase, que es una descripción de un conjunto de objetos que comparten una estructura y un comportamiento común. Y a partir de la clase, se crean tantas "copias" o "instancias" como necesitemos. Esas copias son los objetos de la clase.

**Recuerda**

Las clases constan de datos y métodos que resumen las características comunes de un conjunto de objetos. Un programa informático está compuesto por un conjunto de clases, a partir de las cuales se crean objetos que interactúan entre sí.

En otras palabras, una clase es una plantilla o prototipo donde se especifican:

- Los **atributos** comunes a todos los objetos de la clase.
- Los **métodos** que pueden utilizarse para manejar esos objetos.

Para declarar una clase en Java se utiliza la palabra reservada `class`. La declaración de una clase está compuesta por:

- **Cabecera de la clase.** La cabecera es un poco más compleja que como aquí definimos, pero por ahora sólo nos interesa saber que está compuesta por una serie de modificadores, en este caso hemos puesto `public` que indica que es una clase pública a la que pueden acceder otras clases del programa, la palabra reservada `class` y el nombre de la clase.
- **Cuerpo de la clase.** En él se especifican encerrados entre llaves los atributos y los métodos que va a tener la clase.

**Ejemplo**

Consulta el [Ejemplo02](#) para ver la estructura básica de una clase Java.

**Ejemplo**

En la unidad anterior ya hemos utilizado clases, aunque aún no sabíamos su significado exacto. Por ejemplo, en los ejemplos de la unidad o en la tarea, estábamos utilizando clases, todas ellas eran clases principales, no tenían ningún atributo y el único método del que disponían era el método `main()`.

**Ejemplo**

También es una clase `Math` y su método era `random()`, el que nos permitía usar números aleatorios.

**Ejemplo**

El método `main()` se utiliza para indicar que se trata de una clase principal, a partir de la cual va a empezar la ejecución del programa. Este método no aparece si la clase que estamos creando no va a ser la clase principal del programa.

¿QUÉ SIGNIFICA `public class`?

Significa que la clase que se define es pública. Una clase pública es una clase accesible desde otras clases o, dicho de otra forma, que puede ser utilizada por otras clases. Ya hemos dicho que un programa, de alguna manera, consiste en la creación de objetos de distintas clases, que se relacionan entre sí. Lo más común es que las clases que definimos sean públicas y que en cada fichero de extensión `.java` se defina una única clase.

Sin embargo, en ocasiones se definen clases ( `A` ) que solo van a ser utilizadas por una clase determinada ( `B` ). En ese caso, decimos que la clase `A` es una clase privada de la clase `B`. Las clases `A` y `B` se definen en el mismo fichero `.java`.

**Atención**

En un fichero pueden definirse varias clases pero solo una de ellas puede ser pública. De esta forma, si en un fichero se definen varias clases, una de ellas sería pública y el resto serían clases privadas de la primera, a las que solo ésta tendría acceso.

### 3.1.4 Utilización de Objetos

Una vez que hemos creado una clase, podemos crear objetos en nuestro programa a partir de esas clases.

Cuando creamos un objeto a partir de una clase se dice que hemos creado una "instancia de la clase". A efectos prácticos, "objeto" e "instancia de clase" son términos similares. Es decir, nos referimos a objetos como instancias cuando queremos hacer hincapié que son de una clase particular.

Los objetos se crean a partir de las clases, y representan casos individuales de éstas.



### Ejemplo

Para entender mejor el concepto entre un objeto y su clase, piensa en un molde de galletas y las galletas. El molde sería la clase, que define las características del objeto, por ejemplo su forma y tamaño. Las galletas creadas a partir de ese molde son los objetos o instancias.

Otro ejemplo, imagina una clase `Persona` que reúna las características comunes de las personas (`color de pelo`, `ojos`, `peso`, `altura`, etc.) y las acciones que pueden realizar (`crecer`, `dormir`, `comer`, etc.). Posteriormente dentro del programa podremos crear un objeto `Trabajador` que esté basado en esa clase `Persona`. Entonces se dice que el objeto `Trabajador` es una instancia de la clase `Persona`, o que la clase `Persona` es una abstracción del objeto `Trabajador`.

Cualquier objeto instanciado de una clase contiene una copia de todos los atributos definidos en la clase. En otras palabras, lo que estamos haciendo es reservar un espacio en la memoria del ordenador para guardar sus atributos y métodos. Por tanto, cada objeto tiene una zona de almacenamiento propia donde se guarda toda su información, que será distinta a la de cualquier otro objeto. A las variables miembro instanciadas también se les llama variables instancia. De igual forma, a los métodos que manipulan esas variables se les llama métodos instancia.

En el ejemplo del objeto `Trabajador`, las variables instancia serían `color_de_pelo`, `peso`, `altura`, etc. Y los métodos instancia serían `crecer()`, `dormir()`, `comer()`, etc.

### Ciclo de vida de los objetos

Todo programa en Java parte de una única clase, que como hemos comentado se trata de la clase principal.

Esta clase ejecutará el contenido de su método `main()`, el cual será el que utilice las demás clases del programa, cree objetos y lance mensajes a otros objetos.

Las instancias u objetos tienen un tiempo de vida determinado. Cuando un objeto no se va a utilizar más en el programa, es destruido por el recolector de basura para liberar recursos que pueden ser reutilizados por otros objetos.

A la vista de lo anterior, podemos concluir que los objetos tienen un ciclo de vida, en el cual podemos distinguir las siguientes fases:

- **Creación**, donde se hace la reserva de memoria e inicialización de atributos.
- **Manipulación**, que se lleva a cabo cuando se hace uso de los atributos y métodos del objeto.
- **Destrucción**, eliminación del objeto y liberación de recursos.

### Declaración

Para la creación de un objeto hay que seguir los siguientes pasos:

- **Declaración**: Definir el tipo de objeto.
- **Instanciación**: Creación del objeto utilizando el operador `new`. Pero ¿en qué consisten estos pasos a nivel de programación en Java? Veamos primero cómo declarar un objeto. Para la definición del tipo de objeto debemos emplear la siguiente instrucción:

```
1 <tipo> nombre_objeto;
```

Donde:

- **tipo** es la clase a partir de la cual se va a crear el objeto, y
- **nombre\_objeto** es el nombre de la variable referencia con la cual nos referiremos al objeto.

Los tipos referenciados o referencias se utilizan para guardar la dirección de los datos en la memoria del ordenador.

Nada más crear una referencia, ésta se encuentra vacía. Cuando una referencia a un objeto no contiene ninguna instancia se dice que es una referencia nula, es decir, que contiene el valor `null`.

Esto quiere decir que la referencia está creada pero que el objeto no está instanciado todavía, por eso la referencia apunta a un objeto inexistente llamado "nulo".

Para entender mejor la declaración de objetos veamos un ejemplo. Cuando veíamos los tipos de datos, decíamos que Java proporciona un tipo de dato especial para los textos o cadenas de caracteres que era el tipo de dato `String`. Veíamos que realmente este tipo de dato es un tipo referenciado y creábamos una variable `mensaje` de ese tipo de dato de la siguiente forma:

```
1 String mensaje;
```

Los nombres de la clase empiezan con mayúscula, como `String`, y los nombres de los objetos con minúscula, como `mensaje`, así sabemos qué tipo de elemento utilizando.

Pues bien, `String` es realmente la clase a partir de la cual creamos nuestro objeto llamado `mensaje` 🤖.

Si observas, poco se diferencia esta declaración de las declaraciones de variables que hacíamos para los tipos primitivos. Antes decíamos que `mensaje` era una variable del tipo de dato `String`. Ahora realmente vemos que `mensaje` es un objeto de la clase `String`. Pero `mensaje` aún no contiene el objeto porque no ha sido instanciado, veamos cómo hacerlo.

Por tanto, cuando creamos un objeto estamos haciendo uso de una variable que almacena la dirección de ese objeto en memoria. Esa variable es una referencia o un tipo de datos referenciado, porque no contiene el dato si no la posición del dato en la memoria del ordenador.

```
1 String saludo = new String("Bienvenido a Java");
2 String s; //s vale null
3 s = saludo; //asignación de referencias
```

En las instrucciones anteriores, las variables `s` y `saludo` apuntan al mismo objeto de la clase `String`. Esto implica que cualquier modificación en el objeto `saludo` modifica también el objeto al que hace referencia la variable `s`, ya que realmente son el mismo.

### Instanciación

Una vez creada la referencia al objeto, debemos crear la instancia u objeto que se va a guardar en esa referencia. Para ello utilizamos la orden `new` con la siguiente sintaxis:

```
1 [<tipo>] nombre_objeto = new <Constructor_de_la_Clase>([<par1>, <par2>, ..., <parN>]);
```

Donde:

- **nombre\_objeto** es el nombre de la variable referencia con la cual nos referiremos al objeto.
- **new** es el operador para crear el objeto.
- **Constructor\_de\_la\_Clase** es un método especial de la clase, que se llama igual que ella, y se encarga de inicializar el objeto, es decir, de dar unos valores iniciales a sus atributos.
- **par1-parN**, son parámetros que puede o no necesitar el constructor para dar los valores iniciales a los atributos del objeto.

Durante la instanciación del objeto, se reserva memoria suficiente para el objeto. De esta tarea se encarga Java y juega un papel muy importante el `recolector de basura`, que se encarga de eliminar de la memoria los objetos no utilizados para que ésta pueda volver a ser utilizada.

De este modo, para instanciar un objeto `String`, haríamos lo siguiente:

```
1 mensaje = new String;
```

Así estaríamos instanciando el objeto `mensaje`. Para ello utilizaríamos el operador `new` y el constructor de la clase `String` a la que pertenece el objeto según la declaración que hemos hecho en el apartado anterior. A continuación utilizamos el constructor, que se llama igual que la clase, `String`.

En el ejemplo anterior el objeto se crearía con la cadena vacía (`"`), si queremos que tenga un contenido debemos utilizar parámetros en el constructor, así:

```
1 mensaje = new String ("El primer programa");
```

Java permite utilizar la clase `String` como si de un tipo de dato primitivo se tratara, por eso no hace falta utilizar el operador `new` para instanciar un objeto de la clase `String` (pero no es lo habitual en el resto de clases).

```
1 mensaje = "El primer programa";
```

La declaración e instanciación de un objeto puede realizarse en la misma instrucción, así:

```
1 String mensaje = new String ("El primer programa");
```

o para la clase `String`:

```
1 String mensaje = "El primer programa";
```

### Manipulación

Una vez creado e instanciado el objeto ¿cómo accedemos a su contenido? Para acceder a los atributos y métodos del objeto utilizaremos el nombre del objeto seguido del operador punto ( `.` ) y el nombre del **atributo** o **método** que queremos utilizar. Cuando utilizamos el operador `punto` se dice que estamos enviando un mensaje al objeto. La forma general de enviar un mensaje a un objeto es:

```
1 nombre_objeto.mensaje
```

Por ejemplo, para acceder a las variables instancia o atributos se utiliza la siguiente sintaxis:

```
1 nombre_objeto.atributo
```

Y para acceder a los métodos o funciones miembro del objeto se utiliza la sintaxis es:

```
1 nombre_objeto.método([par1, par2, ..., parN])
```

En la sentencia anterior `par1`, `par2`, etc. son los parámetros que utiliza el método. (*Aparece entre corchetes para indicar son opcionales*).

Para entender mejor cómo se manipulan objetos vamos a utilizar un ejemplo. Para ello necesitamos la Biblioteca de Clases Java o API (Application Programming Interface - Interfaz de programación de aplicaciones). Uno de los paquetes de librerías o bibliotecas es `java.awt`. Este paquete contiene clases destinadas a la creación de objetos gráficos e imágenes. Vemos por ejemplo cómo crear un rectángulo.

En primer lugar, instanciamos el objeto utilizando el método constructor, que se llama igual que el objeto, e indicando los parámetros correspondientes a la posición y a las dimensiones del rectángulo:

```
1 Rectangle rect = new Rectangle(50, 50, 150, 150);
```

Una vez instanciado el objeto rectángulo si queremos cambiar el valor de los atributos utilizamos el operador punto. Por ejemplo, para cambiar la dimensión del rectángulo:

```
1 rect.height=100;
2 rect.width=100;
```

O bien podemos utilizar un método para hacer lo anterior:

```
1 rect.setSize(200, 200);
```



### Ejemplo

Consulta el [Ejemplo03](#) para ver el código completo de manipulación de objetos `Rectangle`.

### Destrucción de objetos y liberación de memoria

Cuando un objeto deja de ser utilizado, es necesario liberar el espacio de memoria y otros recursos que poseía para poder ser reutilizados por el programa. A esta acción se le denomina destrucción del objeto.

En Java la destrucción de objetos corre a cargo del **recolector de basura (garbage collector)**. Es un sistema de destrucción automática de objetos que ya no son utilizados. Lo que se hace es liberar una zona de memoria que había sido reservada previamente mediante el operador `new`. Esto evita que los programadores tengan que preocuparse de realizar la liberación de memoria.

El recolector de basura se ejecuta en modo segundo plano y de manera muy eficiente para no afectar a la velocidad del programa que se está ejecutando. Lo que hace es que periódicamente va buscando objetos que ya no son referenciados, y cuando encuentra alguno lo marca para ser eliminado.

Después los elimina en el momento que considera oportuno.

Justo antes de que un objeto sea eliminado por el recolector de basura, se ejecuta su método `finalize()`. Si queremos forzar que se ejecute el proceso de finalización de todos los objetos del programa podemos utilizar el método `runFinalization()` de la clase `System`. La clase `System` forma parte de la Biblioteca de Clases de Java y contiene diversas clases para la entrada y salida de información, acceso a variables de entorno del programa y otros métodos de diversa utilidad. Para forzar el proceso de finalización ejecutaríamos:

```
1 System.runFinalization();
```

### 3.1.5 Utilización de Métodos

Los métodos, junto con los atributos, forman parte de la estructura interna de un objeto. Los métodos contienen la declaración de variables locales y las operaciones que se pueden realizar para el objeto, y que son ejecutadas cuando el método es invocado. Se definen en el cuerpo de la clase y posteriormente son instanciados para convertirse en métodos instancia de un objeto.

Para utilizar los métodos adecuadamente es conveniente conocer la estructura básica de que disponen.

Al igual que las clases, los métodos están compuestos por una cabecera y un cuerpo. La cabecera también tiene modificadores, en este caso hemos utilizado `public` para indicar que el método es público, lo cual quiere decir que le pueden enviar mensajes no sólo los métodos del objeto sino los métodos de cualquier otro objeto externo.

Dentro de un método nos encontramos el cuerpo del método que contiene el código de la acción a realizar. Las acciones que un método puede realizar son:

- **Inicializar** los atributos del objeto
- **Consultar** los valores de los atributos
- **Modificar** los valores de los atributos
- **Llamar a otros métodos**, del mismo del objeto o de objetos externos

#### Parámetros y valores devueltos

Los métodos se pueden utilizar tanto para consultar información sobre el objeto como para modificar su estado. La información consultada del objeto se devuelve a través de lo que se conoce como valor de retorno, y la modificación del estado del objeto, o sea, de sus atributos, se hace mediante la lista de parámetros. En general, la lista de parámetros de un método se puede declarar de dos formas diferentes:

- **Por valor.** El valor de los parámetros no se devuelve al finalizar el método, es decir, cualquier modificación que se haga en los parámetros no tendrá efecto una vez se salga del método. Esto es así porque cuando se llama al método desde cualquier parte del programa, dicho método recibe una copia de los argumentos, por tanto cualquier modificación que haga será sobre la copia, no sobre las variables originales.
- **Por referencia.** La modificación en los valores de los parámetros sí tienen efecto tras la finalización del método. Cuando pasamos una variable a un método por referencia lo que estamos haciendo es pasar la dirección del dato en memoria, por tanto cualquier cambio en el dato seguirá modificado una vez que salgamos del método.

#### Atención

En el lenguaje Java, todas las variables se pasan por valor, excepto los objetos que se pasan por referencia.

En Java, la declaración de un método tiene dos restricciones:

- **Un método siempre tiene que devolver un valor (no hay valor por defecto).** Este valor de retorno es el valor que devuelve el método cuando termina de ejecutarse, al método o programa que lo llamó. Puede ser un tipo primitivo, un tipo referenciado o bien el tipo `void`, que indica que el método no devuelve ningún valor.

- **Un método tiene un número fijo de argumentos.** Los argumentos son variables a través de las cuales se pasa información al método desde el lugar del que se llame, para que éste pueda utilizar dichos valores durante su ejecución. Los argumentos reciben el nombre de parámetros cuando aparecen en la declaración del método.

#### Recuerda

El valor de retorno es la información que devuelve un método tras su ejecución.

Según hemos visto en el apartado anterior, la cabecera de un método se declara como sigue:

```
1 public tipo_de_dato_devuelto nombreMetodo (lista_de_parametros);
```

Como vemos, el tipo de dato devuelto aparece después del modificador `public` y se corresponde con el valor de retorno.

La lista de parámetros aparece al final de la cabecera del método, justo después del nombre, encerrados entre signos de paréntesis y separados por comas. Se debe indicar el tipo de dato de cada parámetro así:

```
1 (tipo_parámetro1 nombre_parámetro1, ..., tipo_parámetroN nombre_parámetroN)
```

#### Atención

Cuando se llame al método, se deberá utilizar el nombre del método, seguido de los argumentos que deben coincidir con la lista de parámetros.

La lista de argumentos en la llamada a un método debe coincidir en número, tipo y orden con los parámetros del método, ya que de lo contrario se produciría un error de sintaxis.

### Constructores

¿Recuerdas cuando hablábamos de la creación e instanciación de un objeto? Decíamos que utilizábamos el operador `new` seguido del nombre de la clase y una pareja de abrir-cerrar paréntesis.

Además, el nombre de la clase era realmente el constructor de la misma, y lo definíamos como un método especial que sirve para inicializar valores. En este apartado vamos a ver un poco más sobre los constructores.

Un constructor es un método especial con el mismo nombre de la clase y que no devuelve ningún valor tras su ejecución.

Cuando creamos un objeto debemos instanciarlo utilizando el constructor de la clase. Veamos la clase `Date` proporcionada por la Biblioteca de Clases de Java. Si queremos instanciar un objeto a partir de la clase `Date` tan sólo tendremos que utilizar el constructor seguido de una pareja de abrir-cerrar paréntesis:

```
1 Date fecha = new Date();
```

Con la anterior instrucción estamos creando un objeto `fecha` de tipo `Date`, que contendrá la fecha y hora actual del sistema.

La estructura de los constructores es similar a la de cualquier método, salvo que no tiene tipo de dato devuelto porque no devuelve ningún valor. Está formada por una cabecera y un cuerpo, que contiene la inicialización de atributos y resto de instrucciones del constructor.

El método constructor tiene las siguientes particularidades:

- **El constructor es invocado automáticamente en la creación de un objeto, y sólo esa vez.**
- **Los constructores no empiezan con minúscula**, como el resto de los métodos, ya que se llaman igual que la clase y las clases empiezan con letra mayúscula.
- **Puede haber varios constructores** para una clase.
- Como cualquier método, **el constructor puede tener parámetros** para definir qué valores dar a los atributos del objeto.
- **El constructor por defecto es aquél que no tiene argumentos o parámetros.** Cuando creamos un objeto llamando al nombre de la clase sin argumentos, estamos utilizando el constructor por defecto.
- **Es necesario que toda clase tenga al menos un constructor.** Si no definimos constructores para una clase, y sólo en ese caso, el compilador crea un constructor por defecto vacío, que inicializa los atributos a sus valores por defecto, según del

tipo que sean: `0` para los tipos numéricos, `false` para los `boolean` y `null` para los tipo carácter y las referencias. Dicho constructor lo que hace es llamar al constructor sin argumentos de la superclase (clase de la cual hereda); si la superclase no tiene constructor sin argumentos se produce un error de compilación.



#### Atención

Cuando definimos constructores personalizados, el constructor por defecto deja de existir, y si no definimos nosotros un constructor sin argumentos cuando intentemos utilizar el constructor por defecto nos dará un error de compilación.

#### El operador `this`

Los constructores y métodos de un objeto suelen utilizar el operador `this`. Este operador sirve para referirse a los atributos de un objeto cuando estamos dentro de él. Sobre todo se utiliza cuando existe ambigüedad entre el nombre de un parámetro y el nombre de un atributo, entonces en lugar del nombre del atributo solamente escribiremos `this.nombre_atributo`, y así no habrá duda de a qué elemento nos estamos refiriendo.

#### Métodos estáticos

Cuando trabajábamos con cadenas de caracteres utilizando la clase `String`, veíamos las operaciones que podíamos hacer con ellas: obtener longitud, comparar dos cadenas de caracteres, cambiar a mayúsculas o minúsculas, etc. Pues bien, sin saberlo estábamos utilizando métodos estáticos definidos por Java para la clase `String`. Pero ¿qué son los métodos estáticos? Veámoslo.

Los métodos estáticos son aquellos métodos definidos para una clase que se pueden usar directamente, sin necesidad de crear un objeto de dicha clase. También se llaman métodos de clase.

Para llamar a un método estático utilizaremos:

- **El nombre del método**, si lo llamamos desde la misma clase en la que se encuentra definido.
- **El nombre de la clase**, seguido por el operador punto ( `.` ) más el nombre del método estático, si lo llamamos desde una clase distinta a la que se encuentra definido:

```
1 Nombre_clase.nombre_metodo_estatico
```

- **El nombre del objeto**, seguido por el operador punto ( `.` ) más el nombre del método estático. Utilizaremos esta forma cuando tengamos un objeto instanciado de la clase en la que se encuentra definido el método estático, y no podamos utilizar la anterior:

```
1 nombre_objeto.nombre_metodo_no_estatico
```

Los métodos estáticos no afectan al estado de los objetos instanciados de la clase (variables instancia), y suelen utilizarse para realizar operaciones comunes a todos los objetos de la clase. Por ejemplo, si necesitamos contar el número de objetos instanciados de una clase, podríamos utilizar un método estático que fuera incrementando el valor de una variable entera de la clase conforme se van creando los objetos.

En la Biblioteca de Clases de Java existen muchas clases que contienen métodos estáticos. Pensemos en las clases que ya hemos utilizado en unidades anteriores, como hemos comentado la clase `String` con todas las operaciones que podíamos hacer con ella y con los objetos instanciados a partir de ella. O bien la clase `Math` para la conversión de tipos de datos. Todos ellos son métodos estáticos que la API de Java define para esas clases. Lo importante es que tengamos en cuenta que al tratarse de métodos estáticos, para utilizarlos no necesitamos crear un objeto de dichas clases.

Fijémonos en esta secuencia de instrucciones

```
1 //Creamos dos círculos de radio 100 en distintas posiciones
2 Círculo c1 = new Círculo(50,50,100);
3 Círculo c2 = new Círculo(80,80,100);
4 ...
5 //Aumentamos el radio del primer círculo a 200
6 c1.setRadio(200);
```

y en esta otra

```
1 System.out.println(Math.sqrt(4));
```

En el primer ejemplo, `.setRadio(200)` va precedido por un objeto. La variable `c1` es un objeto de la clase `Círculo`, por tanto, la instrucción está modificando el radio de un círculo concreto, el que se encuentra en la posición (50,50). El método `setRadio` es un método no estático. Los métodos no estáticos actúan siempre sobre algún objeto (el que figura a la izquierda del punto).

En el segundo ejemplo, en cambio, a la izquierda de `.sqrt(4)` no se ha puesto el nombre de un objeto, sino el de una clase, la clase `Math`. El método `sqrt` no está actuando sobre un objeto concreto: no tiene sentido hacerlo, solo pretendemos calcular la raíz cuadrada de 4. `sqrt` es un método estático. Los métodos estáticos se usan poniendo delante del punto el nombre de la clase en que se encuentran definidos.



#### Ejemplo completo: clase Pajaro

Consulta el [Ejemplo05](#) para ver un ejemplo completo de clase con constructores, métodos y el uso de `this`.

### 3.1.6 Librerías de Objetos (Paquetes)

Conforme nuestros programas se van haciendo más grandes, el número de clases va creciendo. Meter todas las clases en único directorio no ayuda a que estén bien organizadas, lo mejor es hacer grupos de clases, de forma que todas las clases que estén relacionadas o traten sobre un mismo tema estén en el mismo grupo.

Un **paquete** de clases es una agrupación de clases que consideramos que están relacionadas entre sí o tratan de un tema común.



#### Recuerda

Las clases de un mismo paquete tienen un acceso privilegiado a los atributos y métodos de otras clases de dicho paquete. Es por ello por lo que se considera que los paquetes son también, en cierto modo, unidades de encapsulación y ocultación de información.

Java nos ayuda a organizar las clases en paquetes. En cada fichero `.java` que hagamos, al principio, podemos indicar a qué paquete pertenece la clase que hagamos en ese fichero.

Los paquetes se declaran utilizando la palabra clave `package` seguida del nombre del paquete.

Para establecer el paquete al que pertenece una clase hay que poner una sentencia de declaración como la siguiente al principio de la clase:

```
1 package nombre_de_Paquete;
```

Por ejemplo, si decidimos agrupar en un paquete `ejemplos` un programa llamado `Bienvenida`, pondríamos en nuestro fichero `Bienvenida.java` lo siguiente:

```
1 package ejemplos;
2
3 public class Bienvenida {
4 [...]
5 }
```

El código es exactamente igual que como hemos venido haciendo hasta ahora, solamente hemos añadido la línea `package ejemplos;` al principio.

#### Sentencia `import`

Cuando queramos utilizar una clase que está en un paquete distinto a la clase que estamos utilizando, se suele utilizar la sentencia `import`. Por ejemplo, si queremos utilizar la clase `Scanner` que está en el paquete `java.util` de la Biblioteca de Clases de Java, tendremos que utilizar esta sentencia:

```
1 import java.util.Scanner;
```

Se pueden importar todas las clases de un paquete, así:

```
1 import java.awt.*;
```

Esta sentencia debe aparecer al principio de la clase, justo después de la sentencia `package`, si ésta existiese.

También podemos utilizar la clase sin sentencia `import`, en cuyo caso cada vez que queramos usarla debemos indicar su ruta completa:

```
1 java.util.Scanner teclado = new java.util.Scanner (System.in);
```

Hasta aquí todo correcto. Sin embargo, al trabajar con paquetes, Java nos obliga a organizar los directorios, compilar y ejecutar de cierta forma para que todo funcione adecuadamente.

### Librerías Java

Cuando descargamos el entorno de compilación y ejecución de Java, obtenemos la API de Java. Como ya sabemos, se trata de un conjunto de bibliotecas que nos proporciona paquetes de clases útiles para nuestros programas. Utilizar las clases y métodos de la Biblioteca de Java nos va ayudar a reducir el tiempo de desarrollo considerablemente, por lo que es importante que aprendamos a consultarla y conozcamos las clases más utilizadas.



### Ejemplo

```
1 import java.lang.System; // Se importa la clase System.
2 import java.awt.*; // Se importa todas las clases del paquete awt;
```

Los paquetes más importantes que ofrece el lenguaje Java son:

Paquete o librería	Descripción
<b>java.io</b>	Contiene las clases que gestionan la entrada y salida, ya sea para manipular ficheros, leer o escribir en pantalla, en memoria, etc. Este paquete contiene por ejemplo la clase <code>BufferedReader</code> que se utiliza para la entrada por teclado.
<b>java.lang</b>	Contiene las clases básicas del lenguaje. Este paquete no es necesario importarlo, ya que es importado automáticamente por el entorno de ejecución. En este paquete se encuentra la clase <code>Object</code> , que sirve como raíz para la jerarquía de clases de Java, o la clase <code>System</code> que ya hemos utilizado en algunos ejemplos y que representa al sistema en el que se está ejecutando la aplicación. También podemos encontrar en este paquete las clases que "envuelven" los tipos primitivos de datos. Lo que proporciona una serie de métodos para cada tipo de dato de utilidad, como por ejemplo las conversiones de datos.
<b>java.util</b>	Biblioteca de clases de utilidad general para el programador. Este paquete contiene por ejemplo la clase <code>Scanner</code> utilizada para la entrada por teclado de diferentes tipos de datos, la clase <code>Date</code> , para el tratamiento de fechas, etc.
<b>java.math</b>	Contiene herramientas para manipulaciones matemáticas.
<b>java.awt</b>	Incluye las clases relacionadas con la construcción de interfaces de usuario, es decir, las que nos permiten construir ventanas, cajas de texto, botones, etc. Algunas de las clases que podemos encontrar en este paquete son <code>Button</code> , <code>TextField</code> , <code>Frame</code> , <code>Label</code> , etc.
<b>java.swing</b>	Contiene otro conjunto de clases para la construcción de interfaces avanzadas de usuario. Los componentes que se engloban dentro de este paquete se denominan componentes <code>Swing</code> , y suponen una alternativa mucho más potente que <code>AWT</code> para construir interfaces de usuario.
<b>java.net</b>	Conjunto de clases para la programación en la red local e Internet.
<b>java.sql</b>	Contiene las clases necesarias para programar en Java el acceso a las bases de datos.
<b>java.security</b>	Biblioteca de clases para implementar mecanismos de seguridad.

Como se puede comprobar Java ofrece una completa jerarquía de clases organizadas a través de paquetes.

### 3.1.7 Cadenas de caracteres. La clase `String`

#### Cadenas de caracteres

Hasta ahora hemos utilizado literales de cadenas de caracteres que, como sabemos, se ponen entre comillas dobles, como en la siguiente expresión

```
1 System.out.println("Hola");
```

Para almacenar cadenas de caracteres en variables se utiliza la clase `String`. `String` se encuentra definida en el paquete `java.lang`. *Recordemos que no es necesario importar este paquete para utilizar sus clases.*

La forma de `String` es la siguiente:

```
1 String variable = new String("texto");
```

Ejemplo:

```
1 String nombre = new String("Javier");
2 System.out.println("Mi nombre es " + nombre);
```

**Sin embargo**, debido a que es una clase que se utiliza ampliamente en los programas, Java permite una forma abreviada de crear objetos `String`:

```
1 String nombreVariable = "texto";
```

Ejemplo:

```
1 String nombre = "Javier";
2 System.out.println("Mi nombre es " + nombre);
```

### Leer cadenas desde teclado

CLASE `Scanner`

Para leer cadenas de caracteres desde teclado podemos utilizar la clase `Scanner`. Ésta dispone de dos métodos para leer cadenas:

- `next()`: Lee desde la entrada estándar (teclado) una secuencia de caracteres hasta encontrar un delimitador (un espacio). Devuelve un `String`.
- `nextLine()`: Lee desde la entrada estándar (teclado) una secuencia de caracteres hasta encontrar un salto de línea. Devuelve un `String`.

Ejemplo:

```
1 Scanner tec = new Scanner(System.in);
2 //De lo que introduce el usuario, lee la 1ª palabra.
3 String nombre = tec.next();
4 //Lee lo que introduce el usuario hasta que pulsa intro.
5 String nombreCompleto = tec.nextLine();
```

EJEMPLOS DE LA UD01 PERO UTILIZANDO `Scanner` (COMPATIBLE CON LOS IDE'S)



### Scanner

Consulta el [Ejemplo04](#) para ver el código completo de lectura por teclado con `Scanner`.

### La clase `String`

Además de permitir almacenar cadenas de caracteres, `String` tiene métodos para realizar cálculos u operaciones con ellas.

Así por ejemplo, la clase tiene un método `toUpperCase()` que devuelve el `String` convertido a mayúsculas. El siguiente ejemplo ilustra su uso:

```
1 String nombre = "Javier";
2 System.out.println(nombre.toUpperCase()); // Se muestra JAVIER por pantalla
```

Accede a la documentación en línea de Java y estudia los siguientes métodos de la clase:

- `charAt`
- `indexOf`

- `subString`
- `toLowerCase`
- `trim`

#### `printf` o `format`

El método `printf()` o `format()` (son sinónimos) utilizan unos códigos de conversión para indicar si el contenido a mostrar es de qué tipo es. Estos códigos se caracterizan porque llevan delante el símbolo `%`, algunos de ellos son:

- `%c` : Escribe un carácter.
- `%s` : Escribe una cadena de texto.
- `%d` : Escribe un entero.
- `%f` : Escribe un número en punto flotante.
- `%e` : Escribe un número en punto flotante en notación científica.

Por ejemplo, si queremos escribir el número float `12345.1684` con el punto de los miles y sólo dos cifras decimales la orden sería:

```
1 System.out.printf("%.2f\n", 12345.1684);
```

Esta orden mostraría el número `12.345,17` por pantalla.

Otro ejemplo sería:

```
1 System.out.format("El valor de la variable float es" +
2 "%f, mientras que el valor del entero es %d" +
3 "y el string contiene %s", variableFloat, variableInt, variableString);
```

Puedes investigar más sobre `printf` o `format` en este [enlace](#)

#### Salida de error

La salida de error está representada por el objeto `System.err`. No parece muy útil utilizar `out` y `err` si su destino es la misma pantalla, o al menos en el caso de la consola del sistema donde las dos salidas son representadas con el mismo color y no notamos diferencia alguna. En cambio en la consola de varios entornos integrados de desarrollo como NetBeans o Eclipse la salida de `err` se ve en un color diferente. Teniendo el siguiente código:

```
1 System.out.println("Salida estándar por pantalla");
2 System.err.println("Salida de error por pantalla");
```

Tanto NetBeans, Eclipse como IntelliJ Idea mostrarán el mensaje `err` en color rojo.



#### Ejemplo completo: String, printf y System.err

Consulta el [Ejemplo06](#) para ver un ejemplo completo con métodos de `String`, `printf/format` y `System.err`.

### 3.1.8 Ejemplos UD02

[Descarga el código fuente completo de los ejemplos](#)

#### Ejemplo01

Sintaxis de declaración de un método en Java.

```
1 public static void main (String[] args)
2 [ámbito] [static] tipoDevuelto nombreDelMetodo ([parámetros]){
3 //Cuerpo del método (instrucciones)
4 ...
5 }
```

[↑ Volver a teoría](#)

## Ejemplo02

Estructura básica de una clase Java.

```

1 //Paquete al que pertenece la clase
2 package NombreDePaquete;
3
4 //Paquetes que importa la clase
5 import ...
6
7 public class NombreDeLaClase {
8 // Atributos de la clase
9 ...
10 // Métodos de la clase
11 ...
12 }
```

[↑ Volver a teoría](#)

---

## Ejemplo03

Manipulación de objetos `Rectangle` de la biblioteca `java.awt`.

```

1 /*
2 * Muestra como se manipulan objetos en Java
3 */
4 import java.awt.Rectangle;
5 public class Manipular {
6 public static void main(String[] args) {
7 // Instanciamos el objeto rect indicando posicion y dimensiones
8 Rectangle rect = new Rectangle(50, 50, 150, 150);
9 //Consultamos las coordenadas x e y del rectangulo
10 System.out.println("----- Coordenadas esquina superior izqda. -----");
11 System.out.println("\tx = " + rect.x + "\n\ty = " + rect.y);
12 // Consultamos las dimensiones (altura y anchura) del rectangulo
13 System.out.println("\n----- Dimensiones -----");
14 System.out.println("\tAlto = " + rect.height);
15 System.out.println("\tAncho = " + rect.width);
16 //Cambiar coordenadas del rectangulo
17 rect.height=100;
18 rect.width=100;
19 rect.setSize(200, 200);
20 System.out.println("\n-- Nuevos valores de los atributos --");
21 System.out.println("\tx = " + rect.x + "\n\ty = " + rect.y);
22 System.out.println("\tAlto = " + rect.height);
23 System.out.println("\tAncho = " + rect.width);
24 }
25 }
```

[↑ Volver a teoría](#)

---

## Ejemplo04

Lectura de datos desde teclado con `Scanner` (compatible con IDEs).

```

1 import java.util.Scanner;
2
3 public class EjemploUD02 {
4
5 public static void main(String[] args) {
6
7 Scanner teclado = new Scanner(System.in);
8
9 //Introducir texto desde teclado
10 String texto;
11 System.out.print("Introduce un texto: ");
12 texto = teclado.nextLine();
13 System.out.println("El texto introducido es: " + texto);
14
15 //Introducir un número entero desde teclado
16 String texto2;
17 int entero2;
18 System.out.print("Introduce un número: ");
19 texto2 = teclado.nextLine();
20 entero2 = Integer.parseInt(texto2);
21 System.out.println("El número introducido es:" + entero2);
22
23 //Introducir un número decimal desde teclado
24 String texto3;
25 double doble3;
26 System.out.print("Introduce un número decimal: ");
27 texto3 = teclado.nextLine();
28 doble3 = Double.parseDouble(texto3); // convertimos texto a doble
29 System.out.println("Número decimal introducido es: " + doble3);
30 }
31 }

```

[↑ Volver a teoría](#)

### Ejemplo05

Clase `Pajaro` completa con constructores y método `volar()`.

```

1 public class Pajaro {
2 //atributos/variables
3 private String nombre;
4 private int posX;
5 private int posY;
6 //constructores
7 public Pajaro() {
8 }
9 public Pajaro(String nombre) {
10 this.nombre = nombre;
11 }
12 public Pajaro(String nombre, int posX, int posY) {
13 this.nombre = nombre;
14 this.posX = posX;
15 this.posY = posY;
16 }
17
18 //metodos
19 public double volar(int posX, int posY) {
20 double desplazamiento = Math.sqrt((posX * posX) + (posY * posY));
21 this.posX = posX;
22 this.posY = posY;
23 return desplazamiento;
24 }
25
26 public static void main(String[] args) {
27 //creamos el objeto con parámetros
28 Pajaro pajaro1 = new Pajaro("WoodPecker", 50, 50);
29 double d1 = pajaro1.volar(50, 50);
30 System.out.println("El desplazamiento de " + pajaro1.nombre + " ha sido " + d1);
31
32 Pajaro pajaro2 = new Pajaro();
33 //damos nombre y cambiamos la posición de "Piolin" a mano
34 pajaro2.nombre = "Piolin";
35 pajaro2.posX = 30;
36 pajaro2.posY = 30;
37 double d2 = pajaro2.volar(pajaro2.posX, pajaro2.posY);
38 System.out.println("El desplazamiento de " + pajaro2.nombre + " ha sido " + d2);
39 }
40 }

```

**Salida:**

```

1 El desplazamiento de WoodPecker ha sido 70.71067811865476
2 El desplazamiento de Piolin ha sido 42.42640687119285

```

[↑ Volver a teoría](#)

Ejemplo06

Métodos de la clase `String`, `printf/format` y `System.err`.

```
1 package UD02;
2
3 import java.util.Scanner;
4
5 public class EjemploUD02 {
6
7 public static void main(String[] args) {
8
9 Scanner teclado = new Scanner(System.in);
10
11 //Introducir texto desde teclado
12 String texto;
13 System.out.print("Introduce un texto: ");
14 texto = teclado.nextLine();
15 System.out.println("El texto introducido es: " + texto);
16
17 //Introducir un número entero desde teclado
18 String texto2;
19 int entero2;
20 System.out.print("Introduce un número: ");
21 texto2 = teclado.nextLine();
22 entero2 = Integer.parseInt(texto2);
23 System.out.println("El número introducido es: " + entero2);
24
25 //Introducir un número decimal desde teclado
26 String texto3;
27 double doble3;
28 System.out.print("Introduce un número decimal: ");
29 texto3 = teclado.nextLine();
30 doble3 = Double.parseDouble(texto3);
31 System.out.println("Número decimal introducido es: " + doble3);
32
33 System.out.println("La clase String");
34 String nombre = "Javier "; //Observa que hay un espacio final
35 System.out.println(nombre.toUpperCase()); //JAVIER
36 System.out.println(nombre.charAt(4)); //e
37 System.out.println(nombre.indexOf("i")); //3
38 System.out.println(nombre.substring(0, 3)); //Javi
39 System.out.println(nombre.toLowerCase()); //javier
40 System.out.println(nombre.trim()); //Javier sin espacios finales
41 System.out.printf("%.2f\n", 12345.1684);
42 nombre.toUpperCase().substring(0,3).indexOf("I"); //3
43 System.out.format("El valor de la variable float es %f"
44 + ", mientras que el valor del entero es %d"
45 + " y el string contiene %s", doble3, entero2, texto);
46
47 System.err.println("Salida de error por pantalla");
48 }
49 }
```

[↑ Volver a teoría](#)

3.1.9 Resumen — Conceptos clave

Concepto	Definición
POO	Paradigma de programación basado en objetos que representan entidades del mundo real
Clase	Plantilla o modelo que define atributos y métodos comunes a un conjunto de objetos
Objeto	Instancia concreta de una clase, con identidad, estado y comportamiento propios
Atributo	Dato que almacena el estado de un objeto
Método	Operación que define el comportamiento de un objeto
Encapsulación	Mecanismo que oculta los detalles internos de un objeto
Constructor	Método especial que inicializa un objeto al crearlo

### 3.1.10 Autoevaluación

- ✓ Entiendo los conceptos básicos de la POO
- ✓ Sé crear una clase con atributos y métodos
- ✓ Puedo instanciar objetos y llamar a sus métodos
- ✓ Comprendo la diferencia entre parámetros y argumentos
- ✓ Sé utilizar constructores
- ✓ Reconozco la importancia de la encapsulación

### 3.1.11 Vídeos recomendados

Canal	Vídeo	Contenido
Programación ATS	<a href="#">Playlist completa del curso Java</a>	Vídeos 1-9: introducción, objetos, clases, Scanner
Píldoras Informáticas	<a href="#">Curso Java 2026 — Estructuras principales</a>	Vídeos 2-22: tipos, variables, Scanner, operadores
MoureDev	<a href="#">Curso Completo de Java desde Cero</a>	Sección de objetos, clases y Strings
DiscoDurodeRoer	<a href="#">Variables y tipos</a>	Variables en Java
DiscoDurodeRoer	<a href="#">Scanner — entrada de datos</a>	Lectura por teclado con Scanner

🕒 21 de julio de 2026

## 3.2 Ejercicios de la UD02

### 3.2.1 Actividades

1. (Temperatura) Crear una clase llamada `Temperatura` con dos métodos:

- `celsiusToFahrenheit`. Convierte grados *Celsius* a *Fahrenheit*.

$$F = (1,8 * C) + 32$$

- `fahrenheitToCelsius`. Convierte grados *Fahrenheit* a *Celsius*.

$$C = \frac{F - 32}{1,8}$$

2. (Moto) A partir de la siguiente clase:

```
1 public class Moto {
2
3 private int velocidad;
4
5 public Moto() {
6 velocidad=0;
7 }
8 }
```

Añade los siguientes métodos:

- `int getVelocidad()`. Devuelve la velocidad del objeto moto.
- `void acelera(int mas)`. Permite aumentar la velocidad del objeto moto.
- `void frena(int menos)`. Permite reducir la velocidad del objeto moto.

3. (Rebajas) Crea una clase `Rebajas` con un método `descubrePorcentaje()` que descubra el descuento aplicado en un producto. El método recibe el precio original del producto y el rebajado y devuelve el porcentaje aplicado. Podemos calcular el descuento realizando la operación:

$$porcentajeDescuento = \frac{precioOriginal - precioRebajado}{precioOriginal}$$

4. (Finanzas) Realiza una clase `Finanzas` que convierta dólares a euros y viceversa. Codifica los métodos `dolaresToEuros` y `eurosToDolares`. Prueba que dicha clase funciona correctamente haciendo conversiones entre euros y dólares. La clase tiene que tener:

- Un constructor `Finanzas()` por defecto el cual establece el cambio Dólar-Euro en 1.36.
- Un constructor `Finanzas(double cambio)`, el cual permitirá configurar el cambio Dólar-euro a una cantidad personalizada.

5. (MiNumero) Realiza una clase `MiNumero` que proporcione el doble, triple y cuádruple de un número proporcionado en su constructor (realiza un método para `doble`, otro para `triple` y otro para `cuadruple`).

(Opcional, no hay puntos) Haz que la clase tenga un método `main` y comprueba los distintos métodos.

6. (Numero) Realiza una clase `Numero` que almacene un número entero y tenga las siguientes características:

- Constructor por defecto que inicializa a 0 el número interno.
- Constructor que inicializa el número interno.
- Método `anade` que permite sumarle un número al valor interno.
- Método `resta` que resta un número al valor interno.
- Método `getValor`. Devuelve el valor interno.
- Método `getDoble`. Devuelve el doble del valor interno.
- Método `getTriple`. Devuelve el triple del valor interno.
- Método `setNumero`. Inicializa de nuevo el valor interno.

7. (Peso) Crea la clase `Peso`, la cual tendrá las siguientes características:

- Deberá tener un atributo donde se almacene el peso de un objeto en kilogramos. En el constructor se le pasará el peso y la medida en la que se ha tomado ("Lb" para libras, "Li" para lingotes, "Oz" para onzas, "P" para peniques, "K" para kilos, "G" para gramos y "Q" para quintales).

- Deberá de tener los siguientes métodos:
- `getLibras`. Devuelve el peso en libras.
- `getLingotes`. Devuelve el peso en lingotes.
- `getPeso`. Devuelve el peso en la medida que se pase como parámetro ("Lb" para libras, "Li" para lingotes, "Oz" para onzas, "P" para peniques, "K" para kilos, "G" para gramos y "Q" para quintales).
- Para la realización del ejercicio toma como referencia los siguientes datos:
- 1 Libra = 16 onzas = 453 gramos.
- 1 Lingote = 32,17 libras = 14,59 kg.
- 1 Onza = 0,0625 libras = 28,35 gramos.
- 1 Penique = 0,05 onzas = 1,55 gramos.
- 1 Quintal = 100 libras = 43,3 kg.

(Opcional, no hay puntos) Crea además un método `main` para testear y verificar los métodos de esta clase.

8. (Millas) Crea una clase con un método `millasAMetros()` que toma como parámetro de entrada un valor en millas marinas y las convierte a metros. Una vez tengas este método escribe otro `millasAKilometros()` que realice la misma conversión, pero esta vez exprese el resultado en kilómetros.

Nota: 1 milla marina equivale a 1852 metros.

9. (Coche) Crea la clase `Coche` con dos constructores. Uno no toma parámetros y el otro sí. Los dos constructores inicializarán los atributos `marca` y `modelo` de la clase. El constructor por defecto (sin parametros) crea el coche "Ford" modelo "C-MAX".

(Opcional, no hay puntos) Crea dos objetos (cada objeto llama a un constructor distinto) y verifica que todo funciona correctamente.

10. (Consumo) Implementa una clase `Consumo`, la cual forma parte del "ordenador de a bordo" de un coche y tiene las siguientes características:

- Atributos:
  - `kilometros`.
  - `litros`. Litros de combustible consumido.
  - `vmed`. Velocidad media.
  - `pgas`. Precio de la gasolina.
- Métodos:
  - `getTiempo`. Indicará el tiempo empleado en realizar el viaje.
  - `consumoMedio`. Consumo medio del vehículo (en litros cada 100 kilómetros).
  - `consumoEuros`. Consumo medio del vehículo (en euros cada 100 kilómetros).

No olvides crear un constructor para la clase que establezca el valor de los atributos. Elige el tipo de datos más apropiado para cada atributo.

11. (ConsumoModificadores) Para la clase anterior implementa los siguientes métodos, los cuales podrán modificar los valores de los atributos de la clase:

- `setKilometros`
- `setLitros`
- `setVmed`
- `setPgas`

12. (Restaurante) Un restaurante cuya especialidad son las patatas con carne nos pide diseñar un método (`calcularClientes`) con el que se pueda saber cuántos clientes pueden atender con la materia prima que tienen en el almacén. El método recibe la cantidad de patatas y carne en kilos y devuelve el número de clientes que puede atender el restaurante teniendo en cuenta que por cada tres personas, utilizan un dos kilos de patatas y un kilo de carne.

13. (RestauranteClase) Modifica el programa anterior creando una clase que permita almacenar los kilos de patatas y carne del restaurante. Implementa los siguientes métodos:

- `public void addCarne(int x)`. Añade x kilos de carne a los ya existentes.
- `public void addPatatas(int x)`. Añade x kilos de patatas a los ya existentes.
- `public int getComensales()`. Devuelve el número de clientes que puede atender el restaurante (este es el método del ejercicio anterior).
- `public double getCarne()`. Devuelve los kilos de carne que hay en el almacén.
- `public double getPatatas()`. Devuelve los kilos de patatas que hay en el almacén.

14. (Proveedor) Crear un clase llamada `Proveedor` con las siguientes propiedades:

- CIF
- nombreEmpresa
- descripcion
- sector
- direccion
- telefono
- poblacion
- codPostal
- correo

Crear para la clase `Proveedor` los métodos:

- Constructor que permite crear una instancia con los datos de un proveedor.
- Métodos get (*getters*).
- Métodos set (*setters*).
- Método `verificaCorreo` que devuelve true si la dirección de correo contiene @.
- Método que muestre todos los datos del proveedor.

Crear un método principal `main` ejecutable que:

- Cree una instancia del objeto `Proveedor` llamado `proveedor`.
- Cambie el sector del `proveedor`.
- Muestre el sector del `proveedor`.
- Verifique si el correo es válido.
- Muestre todos los datos del `proveedor`.

15. (Producto) Crear una clase llamada `Producto` con las siguientes propiedades:

- codProducto
- nombreProducto
- descripcion
- categoria
- peso
- precio
- stock

Crear para la clase `Producto` los siguiente métodos:

- `Producto` : Permite crear una instancia con los datos de un producto.
- Getters y Setters para todas las propiedades.
- `aumentaStock` : Permite aumentar el stock de unidades del producto. Se le pasa el dato de unidades que aumentamos.
- `disminuyeStock` : Permite disminuir el stock de unidades del producto. Se le pasa el dato de unidades que disminuimos.
- `ivaProducto` : Permite calcular el IVA aplicado al precio del producto. Se le pasa el dato del porcentaje de IVA.
- `mostrarDatos` : Muestra los datos del producto. (No tiene test)

Crear un método principal `main` ejecutable que:

- Crear dos instancias de la clase `Producto` llamadas `productoHardware` y `productoSoftware`.
- Mostrar los datos de los dos objetos `Producto` que hemos creado.
- Aumenta el stock de unidades del `productoHardware` en 12 unidades.
- Disminuir el stock de unidades del `productoSoftware` en 5 unidades.
- Calcula el IVA de los dos objetos `Producto` que hemos creado.
- Mostrar los datos de los dos objetos `Producto`, así como sus importes de IVA y los precios finales de cada una de las instancias.

16. (Cuenta) Crea una clase llamada `Cuenta` que tendrá los siguientes atributos: `titular` y `cantidad` (puede tener decimales).

Al crear una instancia del objeto `Cuenta`, el titular será obligatorio y la cantidad es opcional. Crea dos constructores que cumplan lo anterior, es decir debemos crear dos métodos constructores con el mismo nombre que será el nombre del objeto.

Crea sus métodos get, set y el método `mostrarDatos` que muestre los datos de la cuenta. Tendrá dos métodos especiales:

- `ingresar(double cantidad)` : se ingresa una cantidad a la cuenta, si la cantidad introducida es negativa, no se hará nada.

- `retirar(double cantidad)`: se retira una cantidad a la cuenta, si restando la cantidad actual a la que nos pasan es negativa, la cantidad de la cuenta pasa a ser 0 retirando el importe máximo en función de la cantidad disponible en el objeto.

Crear un método principal `main` ejecutable:

- Crear una instancia del objeto Cuenta llamada `cuentaParticular1` con el nombre del titular.
- Crear una instancia del objeto Cuenta llamada `cuentaEmpresa1` con el nombre del titular y una cantidad inicial de dinero.
- Mostrar el titular de la instancia `cuentaParticular1`.
- Mostrar el saldo de la instancia `cuentaEmpresa1`.
- Ingresar 1000 € en la instancia `cuentaParticular1`.
- Retirar 500 € en la instancia `cuentaEmpresa1`.
- Mostrar los datos de las dos instancias del objeto `Cuenta`.

17. (Libro) Crea una clase llamada `Libro` que guarde la información de cada uno de los libros de una biblioteca. La clase debe guardar las siguientes propiedades:

- `título`
- `autor`
- `editorial`
- `número de ejemplares totales`
- `número de prestados`

La clase contendrá los siguientes métodos:

- Constructor por defecto.
- Constructor con parámetros.
- Métodos Setters/getters.
- Método `prestamo` que incremente el atributo correspondiente cada vez que se realice un préstamo del libro. No se podrán prestar libros de los que no queden ejemplares disponibles para prestar. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
- Método `devolucion` que decremente el atributo correspondiente cuando se produzca la devolución de un libro. No se podrán devolver libros que no se hayan prestado. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
- Método `perdido` que decremente el atributo número de ejemplares por pérdida de ejemplar. No se podrán perder libros que no tengan ejemplares o no se hayan prestado. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
- Método `mostrarDatos` para mostrar los datos de los libros.

Crear un método principal `main` ejecutable:

- Crear una instancia del objeto libro `libroInformatica1` con los datos de un libro.
- Consultar el título de la instancia `libroInformatica1`.
- Cambiar la editorial de la instancia `libroInformatica1` por Anaya.
- Realiza el préstamo de la instancia `libroInformatica1`.
- Realiza otro préstamo de la instancia `libroInformatica1`.
- Muestra los prestamos de la instancia `libroInformatica1`.
- Realiza la devolución de la instancia `libroInformatica1`.
- Muestra los prestamos de la instancia `libroInformatica1`.
- Gestiona la pérdida de un ejemplar de la instancia `libroInformatica1`.
- Muestra los ejemplares de la instancia `libroInformatica1`.
- Muestra todos los datos de la instancia `libroInformatica1`.

18. (Hospital) Crear una clase llamada `Hospital` con las siguientes propiedades y métodos:

- Propiedades:
  - `codHospital`
  - `nombreHospital`
  - `direccion`
  - `telefono`
  - `poblacion`

- `codPostal`
- `habitacionesTotales`
- `habitacionesOcupadas`
- Métodos:
  - `Hospital`: Permite crear una instancia con los datos de un hospital.
  - Métodos `get`.
  - Métodos `set`.
  - Método `ingreso` que incrementa las habitaciones ocupadas. No puede realizarse el ingreso si las habitaciones ocupadas son iguales a las habitaciones totales del hospital. Devuelve `true` si se ha podido realizar el ingreso.
  - Método `alta` que decrementa las habitaciones ocupadas. No puede realizarse el alta si las habitaciones ocupadas son 0. Devuelve `true` si se ha podido realizar el alta.
  - Método que muestre todos los datos del hospital.
- Crear un método principal `main` ejecutable que:
  - Cree una instancia de la clase `Hospital` llamada `hospitalRibera`.
  - Cambie el número de habitaciones de la instancia `hospitalRibera`.
  - Muestre el número de habitaciones de la instancia `hospitalRibera`.
  - Realiza un ingreso de la instancia `hospitalRibera`.
  - Muestra las habitaciones ocupadas de la instancia `hospitalRibera`.
  - Realiza un alta de la instancia `hospitalRibera`.
  - Muestra las habitaciones ocupadas de la instancia `hospitalRibera`.
  - Muestre todos los datos de la instancia `hospitalRibera`.

19. (Medico) Crear un clase llamada `Medico` con las siguientes propiedades y métodos:

- Propiedades:
  - `codMedico`
  - `nombre`
  - `apellidos`
  - `dni`
  - `direccion`
  - `telefono`
  - `poblacion`
  - `codPostal`
  - `fechaNacimiento`
  - `especialidad`
  - `sueldo`
- Métodos:
  - `Medico`: Permite crear una instancia con los datos de un médico.
  - Métodos `get`. Recuperan datos de la instancia del objeto.
  - Métodos `set`. Asignan datos a la instancia del objeto.
  - `retencionMedico`: Permite calcular la retención aplicada al sueldo del médico. Se le pasa el dato del porcentaje de retención.
  - `mostrarDatos`: Muestra los datos del médico.
- Crear un método principal `main` ejecutable que:
  - Crear dos instancias de la clase `Medico` llamados `medicoDigestivo` y `medicoTraumatologo`.
  - Cambia el sueldo del `medicoTraumatologo`.
  - Muestra el sueldo del `medicoTraumatologo`.
  - Cambia el dni del `medicoDigestivo`.
  - Muestra el dni del `medicoDigestivo`.
  - Calcula la retención de las dos instancias de la clase `Medico` que hemos creado.
  - Mostrar los datos de las dos instancias de la clase `Medico` que hemos creado, así como las retenciones y los sueldos finales de cada una.